# Combinatorial Optimization

## Catalogue of combinatorial problems

The course seen problem by problem: for each problem defined in the course, its definition, the chapters that treat it, its models and bounds, what each technique of the course does on it, and the instances of the examples and exercises, with their code and optimal value.

**Fabio Furini**

fabiofurini.github.io

Department of Computer, Control and Management Engineering — Sapienza University of Rome

Online version and downloadable material:
fabiofurini.github.io/combinatorial-optimization/
github.com/fabiofurini/combinatorial-optimization

Academic year 2026–2027

# Contents

1 The problems at a glance 4
- 1.1 The central problems . . . 4
- 1.2 All the problems of the course . . . 5

2 The knapsack problem 7
- 2.1 Definition . . . 8
- 2.2 Bounds and complexity . . . 9
- 2.3 Formulation . . . 9
- 2.4 Dynamic programming . . . 10
- 2.5 Constructive heuristics . . . 11
- 2.6 Local search . . . 11
- 2.7 Metaheuristics . . . 11
- 2.8 MIP heuristics . . . 11
- 2.9 Approximation . . . 12
- 2.10 Branch-and-bound . . . 12
- 2.11 Instances . . . 12
- 2.12 Exercises . . . 26
- 2.13 Subset sum problem (SSP) . . . 28
- 2.14 Bounded knapsack problem (BKP) . . . 29
- 2.15 Unbounded knapsack problem (UKP) . . . 32
- 2.16 Multidimensional knapsack problem (MDKP) . . . 35
- 2.17 Multiple knapsack problem (MKP) . . . 36
- 2.18 Multiple-choice knapsack problem (MCKP) . . . 37
- 2.19 Knapsack problem with setups (KPS) . . . 39

3 The bin packing problem 40
- 3.1 Definition . . . 41
- 3.2 Bounds and complexity . . . 41
- 3.3 Formulation . . . 42
- 3.4 Constructive heuristics . . . 43
- 3.5 Local search . . . 43
- 3.6 Metaheuristics . . . 43
- 3.7 MIP heuristics . . . 44

3.8 Approximation . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . 44
3.9 Instances . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . 44
3.10 Exercises . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . 54
3.11 Cutting stock problem (CSP) . . . . . . . . . . . . . . . . . . . . . . . 55

**4 The vertex coloring problem 57**
4.1 Definition . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . 57
4.2 Bounds and complexity . . . . . . . . . . . . . . . . . . . . . . . . . . 58
4.3 Formulation . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . 59
4.4 Constructive heuristics . . . . . . . . . . . . . . . . . . . . . . . . . . 59
4.5 Approximation . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . 60
4.6 Instances . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . 60
4.7 Exercises . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . 64
4.8 3-colorability (3-COL) . . . . . . . . . . . . . . . . . . . . . . . . . . 65

**5 The maximum clique problem 66**
5.1 Definition . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . 66
5.2 Bounds and complexity . . . . . . . . . . . . . . . . . . . . . . . . . . 67
5.3 Formulation . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . 67
5.4 Constructive heuristics . . . . . . . . . . . . . . . . . . . . . . . . . . 68
5.5 Instances . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . 68
5.6 Exercises . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . 70
5.7 Maximum stable set problem (MSSP) . . . . . . . . . . . . . . . . . . . 70
5.8 Vertex-weighted maximum clique problem (VWMCP) . . . . . . . . . . . 72
5.9 Edge-weighted maximum clique problem (EWMCP) . . . . . . . . . . . 72

**6 The travelling salesman problem 74**
6.1 Definition . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . 75
6.2 Bounds and complexity . . . . . . . . . . . . . . . . . . . . . . . . . . 75
6.3 Formulation . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . 76
6.4 Constructive heuristics . . . . . . . . . . . . . . . . . . . . . . . . . . 77
6.5 Local search . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . 77
6.6 Metaheuristics . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . 77
6.7 MIP heuristics . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . 78
6.8 Approximation . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . 78
6.9 Instances . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . 78
6.10 Exercises . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . . 92
6.11 Metric travelling salesman problem (metric TSP) . . . . . . . . . . . . 93
6.12 Asymmetric travelling salesman problem (ATSP) . . . . . . . . . . . . 94
6.13 Hamiltonian cycle problem (HCP) . . . . . . . . . . . . . . . . . . . . 95

**7 Scheduling on identical parallel machines 96**
7.1 Definition . . . 97
7.2 Bounds and complexity . . . 97
7.3 Formulation . . . 98
7.4 Constructive heuristics . . . 98
7.5 Local search . . . 99
7.6 Metaheuristics . . . 99
7.7 MIP heuristics . . . 99
7.8 Approximation . . . 99
7.9 Instances . . . 100
7.10 Exercises . . . 107
7.11 Scheduling on a fixed number of identical machines ($m$ not part of the input) ($Pm||C_{\max}$) . . . 108
7.12 Scheduling on parallel machines with different speeds, $Q\,||\,C_{\max}$ . . . 109
7.13 Scheduling on unrelated parallel machines, $R\,||\,C_{\max}$ . . . 109

**8 Capacitated lot sizing 110**
8.1 Definition . . . 110
8.2 Bounds and complexity . . . 111
8.3 Formulation . . . 112
8.4 Dynamic programming . . . 113
8.5 Constructive heuristics . . . 113
8.6 MIP heuristics . . . 113
8.7 Instances . . . 113
8.8 Exercises . . . 116
8.9 Uncapacitated lot sizing problem (ULS) . . . 117
8.10 Capacitated lot sizing with constant capacity (CLS-C) . . . 120
8.11 Lot sizing with several products and setup times (MLS) . . . 120
8.12 Lot sizing with initial stock (LS-SI) . . . 121
8.13 Lot sizing with overtime (LS-OT) . . . 122
8.14 Lot sizing with unsatisfied demand (LS-UD) . . . 122

**9 Knapsack family 123**
9.1 Rod-Cutting Problem . . . 123
9.2 Coin Change Problem . . . 126

**10 DP examples 129**
10.1 Minimum-cost staircase . . . 129
10.2 Billboards with no two consecutive positions . . . 130
10.3 Maximum-profit path on a grid . . . 131
10.4 Shortest path in a layered network . . . 132
10.5 Compatible requests . . . 133

## Chapter 1

# The problems at a glance

The course seen problem by problem: for each problem defined in the course, its definition, the chapters that treat it, its models and bounds, what each technique of the course gives on it, and the instances of the examples and exercises with their optimal value.

## 1.1 The central problems

| **Problem** | **Objective** | **Complexity** | **Simple bound** | **Approximation** |
|---|---|---|---|---|
| KP | max total profit, total weight $\leq c$ | weakly NP-hard, DP in $\Theta(nc)$ | Dantzig: $\mathrm{UB}_\mathrm{D} = \sum_{j<s} p_j + \lfloor p_s\, rc/w_s \rfloor$ | Ext-Greedy 2; $H^\ell$: $\frac{\ell+2}{\ell+1}$; FPTAS $\frac{1}{1-\varepsilon}$ |
| BPP | min number of bins | strongly NP-hard | $\mathrm{LB} = \lceil \sum_j w_j/c \rceil$ | NF, WF 2; FF, BF $\frac{17}{10}$; no ratio $< \frac{3}{2}$ |
| VCP | min number of colors $\chi(G)$ | strongly NP-hard | $\omega(G) \leq \chi(G) \leq \Delta(G)+1$ | no ratio $< \frac{4}{3}$; greedy $GC$ no constant ratio |
| MCP | max clique size $\omega(G)$ | NP-hard | $\omega(G) \leq \chi(G)$, $\omega(G) \leq \Delta + 1$ | no $n^{1-\varepsilon}$ guarantee (Håstad) |
| TSP | min tour length $L(\sigma)$ | strongly NP-hard | $\lceil \frac{1}{2} \sum_i (a_i + b_i) \rceil$; MST $d(T) \leq z(I)$ | metric: double tree 2, Christofides $\frac{3}{2}$; general: no constant ratio |
| $P\|\|C_{\max}$ | min makespan $C_{\max}$ | strongly NP-hard ($m$ in the input) | $\max\{\max_j t_j, \lceil \frac{1}{m} \sum_j t_j \rceil\}$ | LIST $2-\frac{1}{m}$; LPT $\frac{4}{3}-\frac{1}{3m}$ |
| CLS | min setup, production and holding cost | NP-hard; ULS in $\Theta(m^2)$ | $k$ smallest setup costs; LP of the shortest-path formulation | MIP heuristics: relax-and-fix, fix-and-optimize |

## 1.2 All the problems of the course

**Central problems**

- The knapsack problem (KP).
- The bin packing problem (BPP).
- The vertex coloring problem (VCP).
- The maximum clique problem (MCP).
- The travelling salesman problem (TSP).
- Scheduling on identical parallel machines ($P\|C_{\max}$).
- Capacitated lot sizing (CLS).

**Knapsack family**

- Subset sum problem (SSP).
- Bounded knapsack problem (BKP).
- Unbounded knapsack problem (UKP).
- Multidimensional knapsack problem (MDKP).
- Multiple knapsack problem (MKP).
- Multiple-choice knapsack problem (MCKP).
- Knapsack problem with setups (KPS).
- Rod-Cutting Problem (RCP).
- Coin Change Problem (CCP).

**Packing**

- Cutting stock problem (CSP).

**Graphs**

- 3-colorability (3-COL).
- Maximum stable set problem (MSSP).
- Vertex-weighted maximum clique problem (VWMCP).
- Edge-weighted maximum clique problem (EWMCP).

**Routing**

- Metric travelling salesman problem (metric TSP).
- Asymmetric travelling salesman problem (ATSP).

- Hamiltonian cycle problem (HCP).

### Scheduling

- Scheduling on a fixed number of identical machines ($m$ not part of the input) ($Pm||C_{\max}$).
- Scheduling on parallel machines with different speeds, $Q\,||\,C_{\max}$.
- Scheduling on unrelated parallel machines, $R\,||\,C_{\max}$.

### Lot sizing

- Uncapacitated lot sizing problem (ULS).
- Capacitated lot sizing with constant capacity (CLS-C).
- Lot sizing with several products and setup times (MLS).
- Lot sizing with initial stock (LS-SI).
- Lot sizing with overtime (LS-OT).
- Lot sizing with unsatisfied demand (LS-UD).

### DP examples

- Minimum-cost staircase (STAIR).
- Billboards with no two consecutive positions (NOCONS).
- Maximum-profit path on a grid (GRID).
- Shortest path in a layered network (SPP).
- Compatible requests (REQ).

### Chapter 2

# The knapsack problem

Choose a subset of items of maximum total profit whose total weight does not exceed the capacity.

**Where it is treated**

- Chapter 1 «Combinatorial optimization problems»: First definition, brute force on a small instance, greedy primal bound and continuous dual bound
- Chapter 2 «Knapsack problems»: Definition, assumptions, counting, cardinality bound and the seven variants
- Chapter 9 «Formulations of the knapsack problem»: ILP model, the two LP relaxations and their duals, split item, Dantzig and Martello–Toth bounds
- Chapter 16 «Principles of dynamic programming»: The KP and the UKP as running examples of dynamic programming
- Chapter 17 «Dynamic programming for knapsack problems»: DP algorithms by capacities, by lists of states and by profits; DP for the UKP and the BKP
- Chapter 20 «Heuristic algorithms: definitions»: KP reference instance of the heuristic part, gap and the flip and swap neighbourhoods
- Chapter 21 «Constructive heuristics»: Greedy algorithms by profit, by weight and by efficiency
- Chapter 22 «Local search»: Flip and swap local search, first and best improvement, penalties on the excess weight
- Chapter 23 «Metaheuristics»: Simulated annealing with a penalty, GRASP, genetic and memetic algorithms, the medium instance KP20
- Chapter 24 «MIP heuristics for the classical problems»: Rounding, diving, RINS, local branching and proximity search on the KP model
- Chapter 25 «Approximation algorithms: definitions»: The greedy algorithm without a constant guarantee, in the exercises

- Chapter 27 «Approximation algorithms for the knapsack problem»: Greedy, Ext-Greedy, $H^2$, $H^\ell$, the PTAS, the Basic-FPTAS and the absence of an absolute guarantee
- Chapter 29 «Knapsack: branch-and-bound on the critical item»: LP-based branch-and-bound with the Dantzig formulas and branching on the split item

## 2.1 Definition

**Definition 2.1: The knapsack problem**

Given:

1. a knapsack of *capacity* $c \in \mathbb{Z}_{\geq 1}$,
2. a set $\mathscr{I} = \{1, 2, \ldots, n\}$ of $n$ items,
3. for each item $j \in \mathscr{I}$, a *profit* $p_j \in \mathbb{Z}_{\geq 1}$ and a *weight* $w_j \in \mathbb{Z}_{\geq 1}$,

the **Knapsack Problem** (KP) asks to decide which items to insert into the knapsack, so that the total weight of the selected items does not exceed the knapsack capacity, and the total profit of the selected items is maximized. A KP feasible solution is a subset $S \subseteq \mathscr{I}$ with $\sum_{j \in S} w_j \leq c$, and

$$\tilde{S} \in \operatorname*{arg\,max}_{S \subseteq \mathscr{I}} \left\{ \sum_{j \in S} p_j \ : \ \sum_{j \in S} w_j \leq c \right\}.$$

- Defined in Chapter 2 «Knapsack problems».
- The KP models the choice of a subset of objects, projects or investments under one budget or capacity constraint.
- A solution is a **subset** of the items, so an instance with $n$ items has $2^n$ candidate solutions (Chapter 1 «Combinatorial optimization problems»).
- The assumptions $w_j \leq c$ for all $j \in \mathscr{I}$ and $\sum_{j \in \mathscr{I}} w_j > c$ remove the items that never fit and the trivial instances (Chapter 2 «Knapsack problems»).

## 2.2 Bounds and complexity

- **Complexity**: Weakly NP-hard: it contains the SSP as the special case $p_j = w_j$, and it is solved by dynamic programming in $\Theta(nc)$ steps, a pseudo-polynomial time (Chapter 1 «Combinatorial optimization problems»).
- Cardinality bound: if the $\bar{k}$ lightest items are the most that fit, $z(I)$ is at most the sum of the $\bar{k}$ largest profits (Chapter 2 «Knapsack problems»).
- $z(\text{LP-0}_{\text{KP}}) = p_1 \frac{c}{w_1}$, with item 1 of largest efficiency (relaxation without $x_j \leq 1$) (Chapter 9 «Formulations of the knapsack problem»).
- $z(\text{LP}_{\text{KP}}) = \sum_{j=1}^{s-1} p_j + p_s \frac{rc}{w_s}$, with the items sorted by non-increasing efficiency (Chapter 9 «Formulations of the knapsack problem»).
- Dantzig bound $\text{UB}_{\text{D}} = \sum_{j=1}^{s-1} p_j + \left\lfloor p_s \frac{rc}{w_s} \right\rfloor$ (Chapter 9 «Formulations of the knapsack problem»).
- Martello–Toth bound $\text{UB}_{\text{MT}} = \max\{\text{UB}_0, \text{UB}_1\}$, with $\text{UB}_0 = \sum_{j=1}^{s-1} p_j + \left\lfloor rc \frac{p_{s+1}}{w_{s+1}} \right\rfloor$ and $\text{UB}_1 = \sum_{j=1}^{s-1} p_j + \left\lfloor p_s - (w_s - rc) \frac{p_{s-1}}{w_{s-1}} \right\rfloor$ (Chapter 9 «Formulations of the knapsack problem»).
- For every instance $z(\text{LP}_{\text{KP}})/z(\text{ILP}_{\text{KP}}) \leq 2$, and there are instances where the ratio tends to 2 as $c \to +\infty$ (Chapter 9 «Formulations of the knapsack problem»).
- The order by efficiency is not enough to find an optimal solution: on the reference instance it gives $\{1, 2\}$ with profit 11, while the optimum is 15 (Chapter 2 «Knapsack problems»).
- In every node of the branch-and-bound tree at most one variable is fractional, the one of the split item (Chapter 29 «Knapsack: branch-and-bound on the critical item»).

## 2.3 Formulation

- **ILP model of the KP** ($\text{ILP}_{\text{KP}}$) (Chapter 9 «Formulations of the knapsack problem»):

$$\begin{aligned} \max \ & \sum_{j \in \mathscr{I}} p_j x_j & \\ & \sum_{j \in \mathscr{I}} w_j x_j \leq c & \\ & x_j \in \{0, 1\} & j \in \mathscr{I} \end{aligned}$$

- **LP relaxation** $\text{LP}_{\text{KP}}$ (Chapter 9 «Formulations of the knapsack problem»):

$$\begin{aligned} \max \ & \sum_{j \in \mathscr{I}} p_j x_j & \\ & \sum_{j \in \mathscr{I}} w_j x_j \leq c & \\ & x_j \leq 1 & j \in \mathscr{I} \\ & x_j \geq 0 & j \in \mathscr{I} \end{aligned}$$

- **Dual** $D(\mathrm{LP}_{\mathrm{KP}})$ (Chapter 9 «Formulations of the knapsack problem»):

$$\begin{aligned}
\min\ & c\,\pi + \sum_{j \in \mathscr{I}} \mu_j \\
& w_j\,\pi + \mu_j \geq p_j && j \in \mathscr{I} \\
& \pi \geq 0 \\
& \mu_j \geq 0 && j \in \mathscr{I}
\end{aligned}$$

- The $\mathrm{ILP}_{\mathrm{KP}}$ model has one binary variable $x_j$ per item, one capacity constraint and the objective of maximizing the total profit (Chapter 9 «Formulations of the knapsack problem»).
- The relaxation $\mathrm{LP}_{\mathrm{KP}}$ is solved by sorting the items by non-increasing efficiency $p_j/w_j$: the items before the split item $s$ are taken, $s$ for the fraction $rc/w_s$ (Chapter 9 «Formulations of the knapsack problem»).
- The dual $D(\mathrm{LP}_{\mathrm{KP}})$ has the optimal solution $\tilde{\pi} = p_s/w_s$, $\tilde{\mu}_j = p_j - w_j\,p_s/w_s$ for $j < s$, and $z(\mathrm{LP}_{\mathrm{KP}}) \leq 2\,z(\mathrm{ILP}_{\mathrm{KP}})$ (Chapter 9 «Formulations of the knapsack problem»).
- Rounding down $z(\mathrm{LP}_{\mathrm{KP}})$ gives the Dantzig bound $\mathrm{UB}_{\mathrm{D}}$; the Martello–Toth bound $\mathrm{UB}_{\mathrm{MT}}$ treats separately the split item out and in (Chapter 9 «Formulations of the knapsack problem»).

## 2.4 Dynamic programming

- The recurrence $v(j,k) = \max\{v(j-1,k),\ p_j + v(j-1,k-w_j)\}$ over items and capacities is filled bottom-up in $\Theta(nc)$ steps, which is pseudo-polynomial (Chapter 17 «Dynamic programming for knapsack problems»).
- The lists $L_j$ of the non-dominated states $(w,p)$ store only the reachable states, with $|L_j| \leq \min\{2^j, c+1\}$ (Chapter 17 «Dynamic programming for knapsack problems»).
- Exchanging profits and weights, $u(j,q)$ is the minimum weight of a subset of profit $q$, and $z = \max\{q : u(n,q) \leq c\}$ is computed in $\Theta(nP)$ steps (Chapter 17 «Dynamic programming for knapsack problems»).
- The UKP is solved by the recurrence $v(k)$ over the capacities and the BKP by $v(j,k)$ over the numbers of copies (Chapter 17 «Dynamic programming for knapsack problems», Chapter 17 «Dynamic programming for knapsack problems»).

## 2.5 Constructive heuristics

- A greedy algorithm for the KP is determined by the order of the items: by profit ($GR_p$), by weight ($GR_w$) or by efficiency ($GR$) (Chapter 21 «Constructive heuristics»).
- On the reference instance $GR_p$ finds the optimum 17, while $GR$ gives 16 and $GR_w$ gives 10 (Chapter 21 «Constructive heuristics»).
- Each of the three orders can be arbitrarily bad, and taking the better of $GR$ and of the best single item guarantees a ratio of 2 (Chapter 21 «Constructive heuristics»).

## 2.6 Local search

- The flip neighbourhood adds or removes one item, the swap neighbourhood exchanges a selected and an unselected item; being a local optimum depends on the neighbourhood (Chapter 20 «Heuristic algorithms: definitions»).
- From the solution of $GR_w$, first and best improvement reach the optimum along different paths, $10 \to 14 \to 17$ and $10 \to 16 \to 17$ (Chapter 22 «Local search»).
- The penalty flip local search maximizes $g_\alpha(x) = \sum_j p_j x_j - \alpha \max\{0, \sum_j w_j x_j - c\}$ and can cross infeasible vectors; with $\alpha > \max_j p_j$ its local optima are feasible (Chapter 22 «Local search»).

## 2.7 Metaheuristics

- Simulated annealing with the penalized objective $f_\beta$ reaches the optimum of the reference instance through an infeasible solution (Chapter 23 «Metaheuristics»).
- GRASP builds randomized greedy solutions with a restricted candidate list on the efficiency and improves each one by local search (Chapter 23 «Metaheuristics»).
- Genetic algorithms use the binary vector as chromosome and repair infeasible children; on KP20 the memetic algorithm reaches the optimum 521 in every run (Chapter 23 «Metaheuristics»).

## 2.8 MIP heuristics

- Rounding down the LP solution is always feasible on the KP, since the model has a single packing constraint (Chapter 24 «MIP heuristics for the classical problems»).
- Fractional diving on the reference instance fixes $x_2 = 0$ at the first step and ends with profit 16, missing the optimum 17 (Chapter 24 «MIP heuristics for the classical problems»).
- RINS fails on the reference instance, while local branching and proximity search reach and prove the optimum $\{1, 2\}$ (Chapter 24 «MIP heuristics for the classical problems», Chapter 24 «MIP heuristics for the classical problems»).

## 2.9 Approximation

- The greedy algorithm $GR$ has no constant guarantee, while Ext-Greedy $GR^e$ is a 2-approximation algorithm and the bound is tight (Chapter 27 «Approximation algorithms for the knapsack problem»).
- $H^2$ guesses the two items of largest profit and guarantees $\frac{4}{3}$; $H^\ell$ guarantees $\frac{\ell+2}{\ell+1}$ in $O(n^{\ell+1})$ time, and $\{H^\varepsilon\}$ is a PTAS (Chapter 27 «Approximation algorithms for the knapsack problem»).
- The Basic-FPTAS rounds down the profits, solves the scaled instance by DP by profits and guarantees $\frac{1}{1-\varepsilon}$ in $O(n^2/\varepsilon)$ time (Chapter 27 «Approximation algorithms for the knapsack problem»).
- No polynomial algorithm with a fixed absolute performance guarantee exists, unless it solves the KP exactly (Chapter 27 «Approximation algorithms for the knapsack problem»).

## 2.10 Branch-and-bound

- The problem $\mathrm{LP}^k_{\mathrm{KP}}$ of a node is a knapsack LP on the remaining items with capacity $c(k)$, solved by the Dantzig formulas without the simplex algorithm (Chapter 29 «Knapsack: branch-and-bound on the critical item»).
- The algorithm branches on the variable of the split item, the only fractional one, and prunes a node when $\lfloor z(\mathrm{LP}^k_{\mathrm{KP}}) \rfloor \leq \mathrm{LB}$ (Chapter 29 «Knapsack: branch-and-bound on the critical item»).
- On the reference instance the algorithm closes the gap between $\mathrm{LB} = 15$ and $\mathrm{UB} = 16$ with 5 nodes (Chapter 29 «Knapsack: branch-and-bound on the critical item»).

## 2.11 Instances

### 2.11.1 Instance KP1

Reference instance of the KP chapters.

Capacity $c = 9$, $n = 6$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| $p_j$ | 6 | 5 | 8 | 9 | 6 | 7 |
| $w_j$ | 2 | 3 | 6 | 7 | 5 | 9 |

- Optimal value $z(\mathrm{KP1}) = 15$, attained by $\tilde{S} = \{1, 4\}$, the unique optimal solution.
- Used in Example «KP instance», Counting, a first solution and the cardinality bound, Example «$\mathrm{ILP}_{\mathrm{KP}}$ model» and the following examples of the chapter, Example «Dantzig and Martello-Toth upper bound», Example «The $\mathrm{ILP}_{\mathrm{KP}}$ model», Example «Branch-and-bound algorithm for the $\mathrm{ILP}_{\mathrm{KP}}$ model», Exercise 9.4, Exercise 29.4.

### 2.11.2 Instance KP2

Reference instance of the heuristic chapters (four items).
Capacity $c = 9$, $n = 4$ items:

| $j$ | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| $p_j$ | 10 | 7 | 6 | 4 |
| $w_j$ | 5 | 4 | 3 | 3 |

- Optimal value $z(\text{KP2}) = 17$, attained by $\tilde{S} = \{1, 2\}$, the only optimal solution.
- Used in Example «a small knapsack instance», Example «brute force on the knapsack instance», Example «bounds on the small instances», Example «KP reference instance», Example «Bounds and gaps on the reference instances», Example «Two neighbourhoods for the KP», Example «Three orders on the KP reference instance», Example «First and best improvement on the KP», Example «Penalties on the KP reference instance», Example «SA with a penalty on the KP reference instance», Example «GRASP on the KP reference instance», Example «GA on the KP reference instance», Example «The LP solutions of the reference instances», Example «Rounding on the reference instances», Example «Fractional diving on the KP», Example «RINS on the KP», Example «Local branching on the KP», Example «Proximity search on the KP», instance $I_0$.

### 2.11.3 Instance KP3

Brute force, greedy and bound on a knapsack.
Capacity $c = 10$, $n = 4$ items:

| $j$ | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| $p_j$ | 7 | 6 | 5 | 4 |
| $w_j$ | 6 | 5 | 4 | 3 |

- Optimal value $z(\text{KP3}) = 12$, attained by $\tilde{S} = \{1, 3\}$.
- Used in Exercise «Brute force, greedy and bound on a knapsack».

### 2.11.4 Instance KP4

A budget of 14 and five projects.
Capacity $c = 14$, $n = 5$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| $p_j$ | 14 | 9 | 15 | 13 | 11 |
| $w_j$ | 7 | 2 | 9 | 6 | 8 |

- Optimal value $z(\text{KP4}) = 27$, attained by $\tilde{S} = \{1, 4\}$, the only optimal solution.
- Used in Exercise 1.2.

### 2.11.5 Instance KP5

Feasibility, value and bounds.
Capacity $c = 12$, $n = 5$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| $p_j$ | 6 | 12 | 5 | 13 | 8 |
| $w_j$ | 3 | 4 | 5 | 6 | 8 |

- Optimal value $z(\text{KP5}) = 25$, attained by $\tilde{S} = \{2, 4\}$, the only optimal solution.
- Used in Exercise 2.1.

### 2.11.6 Instance KP6

KP bound of the two-knapsack instance (capacity $c_1 + c_2$).
Capacity $c = 16$, $n = 5$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| $p_j$ | 4 | 8 | 9 | 10 | 13 |
| $w_j$ | 2 | 5 | 2 | 6 | 3 |

- Optimal value $z(\text{KP6}) = 40$, attained by $\{2, 3, 4, 5\}$.
- Used in question 3.

### 2.11.7 Instance KP7

Upper bounds (1), solved by branch-and-bound.
Capacity $c = 11$, $n = 5$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| $p_j$ | 8 | 6 | 14 | 6 | 2 |
| $w_j$ | 3 | 3 | 9 | 5 | 2 |

- Optimal value $z(\text{KP7}) = 20$, attained by $\tilde{\boldsymbol{x}} = (1, 1, 0, 1, 0)$.
- Used in Exercise 9.1, Exercise 29.1.

### 2.11.8 Instance KP8

Upper bounds (2), solved by branch-and-bound.
Capacity $c = 8$, $n = 4$ items:

| $j$ | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| $p_j$ | 40 | 42 | 25 | 12 |
| $w_j$ | 4 | 7 | 5 | 3 |

- Optimal value $z(\text{KP8}) = 52$, attained by $\tilde{\boldsymbol{x}} = (1, 0, 0, 1)$.
- Used in Exercise 9.2, Exercise 29.2.

### 2.11.9 Instance KP9

Upper bounds (3), solved by branch-and-bound.

Capacity $c = 15$, $n = 4$ items:

| $j$ | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| $p_j$ | 24 | 17 | 12 | 6 |
| $w_j$ | 10 | 8 | 6 | 5 |

- Optimal value $z(\text{KP9}) = 30$, attained by $\tilde{\boldsymbol{x}} = (1, 0, 0, 1)$.
- Used in Exercise 9.3, Exercise 29.3.

### 2.11.10 Instance KP10

Upper bounds (5).

Capacity $c = 21$, $n = 6$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| $p_j$ | 24 | 25 | 13 | 10 | 9 | 2 |
| $w_j$ | 2 | 6 | 8 | 7 | 10 | 3 |

- Optimal value $z(\text{KP10}) = 64$.
- Used in Exercise 9.5.

### 2.11.11 Instance KP11

Small instance of the DP algorithm by capacities.

Capacity $c = 5$, $n = 3$ items:

| $j$ | 1 | 2 | 3 |
|---|---|---|---|
| $p_j$ | 6 | 7 | 8 |
| $w_j$ | 2 | 3 | 3 |

- Optimal value $z(\text{KP11}) = 14$, attained by $(1, 0, 1)$.
- Used in Example «KP instance» and the examples of the first DP algorithm, Example «The lists of states».

### 2.11.12 Instance KP12

Small instance of the DP algorithm by profits.

Capacity $c = 5$, $n = 3$ items:

| $j$ | 1 | 2 | 3 |
|---|---|---|---|
| $p_j$ | 3 | 3 | 4 |
| $w_j$ | 2 | 3 | 3 |

- Optimal value $z(\text{KP12}) = 7$, attained by $(1, 0, 1)$.
- Used in Example «KP instance» and the examples of the second DP algorithm, Example «The lists of states».

### 2.11.13 Instance KP13

DP exercise 1.

Capacity $c = 6$, $n = 4$ items:

| $j$ | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| $p_j$ | 6 | 7 | 8 | 9 |
| $w_j$ | 2 | 3 | 4 | 5 |

- Optimal value $z(\text{KP13}) = 14$, attained by $(1, 0, 1, 0)$.
- Used in Exercise 17.1.

### 2.11.14 Instance KP14

DP exercise 2 (same data as the UKP instance of the DP chapter).

Capacity $c = 7$, $n = 4$ items:

| $j$ | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| $p_j$ | 4 | 5 | 10 | 11 |
| $w_j$ | 2 | 3 | 5 | 6 |

- Optimal value $z(\text{KP14}) = 14$, attained by $(1, 0, 1, 0)$.
- Used in Exercise 17.2.

### 2.11.15 Instance KP15

DP exercise 3.

Capacity $c = 8$, $n = 5$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| $p_j$ | 3 | 4 | 8 | 8 | 10 |
| $w_j$ | 1 | 3 | 4 | 5 | 6 |

- Optimal value $z(\text{KP15}) = 15$, attained by $(1, 1, 1, 0, 0)$.
- Used in Exercise 17.3.

### 2.11.16 Instance KP16

DP exercise 4.

Capacity $c = 9$, $n = 5$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| $p_j$ | 5 | 6 | 9 | 10 | 13 |
| $w_j$ | 2 | 4 | 5 | 6 | 7 |

- Optimal value $z(\text{KP16}) = 18$, attained by $(1, 0, 0, 0, 1)$.
- Used in Exercise 17.4.

### 2.11.17 Instance KP17

DP exercise 5.
Capacity $c = 10$, $n = 5$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| $p_j$ | 2 | 7 | 9 | 12 | 14 |
| $w_j$ | 1 | 4 | 5 | 7 | 8 |

- Optimal value $z(\text{KP17}) = 18$, attained by $(1, 1, 1, 0, 0)$.
- Used in Exercise 17.5.

### 2.11.18 Instance KP18

Gap and relative error.
Capacity $c = 12$, $n = 5$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| $p_j$ | 20 | 18 | 14 | 9 | 6 |
| $w_j$ | 6 | 5 | 4 | 3 | 2 |

- Optimal value $z(\text{KP18}) = 41$, attained by $\{2, 3, 4\}$.
- Used in Exercise 20.1.

### 2.11.19 Instance KP19

Neighbourhoods and local optima.
Capacity $c = 11$, $n = 5$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| $p_j$ | 12 | 10 | 9 | 7 | 4 |
| $w_j$ | 6 | 5 | 4 | 3 | 2 |

- Optimal value $z(\text{KP19}) = 23$, attained by $\{1, 4, 5\}$.
- Used in Exercise 20.4.

### 2.11.20 Instance KP20

Greedy orders 1.
Capacity $c = 9$, $n = 5$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| $p_j$ | 4 | 15 | 11 | 8 | 2 |
| $w_j$ | 3 | 8 | 7 | 2 | 5 |

- Optimal value $z(\text{KP20}) = 19$, attained by $\{3, 4\}$.
- Used in Exercise 21.1.

### 2.11.21 Instance KP21

Greedy orders 2.

Capacity $c = 13$, $n = 5$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| $p_j$ | 9 | 4 | 8 | 6 | 7 |
| $w_j$ | 7 | 2 | 9 | 5 | 6 |

- Optimal value $z(\text{KP21}) = 17$, attained by $\{2, 4, 5\}$.
- Used in Exercise 21.2.

### 2.11.22 Instance KP22

Greedy orders 3.

Capacity $c = 10$, $n = 5$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| $p_j$ | 8 | 13 | 5 | 12 | 7 |
| $w_j$ | 5 | 9 | 4 | 8 | 2 |

- Optimal value $z(\text{KP22}) = 19$, attained by $\{4, 5\}$.
- Used in Exercise 21.3.

### 2.11.23 Instance KP23

Local search KP 1.

Capacity $c = 18$, $n = 5$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| $p_j$ | 10 | 7 | 13 | 14 | 8 |
| $w_j$ | 2 | 3 | 8 | 9 | 7 |

- Optimal value $z(\text{KP23}) = 32$.
- Used in Exercise 22.1.

### 2.11.24 Instance KP24

Local search KP 2.

Capacity $c = 13$, $n = 5$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| $p_j$ | 8 | 9 | 14 | 11 | 9 |
| $w_j$ | 3 | 5 | 9 | 9 | 8 |

- Optimal value $z(\text{KP24}) = 22$, attained by $x = (1, 0, 1, 0, 0)$.
- Used in Exercise 22.2.

### 2.11.25 Instance KP25

Local search KP 3 (penalties).
Capacity $c = 11$, $n = 5$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| $p_j$ | 14 | 7 | 10 | 8 | 6 |
| $w_j$ | 4 | 4 | 7 | 7 | 9 |

- Optimal value $z(\text{KP25}) = 24$, attained by $x = (1, 0, 1, 0, 0)$.
- Used in Exercise 22.3.

### 2.11.26 Instance KP26

Instance of the metaheuristic exercises.
Capacity $c = 16$, $n = 5$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| $p_j$ | 11 | 5 | 18 | 8 | 8 |
| $w_j$ | 5 | 3 | 11 | 6 | 7 |

- Optimal value $z(\text{KP26}) = 29$, attained by $\{1, 3\}$, which fills the capacity exactly.
- Used in Exercise 23.2, Exercise 23.10, Exercise 23.12, Exercise 23.16, Exercise 23.17.

### 2.11.27 Instance KP27

Medium instance KP20 ($c$ is half of the total weight, rounded down).
Capacity $c = 389$, $n = 20$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| $p_j$ | 54 | 55 | 55 | 52 | 58 | 42 | 53 | 53 | 22 | 64 | 54 | 51 |
| $w_j$ | 39 | 43 | 49 | 47 | 44 | 35 | 43 | 44 | 16 | 50 | 40 | 42 |

| $j$ | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 |
|---|---|---|---|---|---|---|---|---|
| $p_j$ | 39 | 49 | 53 | 46 | 28 | 64 | 58 | 34 |
| $w_j$ | 28 | 38 | 46 | 39 | 14 | 50 | 45 | 27 |

- Optimal value $z(\text{KP27}) = 521$, attained by greedy by efficiency 490, greedy plus local search 514.
- Used in Example «The medium instances of the KP, the BPP and $P\,||\,C_{\max}$», GRASP experiment, GA and memetic experiment, Comparison of the methods.

### 2.11.28 Instance KP28

Rounding and diving (KP1).
Capacity $c = 16$, $n = 5$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| $p_j$ | 11 | 15 | 18 | 14 | 10 |
| $w_j$ | 3 | 5 | 7 | 10 | 9 |

- Optimal value $z(\text{KP28}) = 44$.
- Used in Exercise 24.1.

### 2.11.29 Instance KP29

Rounding and diving (KP2).
Capacity $c = 15$, $n = 5$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| $p_j$ | 13 | 12 | 14 | 6 | 5 |
| $w_j$ | 2 | 5 | 10 | 6 | 7 |

- Optimal value $z(\text{KP29}) = 31$, attained by $\{1, 2, 4\}$.
- Used in Exercise 24.2.

### 2.11.30 Instance KP30

RINS, local branching and proximity search (KP3).
Capacity $c = 22$, $n = 6$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| $p_j$ | 18 | 21 | 4 | 19 | 10 | 7 |
| $w_j$ | 8 | 10 | 2 | 12 | 7 | 10 |

- Optimal value $z(\text{KP30}) = 43$, attained by $(1, 1, 1, 0, 0, 0)$.
- Used in Exercise 24.3.

### 2.11.31 Instance KP31

RINS, local branching and proximity search (KP4).
Capacity $c = 13$, $n = 6$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| $p_j$ | 16 | 13 | 8 | 16 | 10 | 9 |
| $w_j$ | 2 | 2 | 4 | 9 | 6 | 8 |

- Optimal value $z(\text{KP31}) = 45$.
- Used in Exercise 24.4.

### 2.11.32 Instance KP32

Reference instance of the approximation chapter (the KP reference instance with a seventh item).
Capacity $c = 9$, $n = 7$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|
| $p_j$ | 6 | 5 | 8 | 9 | 6 | 7 | 3 |
| $w_j$ | 2 | 3 | 6 | 7 | 5 | 9 | 4 |

- Optimal value $z(\text{KP32}) = 15$, attained by $\{1, 4\}$ (found by $H^2$, $H^3$ and $FP^{\varepsilon}$).
- Used in Example «split item and residual capacity», Example «Execution of $GR$», Example «Execution of $GR^e$», Dashboard of the chapter.

### 2.11.33 Instance KP33

The seven items of the approximation chapter with capacity 12.
Capacity $c = 12$, $n = 7$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|
| $p_j$ | 6 | 5 | 8 | 9 | 6 | 7 | 3 |
| $w_j$ | 2 | 3 | 6 | 7 | 5 | 9 | 4 |

- Optimal value $z(\text{KP33}) = 20$, attained by $\tilde{S} = \{1, 2, 4\}$.
- Used in Example «$H^2$ algorithm».

### 2.11.34 Instance KP34

Greedy against the item of largest profit.
Capacity $c = 10$, $n = 4$ items:

| $j$ | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| $p_j$ | 6 | 8 | 19 | 4 |
| $w_j$ | 2 | 3 | 9 | 5 |

- Optimal value $z(\text{KP34}) = 19$, attained by the notes state that item 3 alone is optimal, i.e. $z(I) = 19$.
- Used in Example «Execution of $GR$», Example «Execution of $GR^e$».

### 2.11.35 Instance KP35

Greedy selects items after the split item.
Capacity $c = 10$, $n = 5$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| $p_j$ | 4 | 6 | 5 | 12 | 3 |
| $w_j$ | 1 | 2 | 2 | 6 | 2 |

- Optimal value $z(\text{KP35}) = 23$, attained by $\tilde{S} = \{2, 3, 4\}$.
- Used in Example «Execution of $GR$».

### 2.11.36 Instance KP36

The most profitable item comes last.
Capacity $c = 21$, $n = 6$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| $p_j$ | 4 | 6 | 4 | 3 | 2 | 20 |
| $w_j$ | 1 | 2 | 2 | 2 | 2 | 21 |

- Optimal value $z(\text{KP36}) = 20$, attained by $\tilde{S} = \{6\}$.
- Used in Example «Execution of $GR$».

### 2.11.37 Instance KP37

$H^2$ fails, $H^3$ finds the optimum.
Capacity $c = 12$, $n = 5$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| $p_j$ | 2 | 3 | 3 | 3 | 3 |
| $w_j$ | 1 | 3 | 3 | 3 | 3 |

- Optimal value $z(\text{KP37}) = 12$, attained by the four items of weight 3.
- Used in Example «Execution of $H^\ell$».

### 2.11.38 Instance KP38

Basic-FPTAS with $\varepsilon = \frac{1}{2}$.
Capacity $c = 8$, $n = 4$ items:

| $j$ | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| $p_j$ | 19 | 19 | 14 | 14 |
| $w_j$ | 2 | 4 | 3 | 3 |

- Optimal value $z(\text{KP38}) = 47$, attained by $\tilde{S} = \{1, 3, 4\}$.
- Used in Example «Execution of $FP^\varepsilon$».

### 2.11.39 Instance KP39

Greedy 1.
Capacity $c = 15$, $n = 5$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| $p_j$ | 12 | 6 | 6 | 8 | 6 |
| $w_j$ | 3 | 2 | 3 | 9 | 7 |

- Optimal value $z(\text{KP39}) = 30$.
- Used in Exercise 27.1.

### 2.11.40 Instance KP40

Greedy 2.
Capacity $c = 13$, $n = 6$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| $p_j$ | 10 | 6 | 3 | 7 | 11 | 2 |
| $w_j$ | 1 | 1 | 1 | 4 | 8 | 3 |

- Optimal value $z(\text{KP40}) = 30$, attained by $\{1, 2, 3, 5\}$.
- Used in Exercise 27.2.

### 2.11.41 Instance KP41

Greedy 3.
Capacity $c = 12$, $n = 5$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| $p_j$ | 8 | 7 | 18 | 14 | 10 |
| $w_j$ | 3 | 3 | 9 | 8 | 7 |

- Optimal value $z(\text{KP41}) = 26$, attained by $\{1, 3\}$.
- Used in Exercise 27.3.

### 2.11.42 Instance KP42

Ext-Greedy 1.
Capacity $c = 22$, $n = 5$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| $p_j$ | 11 | 5 | 3 | 4 | 3 |
| $w_j$ | 8 | 4 | 3 | 9 | 8 |

- Optimal value $z(\text{KP42}) = 20$, attained by $\{1, 2, 4\}$.
- Used in Exercise 27.4.

### 2.11.43 Instance KP43

Ext-Greedy 2.
Capacity $c = 8$, $n = 5$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| $p_j$ | 16 | 19 | 18 | 14 | 8 |
| $w_j$ | 5 | 8 | 8 | 7 | 6 |

- Optimal value $z(\text{KP43}) = 19$, attained by $\{2\}$.
- Used in Exercise 27.5.

### 2.11.44 Instance KP44

Ext-Greedy 3.
Capacity $c = 16$, $n = 3$ items:

| $j$ | 1 | 2 | 3 |
|---|---|---|---|
| $p_j$ | 2 | 8 | 8 |
| $w_j$ | 1 | 8 | 8 |

- Optimal value $z(\text{KP44}) = 16$, attained by $\{2, 3\}$.
- Used in Exercise 27.6.

### 2.11.45 Instance KP45

$H^2$ 1.

Capacity $c = 11$, $n = 5$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| $p_j$ | 8 | 9 | 2 | 3 | 2 |
| $w_j$ | 5 | 8 | 3 | 7 | 9 |

- Optimal value $z(\text{KP45}) = 11$, attained by $\{2, 3\}$.
- Used in Exercise 27.7.

### 2.11.46 Instance KP46

$H^2$ 2.

Capacity $c = 15$, $n = 5$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| $p_j$ | 14 | 9 | 5 | 10 | 15 |
| $w_j$ | 1 | 1 | 2 | 5 | 8 |

- Optimal value $z(\text{KP46}) = 48$, attained by $\{1, 2, 4, 5\}$.
- Used in Exercise 27.8.

### 2.11.47 Instance KP47

$H^2$ 3.

Capacity $c = 10$, $n = 5$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| $p_j$ | 13 | 5 | 8 | 5 | 7 |
| $w_j$ | 1 | 1 | 3 | 2 | 5 |

- Optimal value $z(\text{KP47}) = 33$, attained by $\{1, 2, 3, 5\}$.
- Used in Exercise 27.9.

### 2.11.48 Instance KP48

$H^\ell$ 1.

Capacity $c = 16$, $n = 5$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| $p_j$ | 14 | 13 | 13 | 10 | 11 |
| $w_j$ | 1 | 2 | 3 | 3 | 9 |

- Optimal value $z(\text{KP48}) = 51$, attained by $\{1, 2, 3, 5\}$.
- Used in Exercise 27.10.

### 2.11.49 Instance KP49

$H^\ell$ 2.

Capacity $c = 20$, $n = 6$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| $p_j$ | 2 | 4 | 4 | 4 | 4 | 4 |
| $w_j$ | 1 | 4 | 4 | 4 | 4 | 4 |

- Optimal value $z(\text{KP49}) = 20$, attained by $\{2, 3, 4, 5, 6\}$.
- Used in Exercise 27.11.

### 2.11.50 Instance KP50

$H^\ell$ 3.

Capacity $c = 18$, $n = 6$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| $p_j$ | 15 | 15 | 14 | 8 | 12 | 13 |
| $w_j$ | 1 | 2 | 3 | 2 | 6 | 9 |

- Optimal value $z(\text{KP50}) = 65$, attained by $\{1, 2, 3, 4, 6\}$.
- Used in Exercise 27.12.

### 2.11.51 Instance KP51

Basic-FPTAS 1.

Capacity $c = 13$, $n = 4$ items:

| $j$ | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| $p_j$ | 24 | 11 | 23 | 5 |
| $w_j$ | 2 | 3 | 9 | 3 |

- Optimal value $z(\text{KP51}) = 47$, attained by $\{1, 3\}$.
- Used in Exercise 27.13.

### 2.11.52 Instance KP52

Basic-FPTAS 2 and 3.

Capacity $c = 16$, $n = 4$ items:

| $j$ | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| $p_j$ | 14 | 12 | 14 | 18 |
| $w_j$ | 3 | 4 | 5 | 7 |

- Optimal value $z(\text{KP52}) = 46$, attained by $\{1, 3, 4\}$.
- Used in Exercise 27.14, Exercise 27.15.

## 2.12 Exercises

- Exercise «Brute force, greedy and bound on a knapsack» (in the chapter)
- «A greedy choice that is not optimal (KP)»
- «Counting the solutions» (part (a))
- «Feasibility, value and bounds (KP)»
- «Subset sum and its solution as a KP (SSP)»
- «Copies of the items (BKP and UKP)»
- «Two knapsacks (MKP)»
- «One item per class (MCKP)»
- «Upper bounds for the Knapsack Problem (1)»
- «Upper bounds for the Knapsack Problem (2)»
- «Upper bounds for the Knapsack Problem (3)»
- «Upper bounds for the Knapsack Problem (4)»
- «Upper bounds for the Knapsack Problem (5)»
- «Knapsack Problem» (DP 1)
- «Knapsack Problem» (DP 2)
- «Knapsack Problem» (DP 3)
- «Knapsack Problem» (DP 4)
- «Knapsack Problem» (DP 5)
- «Unbounded Knapsack Problem» (DP 1)
- «Unbounded Knapsack Problem» (DP 2)
- «Unbounded Knapsack Problem» (DP 3)
- «Unbounded Knapsack Problem» (DP 4)
- «Unbounded Knapsack Problem» (DP 5)
- «Bounded Knapsack Problem» (DP 1)
- «Bounded Knapsack Problem» (DP 2)
- «Bounded Knapsack Problem» (DP 3)
- «Bounded Knapsack Problem» (DP 4)
- «Bounded Knapsack Problem» (DP 5)

- «Gap and relative error»
- «Neighbourhoods and local optima»
- «Greedy orders for the KP 1»
- «Greedy orders for the KP 2»
- «Greedy orders for the KP 3»
- «Local search KP 1»
- «Local search KP 2»
- «Local search KP 3»
- «Simulated annealing 2» (KP)
- «GRASP 1» (KP)
- «GRASP 3» (KP)
- «Genetic algorithms 1» (KP)
- «Genetic algorithms 2» (KP)
- «Rounding and diving on the KP (KP1)»
- «Rounding and diving on the KP (KP2)»
- «RINS, local branching and proximity search on the KP (KP3)»
- «RINS, local branching and proximity search on the KP (KP4)»
- «Unbounded ratio of a greedy algorithm»
- «Greedy 1»
- «Greedy 2»
- «Greedy 3»
- «Ext-Greedy 1»
- «Ext-Greedy 2»
- «Ext-Greedy 3»
- «$H^2$ 1»
- «$H^2$ 2»
- «$H^2$ 3»
- «$H^\ell$ 1»
- «$H^\ell$ 2»

- «$H^\ell$ 3»
- «Basic-FPTAS 1»
- «Basic-FPTAS 2»
- «Basic-FPTAS 3»
- «Branch-and-Bound Algorithm for the Knapsack Problem (P1)»
- «Branch-and-Bound Algorithm for the Knapsack Problem (P2)»
- «Branch-and-Bound Algorithm for the Knapsack Problem (P3)»
- «Branch-and-bound Algorithm for the Knapsack Problem (P4)»

## 2.13 Subset sum problem (SSP)

**Definition 2.2: Subset sum problem**

Given:

1. a *target value* $c \in \mathbb{Z}_{\geq 1}$,
2. a set $\mathscr{I} = \{1, 2, \dots, n\}$ of $n$ items,
3. for each item $j \in \mathscr{I}$, a *weight* $w_j \in \mathbb{Z}_{\geq 1}$,

the **Subset Sum Problem** (SSP) asks to decide whether there exists a subset of items whose total weight is exactly equal to the target value:

$$\text{find } S \subseteq \mathscr{I} \quad \text{such that} \quad \sum_{j \in S} w_j = c.$$

- Defined in Chapter 2 «Knapsack problems».
- The SSP is solved as the KP with $p_j = w_j$ for all $j \in \mathscr{I}$: the answer is yes if and only if the optimal value of this KP equals $c$.
- **Complexity**: Karp (1972) put the SSP among the first 21 NP-complete problems (Chapter 2 «Knapsack problems»).

**Where it is treated**

- Chapter 2 «Knapsack problems»: Definition, assumptions and solution as a KP
- Chapter 2 «Knapsack problems»: Enumeration of the reference instance: three subsets reach the target
- Exercise 2.2:

### 2.13.1 Instance SSP1

Reference instance of the SSP.
Capacity $c = 9$, $n = 6$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| $w_j$ | 2 | 3 | 6 | 7 | 5 | 9 |

- Answer: yes ($\{6\}$, $\{1,4\}$, $\{2,3\}$).
- Used in Example «SSP instance», The variants at a glance.

### 2.13.2 Instance SSP2

Subset sum with target 15.
Capacity $c = 15$, $n = 6$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| $w_j$ | 5 | 14 | 3 | 6 | 15 | 4 |

- Answer: yes ($\{5\}$ and $\{1,4,6\}$).
- Used in questions 1–3.

### 2.13.3 Instance SSP3

Subset sum with target 16, solved as a KP.
Capacity $c = 16$, $n = 6$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| $w_j$ | 5 | 14 | 3 | 6 | 15 | 4 |

- Optimal value $z(\text{SSP3}) = 15$, attained by the KP with $p_j = w_j$ reaches at most 15, e.g. $\{5\}$.
- Used in question 4.

## 2.14 Bounded knapsack problem (BKP)

**Definition 2.3: Bounded knapsack problem**

Given:

1. a knapsack of *capacity* $c \in \mathbb{Z}_{\geq 1}$,
2. a set $\mathscr{I} = \{1, 2, \dots, n\}$ of $n$ items,
3. for each item $j \in \mathscr{I}$, a *profit* $p_j \in \mathbb{Z}_{\geq 1}$, a *weight* $w_j \in \mathbb{Z}_{\geq 1}$, and a *number of available copies* $b_j \in \mathbb{Z}_{\geq 1}$,

the **Bounded Knapsack Problem** (BKP) asks to decide how many copies of each item to insert into the knapsack, so that the total weight of all selected item copies does not exceed the knapsack capacity, each item is taken no more than its available copies, and the total profit of all selected item copies is maximized. With the extended set of items $\hat{\mathscr{I}} = \{(j,k) : j \in \mathscr{I},\ k \in \{1,2,\dots,b_j\}\}$,

$$\tilde{S} \in \operatorname*{arg\,max}_{S \subseteq \hat{\mathscr{I}}} \left\{ \sum_{(j,k)\in S} p_j \ : \ \sum_{(j,k)\in S} w_j \le c \right\}.$$

- Defined in Chapter 2 «Knapsack problems».
- The BKP reduces to the KP when $b_j = 1$ for every item, and to the UKP when every bound $b_j$ is large enough not to be binding.
- **Complexity**: The DP algorithm takes $\Theta(nc\,b_{\max})$ steps, which is pseudo-polynomial (Chapter 17 «Dynamic programming for knapsack problems»).

**Where it is treated**

- Chapter 2 «Knapsack problems»: Definition, extended set of items and assumptions
- Chapter 19 «Dynamic programming for knapsack problems»: DP algorithm $v(j,k)$ over the numbers of copies, $\Theta(nc\,b_{\max})$ steps
- Exercise 2.3:
- Exercise 19.11:

### 2.14.1 Instance BKP1

Reference instance of the BKP.
Capacity $c = 9$, $n = 6$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| $p_j$ | 6 | 5 | 8 | 9 | 6 | 7 |
| $w_j$ | 2 | 3 | 6 | 7 | 5 | 9 |
| $b_j$ | 2 | 2 | 1 | 1 | 1 | 1 |

- Optimal value $z(\text{BKP1}) = 18$, attained by two copies of item 1 and item 5.
- Used in Example «BKP instance», The variants at a glance.

### 2.14.2 Instance BKP2

Copies of the items (BKP).
Capacity $c = 10$, $n = 3$ items:

| $j$ | 1 | 2 | 3 |
|---|---|---|---|
| $p_j$ | 4 | 13 | 13 |
| $w_j$ | 3 | 5 | 2 |
| $b_j$ | 1 | 2 | 2 |

- Optimal value $z(\text{BKP2}) = 39$, attained by $\tilde{x} = (0,1,2)$.
- Used in questions 1–3.

### 2.14.3 Instance BKP3

BKP instance of the DP chapter.
Capacity $c = 10$, $n = 3$ items:

| $j$ | 1 | 2 | 3 |
|---|---|---|---|
| $p_j$ | 6 | 10 | 12 |
| $w_j$ | 2 | 3 | 5 |
| $b_j$ | 2 | 2 | 1 |

- Optimal value $z(\text{BKP3}) = 32$, attained by $(2, 2, 0)$.
- Used in Example «BKP instance» and the examples of the BKP section.

### 2.14.4 Instance BKP4

BKP DP exercise 1.
Capacity $c = 6$, $n = 2$ items:

| $j$ | 1 | 2 |
|---|---|---|
| $p_j$ | 4 | 5 |
| $w_j$ | 2 | 3 |
| $b_j$ | 2 | 1 |

- Optimal value $z(\text{BKP4}) = 9$, attained by $(1, 1)$.
- Used in Exercise 17.11.

### 2.14.5 Instance BKP5

BKP DP exercise 2.
Capacity $c = 7$, $n = 2$ items:

| $j$ | 1 | 2 |
|---|---|---|
| $p_j$ | 3 | 8 |
| $w_j$ | 1 | 4 |
| $b_j$ | 3 | 1 |

- Optimal value $z(\text{BKP5}) = 17$, attained by $(3, 1)$.
- Used in Exercise 17.12.

### 2.14.6 Instance BKP6

BKP DP exercise 3.
Capacity $c = 5$, $n = 3$ items:

| $j$ | 1 | 2 | 3 |
|---|---|---|---|
| $p_j$ | 2 | 3 | 4 |
| $w_j$ | 1 | 2 | 3 |
| $b_j$ | 1 | 1 | 1 |

- Optimal value $z(\text{BKP6}) = 7$, attained by $(0, 1, 1)$.
- Used in Exercise 17.13.

### 2.14.7 Instance BKP7

BKP DP exercise 4.
Capacity $c = 8$, $n = 2$ items:

| $j$ | 1 | 2 |
|---|---|---|
| $p_j$ | 5 | 9 |
| $w_j$ | 2 | 5 |
| $b_j$ | 3 | 1 |

- Optimal value $z(\text{BKP7}) = 15$, attained by $(3, 0)$.
- Used in Exercise 17.14.

### 2.14.8 Instance BKP8

BKP DP exercise 5.
Capacity $c = 9$, $n = 3$ items:

| $j$ | 1 | 2 | 3 |
|---|---|---|---|
| $p_j$ | 4 | 5 | 3 |
| $w_j$ | 3 | 4 | 2 |
| $b_j$ | 2 | 1 | 2 |

- Optimal value $z(\text{BKP8}) = 12$, attained by $(1, 1, 1)$.
- Used in Exercise 17.15.

## 2.15 Unbounded knapsack problem (UKP)

**Definition 2.4: Unbounded knapsack problem**

Given:

1. a knapsack of *capacity* $c \in \mathbb{Z}_{\geq 1}$,
2. a set $\mathscr{I} = \{1, 2, \dots, n\}$ of $n$ items,
3. for each item $j \in \mathscr{I}$, which is available in an unlimited number of copies, a *profit* $p_j \in \mathbb{Z}_{\geq 1}$ and a *weight* $w_j \in \mathbb{Z}_{\geq 1}$,

the **Unbounded Knapsack Problem** (UKP) asks to decide how many copies of each item to insert into the knapsack, so that the total weight of all selected item copies does not exceed the knapsack capacity and the total profit of all selected item copies is maximized. With $\hat{b}_j = \lfloor c/w_j \rfloor$ and the extended set $\hat{\mathscr{I}} = \{(j,k) : j \in \mathscr{I},\ k \in \{1, 2, \dots, \hat{b}_j\}\}$,

$$\tilde{S} \in \underset{S \subseteq \hat{\mathscr{I}}}{\arg\max} \left\{ \sum_{(j,k) \in S} p_j \ :\ \sum_{(j,k) \in S} w_j \leq c \right\}.$$

- Defined in Chapter 2 «Knapsack problems».

- The UKP is the BKP with $b_j = \hat{b}_j = \lfloor c/w_j \rfloor$: each item can be taken as many times as it fits.
- **Complexity**: Both DP algorithms take $\Theta(nc)$ steps, which is pseudo-polynomial (Chapter 17 «Dynamic programming for knapsack problems»).

**Where it is treated**

- Chapter 2 «Knapsack problems»: Definition, maximum feasible number of copies and assumptions
- Chapter 18 «Principles of dynamic programming»: The recurrence $v(k)$ over the capacities among the DP algorithms of the module
- Chapter 19 «Dynamic programming for knapsack problems»: Two DP algorithms, $v(k)$ and $v(j,k)$, both in $\Theta(nc)$ steps
- Chapter 20 «Rod cutting and coin change»: Rod cutting and coin change reuse the DP of the UKP
- Exercise 2.3:
- Exercise 19.6:

### 2.15.1 Instance UKP1

Reference instance of the UKP.
Capacity $c = 9$, $n = 6$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| $p_j$ | 6 | 5 | 8 | 9 | 6 | 7 |
| $w_j$ | 2 | 3 | 6 | 7 | 5 | 9 |

- Optimal value $z(\text{UKP1}) = 24$, attained by four copies of item 1.
- Used in Example «UKP instance», The variants at a glance.

### 2.15.2 Instance UKP2

Copies of the items (UKP).
Capacity $c = 10$, $n = 3$ items:

| $j$ | 1 | 2 | 3 |
|---|---|---|---|
| $p_j$ | 4 | 13 | 13 |
| $w_j$ | 3 | 5 | 2 |

- Optimal value $z(\text{UKP2}) = 65$, attained by $\tilde{x} = (0, 0, 5)$.
- Used in question 4.

### 2.15.3 Instance UKP3

UKP instance of the DP chapter.
Capacity $c = 7$, $n = 4$ items:

| $j$ | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| $p_j$ | 4 | 5 | 10 | 11 |
| $w_j$ | 2 | 3 | 5 | 6 |

- Optimal value $z(\text{UKP3}) = 14$, attained by $(1, 0, 1, 0)$.
- Used in Example «UKP instance» and the examples of the UKP section, Exercise 17.6.

### 2.15.4 Instance UKP4

UKP DP exercise 2.
Capacity $c = 8$, $n = 5$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| $p_j$ | 2 | 5 | 7 | 8 | 10 |
| $w_j$ | 1 | 2 | 3 | 4 | 5 |

- Optimal value $z(\text{UKP4}) = 20$, attained by $(0, 4, 0, 0, 0)$.
- Used in Exercise 17.7.

### 2.15.5 Instance UKP5

UKP DP exercise 3.
Capacity $c = 10$, $n = 4$ items:

| $j$ | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| $p_j$ | 6 | 10 | 12 | 13 |
| $w_j$ | 3 | 4 | 6 | 7 |

- Optimal value $z(\text{UKP5}) = 22$, attained by $(2, 1, 0, 0)$.
- Used in Exercise 17.8.

### 2.15.6 Instance UKP6

UKP DP exercise 4.
Capacity $c = 9$, $n = 4$ items:

| $j$ | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| $p_j$ | 5 | 6 | 14 | 18 |
| $w_j$ | 2 | 3 | 6 | 7 |

- Optimal value $z(\text{UKP6}) = 23$, attained by $(1, 0, 0, 1)$.
- Used in Exercise 17.9.

### 2.15.7 Instance UKP7

UKP DP exercise 5.

Capacity $c = 11$, $n = 5$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| $p_j$ | 3 | 9 | 10 | 16 | 17 |
| $w_j$ | 2 | 5 | 6 | 9 | 10 |

- Optimal value $z(\text{UKP7}) = 19$, attained by $(1, 0, 0, 1, 0)$ with the first DP algorithm, $(0, 1, 1, 0, 0)$ with the second one.
- Used in Exercise 17.10.

## 2.16 Multidimensional knapsack problem (MDKP)

**Definition 2.5: Multidimensional knapsack problem**

Given:

1. a $k$-dimensional knapsack,
2. for each dimension $d \in \{1, 2, \dots, k\} = \mathscr{D}$, a *capacity* $c_d \in \mathbb{Z}_{\geq 1}$,
3. a set $\mathscr{I} = \{1, 2, \dots, n\}$ of $n$ items,
4. for each item $j \in \mathscr{I}$, a *profit* $p_j \in \mathbb{Z}_{\geq 1}$,
5. for each dimension $d \in \mathscr{D}$ and each item $j \in \mathscr{I}$, a *weight* $w_{dj} \in \mathbb{Z}_{\geq 1}$,

the **Multidimensional Knapsack Problem** (MDKP) asks to decide which items to insert into the knapsack, so that for every dimension the total weight of the selected items does not exceed the corresponding capacity, and the total profit of the selected items is maximized:

$$\tilde{S} \in \arg\max_{S \subseteq \mathscr{I}} \left\{ \sum_{j \in S} p_j \quad : \quad \sum_{j \in S} w_{dj} \leq c_d \quad \forall\, d \in \mathscr{D} \right\}.$$

- Defined in Chapter 2 «Knapsack problems».
- The MDKP is the KP with $k$ capacity constraints, one for each resource; with $k = 1$ it is the KP.

**Where it is treated**

- Chapter 2 «Knapsack problems»: Definition, assumptions and an instance with two dimensions
- Chapter 2 «Knapsack problems»: Enumeration of the reference instance

### 2.16.1 Instance MDKP1

Reference instance of the MDKP.
Capacities $c_1 = 10, c_2 = 12$, $n = 4$ items:

| $j$ | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| $p_j$ | 6 | 7 | 5 | 9 |
| $w_{1j}$ | 4 | 5 | 3 | 7 |
| $w_{2j}$ | 6 | 4 | 5 | 3 |

- Optimal value $z(\text{MDKP1}) = 14$, attained by $\{3, 4\}$.
- Used in Example «MDKP instance», The variants at a glance.

## 2.17 Multiple knapsack problem (MKP)

**Definition 2.6: Multiple knapsack problem**

Given:

1. $m \in \mathbb{Z}_{\geq 1}$ knapsacks,
2. for each knapsack $k \in \{1, 2, \ldots, m\} = \mathscr{K}$, a *capacity* $c_k \in \mathbb{Z}_{\geq 1}$,
3. a set $\mathscr{I} = \{1, 2, \ldots, n\}$ of $n$ items,
4. for each item $j \in \mathscr{I}$, a *profit* $p_j \in \mathbb{Z}_{\geq 1}$ and a *weight* $w_j \in \mathbb{Z}_{\geq 1}$,

the **Multiple Knapsack Problem** (MKP) asks to decide which items to insert into each knapsack, so that in every knapsack the total weight of the assigned items does not exceed its capacity, and the total profit of the selected items is maximized. A feasible solution is a family $\mathcal{S} = \{S_k\}_{k \in \mathscr{K}}$ of pairwise disjoint subsets $S_k \subseteq \mathscr{I}$ with $\sum_{j \in S_k} w_j \leq c_k$ for all $k \in \mathscr{K}$, and the objective is $\sum_{k \in \mathscr{K}} \sum_{j \in S_k} p_j$.

- Defined in Chapter 2 «Knapsack problems».
- The MKP is the KP with $m$ knapsacks; every feasible solution of the MKP is feasible for the KP with capacity $\sum_{k \in \mathscr{K}} c_k$, which gives a dual bound.

**Where it is treated**

- Chapter 2 «Knapsack problems»: Definition, assumptions and an instance with two knapsacks
- Chapter 2 «Knapsack problems»: Enumeration of the reference instance: three optimal solutions
- Exercise 2.4:

### 2.17.1 Instance MKP1

Reference instance of the MKP.
$m = 2$ knapsacks with capacities $c_1 = 10, c_2 = 12$, $n = 5$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| $p_j$ | 6 | 8 | 5 | 7 | 9 |
| $w_j$ | 4 | 6 | 5 | 3 | 7 |

- Optimal value $z(\text{MKP1}) = 30$, attained by $(S_1, S_2) = (\{1,2\},\{4,5\})$, $(\{2,4\},\{1,5\})$, $(\{4,5\},\{1,2\})$.
- Used in Example «MKP instance», The variants at a glance.

### 2.17.2 Instance MKP2

Two knapsacks.
$m = 2$ knapsacks with capacities $c_1 = 10, c_2 = 6$, $n = 5$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| $p_j$ | 4 | 8 | 9 | 10 | 13 |
| $w_j$ | 2 | 5 | 2 | 6 | 3 |

- Optimal value $z(\text{MKP2}) = 40$, attained by $S_1 = \{2,3,5\}$, $S_2 = \{4\}$, the only optimal solution.
- Used in Exercise 2.4.

## 2.18 Multiple-choice knapsack problem (MCKP)

**Definition 2.7: Multiple-choice knapsack problem**

Given:

1. a knapsack of *capacity* $c \in \mathbb{Z}_{\geq 1}$,
2. a set $\mathscr{I} = \{1, 2, \ldots, n\}$ of $n$ items,
3. the set of items is partitioned into $q \in \mathbb{Z}_{\geq 2}$ classes,
4. for each class $h \in \{1, 2, \ldots, q\}$, the subset $\mathscr{I}_h \subseteq \mathscr{I}$ contains the items of that class,
5. for each item $j \in \mathscr{I}$, a *profit* $p_j \in \mathbb{Z}_{\geq 1}$ and a *weight* $w_j \in \mathbb{Z}_{\geq 1}$,

the **Multiple-Choice Knapsack Problem** (MCKP) asks to decide which items to insert into the knapsack, so that exactly one item is selected from each class, the total weight of the selected items does not exceed the knapsack capacity, and the total profit of the selected items is maximized:

$$\tilde{S} \in \underset{S \subseteq \mathscr{I}}{\arg\max} \left\{ \sum_{j \in S} p_j \ : \ \sum_{j \in S} w_j \leq c, \quad |S \cap \mathscr{I}_h| = 1 \ \ \forall\, h \in \{1, 2, \ldots, q\} \right\}.$$

- Defined in Chapter 2 «Knapsack problems».
- The MCKP is the KP with the additional constraint that exactly one item of every class is selected, so an instance can be infeasible.

**Where it is treated**

- Chapter 2 «Knapsack problems»: Definition, feasibility condition $\sum_{h=1}^{q} \min_{j \in \mathscr{J}_h} w_j \leq c$ and an instance
- Chapter 2 «Knapsack problems»: Enumeration of the reference instance: only 3 of the 8 choices fit
- Exercise 2.5:

### 2.18.1 Instance MCKP1

Reference instance of the MCKP (classes $\{1,2\}$, $\{3,4\}$, $\{5,6\}$).
Capacity $c = 15$, 3 classes (pairs $(p, w)$ of the items of each class):

- class 1: $(6,4)$, $(8,6)$;
- class 2: $(5,5)$, $(7,8)$;
- class 3: $(4,3)$, $(9,7)$.

- Optimal value $z(\text{MCKP1}) = 17$, attained by $\{1,4,5\}$ and $\{2,3,5\}$.
- Used in Example «MCKP instance», The variants at a glance.

### 2.18.2 Instance MCKP2

One item per class (classes $\{1,2\}$, $\{3,4,5\}$, $\{6,7\}$).
Capacity $c = 16$, 3 classes (pairs $(p, w)$ of the items of each class):

- class 1: $(5,6)$, $(8,9)$;
- class 2: $(14,4)$, $(13,6)$, $(11,7)$;
- class 3: $(12,8)$, $(6,2)$.

- Optimal value $z(\text{MCKP2}) = 28$, attained by $\{2,3,7\}$, the only optimal solution.
- Used in Exercise 2.5.

## 2.19 Knapsack problem with setups (KPS)

**Definition 2.8: Knapsack problem with setups**

Given:

1. a knapsack of *capacity* $c \in \mathbb{Z}_{\geq 1}$,
2. a set $\mathscr{I} = \{1, 2, \ldots, n\}$ of $n$ items,
3. the set of items is partitioned into $q \in \mathbb{Z}_{\geq 2}$ classes,
4. for each class $h \in \{1, 2, \ldots, q\}$, the subset $\mathscr{J}_h \subseteq \mathscr{I}$ contains the items of that class, with an associated *setup cost* $f_h \in \mathbb{Z}_{\geq 1}$ and a *setup weight* $s_h \in \mathbb{Z}_{\geq 1}$,
5. for each item $j \in \mathscr{I}$, a *revenue* $p_j \in \mathbb{Z}_{\geq 1}$ and a *weight* $w_j \in \mathbb{Z}_{\geq 1}$,

the **Knapsack Problem with Setups** (KPS) asks to decide which items to insert into the knapsack, so that the total weight of the selected items plus the setup weights of their classes does not exceed the knapsack capacity, and the total revenue of the selected items minus the setup costs of their classes is maximized. With the activated classes $\mathscr{H}(S) := \{h : S \cap \mathscr{J}_h \neq \emptyset\}$,

$$\tilde{S} \in \underset{S \subseteq \mathscr{I}}{\arg\max} \left\{ \sum_{j \in S} p_j - \sum_{h \in \mathscr{H}(S)} f_h \; : \; \sum_{j \in S} w_j + \sum_{h \in \mathscr{H}(S)} s_h \leq c \right\}.$$

- Defined in Chapter 2 «Knapsack problems».
- The KPS is the KP in which selecting the first item of a class costs the setup $f_h$ and consumes the setup weight $s_h$; a class with no selected item costs nothing.

**Where it is treated**

- Chapter 2 «Knapsack problems»: Definition, activated classes, assumptions and an instance with two classes
- Chapter 2 «Knapsack problems»: Enumeration of the reference instance: three optimal solutions

### 2.19.1 Instance KPS1

Reference instance of the KPS.

Capacity $c = 20$, $n = 6$ items; class 1: items $\{1, 2, 3\}$, setup cost $f_1 = 3$, setup weight $s_1 = 2$; class 2: items $\{4, 5, 6\}$, setup cost $f_2 = 4$, setup weight $s_2 = 3$:

| $j$ | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| $p_j$ | 6 | 8 | 5 | 7 | 9 | 4 |
| $w_j$ | 4 | 5 | 3 | 6 | 7 | 2 |

- Optimal value $z(\text{KPS1}) = 16$, attained by $\{1, 2, 3\}$, $\{4, 5, 6\}$, $\{1, 2, 3, 6\}$.
- Used in Example «KPS instance», The variants at a glance.

## Chapter 3

# The bin packing problem

Pack items of given weights into the minimum number of identical bins of capacity $c$.

**Where it is treated**

- Chapter 1 «Combinatorial optimization problems»: First statement, a small instance, counting of the partitions and first bounds
- Chapter 3 «Bin packing and cutting stock»: Definition, assumptions, weight bound, and the cutting stock problem
- Chapter 7 «Parallel machine scheduling»: Relation with $P\,||\,C_{\max}$: machines as bins, makespan as capacity
- Chapter 10 «Formulations of the bin packing problem»: The ILP$_{\text{BPP}}$ model, its two LP relaxations and their duals, and the ratio $\frac{1}{2}$
- Chapter 20 «Heuristic algorithms: definitions»: Reference instance of the heuristic algorithms, primal and dual bounds, gap
- Chapter 21 «Constructive heuristics»: First Fit Decreasing and Best Fit Decreasing
- Chapter 22 «Local search»: Move and swap neighbourhoods, plateaus and the auxiliary objective $\sum_b L_b^2$
- Chapter 23 «Metaheuristics»: Large neighbourhood search with heuristic and exact repair
- Chapter 24 «MIP heuristics for the classical problems»: Rounding, diving, feasibility pump and the heuristics of Gurobi
- Chapter 26 «Approximation algorithms for bin packing»: Next Fit and the Any Fit class (First Fit, Best Fit, Worst Fit) with their worst-case ratios
- Chapter 28 «$P||C_{\max}$, metric TSP and the limits of approximation»: No approximation ratio below $\frac{3}{2}$, since deciding whether 2 bins suffice is PARTITION

## 3.1 Definition

**Definition 3.1: The bin packing problem**

Given:

1. an infinite number of bins, each with *capacity* $c \in \mathbb{Z}_{\geq 1}$,
2. a set $\mathscr{I} = \{1, 2, \dots, n\}$ of $n$ items,
3. for each item $j \in \mathscr{I}$, a *weight* $w_j \in \mathbb{Z}_{\geq 1}$,

the **Bin Packing Problem** (BPP) asks to decide in which bin to place each item, so that the total weight of the items in any bin does not exceed the bin capacity, and the total number of bins used is minimized. A feasible solution is a family $\mathcal{P} \subseteq 2^{\mathscr{I}}$ of nonempty, pairwise disjoint packing patterns whose union is $\mathscr{I}$, with $\sum_{j \in P} w_j \leq c$ for all $P \in \mathcal{P}$:

$$\tilde{\mathcal{P}} \in \operatorname*{arg\,min}_{\mathcal{P} \subseteq 2^{\mathscr{I}}} \left\{ |\mathcal{P}| \;:\; \begin{array}{l} P \neq \emptyset, \quad \forall\, P \in \mathcal{P}, \\ P \cap P' = \emptyset, \quad \forall\, P, P' \in \mathcal{P},\, P \neq P', \\ \displaystyle\bigcup_{P \in \mathcal{P}} P = \mathscr{I}, \\ \displaystyle\sum_{j \in P} w_j \leq c, \quad \forall\, P \in \mathcal{P} \end{array} \right\}.$$

- Defined in Chapter 3 «Bin packing and cutting stock».
- A solution is a **partition** of the items into packing patterns, one per bin, so the candidate solutions are counted by the Bell numbers (Chapter 1 «Combinatorial optimization problems»).
- The bins model containers, vans, disks or time slots of fixed size, and the problem asks to use as few of them as possible.
- The assumptions $w_j \leq c$ for all $j \in \mathscr{I}$ and $\sum_{j \in \mathscr{I}} w_j > c$ guarantee that every item fits in a bin and that more than one bin is needed (Chapter 3 «Bin packing and cutting stock»).

## 3.2 Bounds and complexity

- **Complexity**: Strongly NP-hard: no pseudo-polynomial algorithm exists unless $P = NP$.
- $\text{LB} = \left\lceil \sum_{j \in \mathscr{I}} w_j / c \right\rceil$ (weight bound): every feasible solution uses at least this number of bins (Chapter 3 «Bin packing and cutting stock»).
- $z(\text{LP-0}_{\text{BPP}}) = z(\text{LP}_{\text{BPP}}) = \sum_{j \in \mathscr{I}} w_j / c$, so rounding the LP bound up gives the weight bound again (Chapter 10 «Formulations of the bin packing problem»).
- The weight bound is not always tight: with three items of weight 6 and $c = 10$ it gives $\lceil 18/10 \rceil = 2$, while 3 bins are needed (Chapter 3 «Bin packing and cutting stock»).
- In an optimal solution no two bins have total weight at most $c/2$, which yields the ratio $\frac{1}{2}$ of the LP bound (Chapter 10 «Formulations of the bin packing problem»).

- In any solution of an Any Fit algorithm the items of two bins never fit together into one bin (Chapter 26 «Approximation algorithms for bin packing»).

## 3.3 Formulation

- **ILP model of the BPP** ($\text{ILP}_{\text{BPP}}$) (Chapter 10 «Formulations of the bin packing problem»):

$$\begin{aligned} \min \quad & \sum_{b\in\mathscr{B}} y_b \\ \text{subject to} \quad & \sum_{b\in\mathscr{B}} x_{jb} = 1, && \forall j \in \mathscr{I}, \\ & -\sum_{j\in\mathscr{I}} w_j\, x_{jb} + c\, y_b \geq 0, && \forall b \in \mathscr{B}, \\ & x_{jb} \in \{0,1\}, && \forall j \in \mathscr{I},\ \forall b \in \mathscr{B}, \\ & y_b \in \{0,1\}, && \forall b \in \mathscr{B} \end{aligned}$$

- **Dual of the first LP relaxation** ($D(\text{LP-0}_{\text{BPP}})$) (Chapter 10 «Formulations of the bin packing problem»):

$$\begin{aligned} \max \quad & \sum_{j\in\mathscr{I}} \mu_j \\ \text{subject to} \quad & \mu_j - w_j\, \pi_b \leq 0, && \forall j \in \mathscr{I},\ \forall b \in \mathscr{B}, \\ & c\, \pi_b \leq 1, && \forall b \in \mathscr{B}, \\ & \mu_j \gtreqless 0, && \forall j \in \mathscr{I}, \\ & \pi_b \geq 0, && \forall b \in \mathscr{B} \end{aligned}$$

- Given an upper bound $m$ on the number of bins, the $\text{ILP}_{\text{BPP}}$ model uses $mn$ assignment variables $x_{jb}$ and $m$ activation variables $y_b$, linked by the capacity constraints (Chapter 10 «Formulations of the bin packing problem»).
- Both LP relaxations, with $y_b \geq 0$ and with $0 \leq y_b \leq 1$, have optimal value $\sum_{j\in\mathscr{I}} w_j/c$, attained by $\tilde{x}_{jb} = \frac{1}{m}$ and certified by the dual solution $\tilde{\mu}_j = w_j/c$, $\tilde{\pi}_b = 1/c$ (Chapter 10 «Formulations of the bin packing problem»).
- The ratios $z(\text{LP}_{\text{BPP}})/z(\text{ILP}_{\text{BPP}})$ and $\lceil z(\text{LP}_{\text{BPP}})\rceil/z(\text{ILP}_{\text{BPP}})$ are at least $\frac{1}{2}$, and the value $\frac{1}{2}$ is approached asymptotically (Chapter 10 «Formulations of the bin packing problem», Chapter 10 «Formulations of the bin packing problem»).

## 3.4 Constructive heuristics

- First Fit Decreasing (FFD) sorts the items by non-increasing weight and packs each one into the first bin where it fits, in $O(n^2)$ time (Chapter 21 «Constructive heuristics»).
- Best Fit Decreasing (BFD) sorts the items in the same way and packs each one into the fullest bin where it fits; both find the optimum 3 on the reference instance (Chapter 21 «Constructive heuristics»).
- FFD satisfies $FFD(I) \leq \frac{11}{9}\,z(I) + \frac{6}{9}$ and the bound is tight, yet on some instances FFD is worse than First Fit in the input order (Chapter 21 «Constructive heuristics»).

## 3.5 Local search

- The move neighbourhood transfers one item to another non-empty bin where it fits, and the swap neighbourhood exchanges two items of different weight in different bins (Chapter 22 «Local search»).
- The number of bins is flat, so the search is guided by the auxiliary objective $\max \sum_b L_b^2$, whose change for a transfer of $\delta$ from $L_a$ to $L_b$ is $2\,\delta\,(L_b - L_a + \delta)$ (Chapter 22 «Local search»).
- Every move that empties a bin increases $\sum_b L_b^2$, and with this objective the move search turns the First Fit solution of the reference instance into an optimal one (Chapter 22 «Local search»).
- The variable neighbourhood descent with move and swap and the auxiliary objective reaches the optimum in 89 of 100 runs on the medium instance with $c = 10$ and $n = 12$ (Chapter 22 «Local search»).

## 3.6 Metaheuristics

- Large neighbourhood search empties the least loaded bin and $q-1$ random bins, repacks their items by FFD or exactly, and accepts packings with fewer bins or with a larger $\sum_b(\sum_{j \in P_b} w_j)^2$ (Chapter 23 «Metaheuristics»).
- On the instance BPP24 the FFD repair never improves the FFD packing, while the exact repair with $q \geq 3$ reaches the optimum 9 in every run (Chapter 23 «Metaheuristics»).

## 3.7 MIP heuristics

- Rounding and fractional diving fail on the reference instance, because the LP spreads an item over several bins and its value stays 3 at every fixing (Chapter 24 «MIP heuristics for the classical problems»).
- The feasibility pump finds a feasible packing with every seed, but it may leave empty bins open, since the distance function ignores the objective (Chapter 24 «MIP heuristics for the classical problems»).
- On a medium instance with $n = 60$ and $\mathrm{LB} = 23 = z(I)$ the heuristics of Gurobi decide the solution time, since the whole difficulty is primal (Chapter 24 «MIP heuristics for the classical problems»).

## 3.8 Approximation

- Every Any Fit algorithm satisfies the half-full property, hence $\mathscr{A}(I) \le 2\,z(I)$ (Chapter 26 «Approximation algorithms for bin packing», Chapter 26 «Approximation algorithms for bin packing»).
- Next Fit and Worst Fit have $r(NF) = r(WF) = 2$, shown by the instances with $c = n$ and weights alternating $\frac{n}{2}$ and 2 (Chapter 26 «Approximation algorithms for bin packing», Chapter 26 «Approximation algorithms for bin packing»).
- First Fit and Best Fit have $r(FF) = r(BF) = \frac{17}{10}$ (Chapter 26 «Approximation algorithms for bin packing»).
- No polynomial algorithm has a ratio below $\frac{3}{2}$ unless $P = NP$, since deciding whether 2 bins suffice is the NP-complete PARTITION problem (Chapter 28 «$P||C_{\max}$, metric TSP and the limits of approximation»).

## 3.9 Instances

### 3.9.1 Instance BPP1

Reference instance of the bin packing chapter.
Bin capacity $c = 10$, $n = 5$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| $w_j$ | 6 | 5 | 4 | 7 | 3 |

- Optimal value $z(\mathrm{BPP1}) = 3$, attained by $\{\{4,5\},\{1,3\},\{2\}\}$ (one of 4 optimal partitions).
- Used in Example «BPP instances», Section «Counting the solutions, a first solution and a simple bound», Example «Execution of NF», Example «Execution of FF», Example «Execution of BF», Example «Execution of WF».

### 3.9.2 Instance BPP2

Small instance of the first chapter, reference instance of the heuristic algorithms.
Bin capacity $c = 7$, $n = 6$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| $w_j$ | 4 | 4 | 5 | 3 | 2 | 3 |

- Optimal value $z(\text{BPP2}) = 3$, attained by $\{1, 4\}, \{2, 6\}, \{3, 5\}$, three full bins.
- Used in Example «a small bin packing instance», Example «how many candidate solutions», Example «bounds on the small instances», Example «BPP reference instance», Example «Bounds and gaps on the reference instances», Example «FF and FFD on the BPP reference instance», Example «BFD on the BPP reference instance», Example «The number of bins is blind, the auxiliary objective is not», Example «Starting from NF», Example «LNS on the BPP reference instance», Example «The LP solutions of the reference instances», Example «Rounding on the reference instances», Section «Diving», Example «The feasibility pump on the BPP».

### 3.9.3 Instance BPP3

Counting the partitions, instance (c).
Bin capacity $c = 8$, $n = 5$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| $w_j$ | 5 | 4 | 3 | 3 | 2 |

- Optimal value $z(\text{BPP3}) = 3$, attained by for example $\{\{1\}, \{2\}, \{3, 4, 5\}\}$ (13 optimal partitions).
- Used in «Counting the solutions».

### 3.9.4 Instance BPP4

First Fit is not optimal.
Bin capacity $c = 10$, $n = 6$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| $w_j$ | 5 | 4 | 5 | 6 | 8 | 2 |

- Optimal value $z(\text{BPP4}) = 3$, attained by $\{\{1, 3\}, \{2, 4\}, \{5, 6\}\}$.
- Used in «A greedy choice that is not optimal (BPP)».

### 3.9.5 Instance BPP5

Checking a packing.
Bin capacity $c = 12$, $n = 8$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|---|---|---|---|---|---|---|---|---|
| $w_j$ | 7 | 5 | 4 | 6 | 3 | 8 | 2 | 4 |

- Optimal value $z(\text{BPP5}) = 4$, attained by $\{\{1, 2\}, \{3, 6\}, \{4, 5, 7\}, \{8\}\}$.
- Used in «Checking a packing (BPP)».

### 3.9.6 Instance BPP6

Weight bound not tight.
Bin capacity $c = 10$, $n = 6$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| $w_j$ | 7 | 6 | 6 | 5 | 3 | 3 |

- Optimal value $z(\text{BPP6}) = 4$, attained by $\{\{1\},\{2\},\{3,6\},\{4,5\}\}$ (one of 12 optimal partitions).
- Used in «When the weight bound is not tight (BPP)».

### 3.9.7 Instance BPP7

Counting the packings.
Bin capacity $c = 12$, $n = 5$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| $w_j$ | 7 | 6 | 5 | 4 | 2 |

- Optimal value $z(\text{BPP7}) = 2$, attained by $\{\{1,3\},\{2,4,5\}\}$ (unique).
- Used in «Counting the packings (BPP)».

### 3.9.8 Instance BPP8

Filling every bin exactly.
Bin capacity $c = 10$, $n = 8$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|---|---|---|---|---|---|---|---|---|
| $w_j$ | 5 | 5 | 4 | 4 | 4 | 3 | 3 | 2 |

- Optimal value $z(\text{BPP8}) = 3$, attained by $\{\{1,2\},\{3,4,8\},\{5,6,7\}\}$.
- Used in «Filling every bin exactly (BPP)».

### 3.9.9 Instance BPP9

Filling every bin exactly, with item 8 of weight 3 (Question 4).
Bin capacity $c = 10$, $n = 8$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|---|---|---|---|---|---|---|---|---|
| $w_j$ | 5 | 5 | 4 | 4 | 4 | 3 | 3 | 3 |

- Optimal value $z(\text{BPP9}) = 4$.
- Used in «Filling every bin exactly (BPP)», Question 4.

### 3.9.10 Instance BPP10

Three items larger than $c/2$.
Bin capacity $c = 10$, $n = 3$ items:

| $j$ | 1 | 2 | 3 |
|---|---|---|---|
| $w_j$ | 6 | 6 | 6 |

- Optimal value $z(\text{BPP10}) = 3$, attained by one item per bin.
- Used in Section «Typical mistakes», «The bounds $y_b \leq 1$ and the choice of m (1)».

### 3.9.11 Instance BPP11

Example 1 of the formulation chapter.
Bin capacity $c = 10$, $n = 3$ items:

| $j$ | 1 | 2 | 3 |
|---|---|---|---|
| $w_j$ | 7 | 6 | 3 |

- Optimal value $z(\text{BPP11}) = 2$, attained by $\tilde{P}_1 = \{1, 3\}$, $\tilde{P}_2 = \{2\}$; $z(\text{LP}_{\text{BPP}}) = \frac{8}{5}$.
- Used in Example «ILP$_{\text{BPP}}$ model», Example «LP-0$_{\text{BPP}}$ model», Example «$D$(LP-0$_{\text{BPP}}$) model», Example «LP$_{\text{BPP}}$ model», Example «$D$(LP$_{\text{BPP}}$) model».

### 3.9.12 Instance BPP12

ILP model with $m = 3$; instance $I_0$ of the algorithm SEP.
Bin capacity $c = 10$, $n = 5$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| $w_j$ | 6 | 5 | 4 | 3 | 2 |

- Optimal value $z(\text{BPP12}) = 2$, attained by $\{\{1, 3\}, \{2, 4, 5\}\}$.
- Used in «ILP model of the Bin Packing Problem (1)», «An algorithm without a guarantee (1)».

### 3.9.13 Instance BPP13

LP relaxations and their duals.
Bin capacity $c = 12$, $n = 5$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| $w_j$ | 7 | 6 | 5 | 4 | 3 |

- Optimal value $z(\text{BPP13}) = 3$, attained by $\{1, 3\}, \{2, 4\}, \{5\}$; $z(\text{LP}_{\text{BPP}}) = \frac{25}{12}$.
- Used in «LP relaxations of the BPP model (1)».

### 3.9.14 Instance BPP14

Feasible solutions of the ILP model.
Bin capacity $c = 10$, $n = 6$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| $w_j$ | 5 | 4 | 4 | 3 | 2 | 2 |

- Optimal value $z(\text{BPP14}) = 2$, attained by $\{1, 4, 5\}, \{2, 3, 6\}$.
- Used in «Feasible solutions of the BPP model (1)».

### 3.9.15 Instance BPP15

Integrality gap, first family with $\vartheta = 4$, $n = 4$.
Bin capacity $c = 8$, $n = 4$ items:

| $j$ | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| $w_j$ | 5 | 5 | 5 | 5 |

- Optimal value $z(\text{BPP15}) = 4$.
- Used in «Integrality gap of the BPP model (1)».

### 3.9.16 Instance BPP16

Integrality gap, second family with $n = 6$.
Bin capacity $c = 6$, $n = 6$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| $w_j$ | 4 | 4 | 4 | 4 | 4 | 4 |

- Optimal value $z(\text{BPP16}) = 6$.
- Used in «Integrality gap of the BPP model (1)», Question 4.

### 3.9.17 Instance BPP17

Gap equal to the relative error.
Bin capacity $c = 10$, $n = 8$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|---|---|---|---|---|---|---|---|---|
| $w_j$ | 6 | 5 | 4 | 4 | 3 | 3 | 2 | 2 |

- Optimal value $z(\text{BPP17}) = 3$, attained by $\{1, 3\}, \{2, 5, 7\}, \{4, 6, 8\}$.
- Used in «Gaps for minimization problems (1)», instance (a).

### 3.9.18 Instance BPP18

Weak dual bound.
Bin capacity $c = 10$, $n = 5$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| $w_j$ | 6 | 6 | 6 | 5 | 2 |

- Optimal value $z(\text{BPP18}) = 4$.
- Used in «Gaps for minimization problems (1)», instance (b).

### 3.9.19 Instance BPP19

Sorting can hurt.
Bin capacity $c = 7$, $n = 6$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| $w_j$ | 2 | 2 | 3 | 2 | 2 | 3 |

- Optimal value $z(\text{BPP19}) = 2$, attained by $\{1, 2, 3\}, \{4, 5, 6\}$ (First Fit in the input order); FFD uses 3 bins.
- Used in Example «Sorting can hurt».

### 3.9.20 Instance BPP20

Decreasing bin packing 1.
Bin capacity $c = 10$, $n = 7$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|
| $w_j$ | 3 | 3 | 5 | 2 | 6 | 2 | 6 |

- Optimal value $z(\text{BPP20}) = 3$, attained by $\{1, 5\}, \{2, 7\}, \{3, 4, 6\}$ (FFD and BFD).
- Used in «Decreasing bin packing 1».

### 3.9.21 Instance BPP21

Decreasing bin packing 2.
Bin capacity $c = 10$, $n = 8$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|---|---|---|---|---|---|---|---|---|
| $w_j$ | 2 | 6 | 3 | 5 | 4 | 2 | 6 | 2 |

- Optimal value $z(\text{BPP21}) = 3$, attained by $\{1, 2, 6\}, \{3, 4, 8\}, \{5, 7\}$ (First Fit); FFD and BFD use 4 bins.
- Used in «Decreasing bin packing 2».

### 3.9.22 Instance BPP22

Decreasing bin packing 3.
Bin capacity $c = 10$, $n = 7$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|
| $w_j$ | 4 | 4 | 5 | 7 | 2 | 2 | 4 |

- Optimal value $z(\text{BPP22}) = 3$, attained by $\{4, 5\}, \{1, 3\}, \{2, 6, 7\}$ (FFD).
- Used in «Decreasing bin packing 3».

### 3.9.23 Instance BPP23

Medium instance of the local search chapter ($n = 12$).
Bin capacity $c = 10$, $n = 12$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| $w_j$ | 7 | 6 | 6 | 5 | 5 | 4 | 4 | 4 | 3 | 3 | 2 | 1 |

- Optimal value $z(\text{BPP23}) = 5$.
- Used in Section «Flat landscapes and auxiliary objectives», Section «Experiments on two medium instances», Section «Variable neighbourhood descent».

### 3.9.24 Instance BPP24

Local search BPP 1.
Bin capacity $c = 10$, $n = 7$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|
| $w_j$ | 2 | 3 | 2 | 7 | 4 | 7 | 4 |

- Optimal value $z(\text{BPP24}) = 3$.
- Used in «Local search BPP 1».

### 3.9.25 Instance BPP25

Local search BPP 2.
Bin capacity $c = 10$, $n = 6$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| $w_j$ | 2 | 3 | 4 | 5 | 4 | 2 |

- Optimal value $z(\text{BPP25}) = 2$, attained by $\{1, 3, 5\}, \{2, 4, 6\}$.
- Used in «Local search BPP 2».

### 3.9.26 Instance BPP26

Local search BPP 3.
Bin capacity $c = 10$, $n = 7$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|
| $w_j$ | 5 | 3 | 3 | 2 | 8 | 4 | 4 |

- Optimal value $z(\text{BPP26}) = 3$, attained by $\{1, 6\}, \{4, 5\}, \{2, 3, 7\}$.
- Used in «Local search BPP 3».

### 3.9.27 Instance BPP27

Medium instance BPP24 of the metaheuristics chapter.
Bin capacity $c = 100$, $n = 24$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| $w_j$ | 30 | 40 | 57 | 22 | 37 | 23 | 34 | 34 | 55 | 51 | 21 | 47 |

| $j$ | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 | 21 | 22 | 23 | 24 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| $w_j$ | 22 | 24 | 22 | 34 | 20 | 51 | 25 | 54 | 35 | 27 | 55 | 48 |

- Optimal value $z(\text{BPP27}) = 9$.
- Used in Example «The medium instances of the KP, the BPP and $P\,||\,C_{\max}$», Section «Large neighbourhood search for the BPP: heuristic and exact repair», Section «The methods on the medium instances».

### 3.9.28 Instance BPP28

Large neighbourhood search 1.
Bin capacity $c = 10$, $n = 8$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|---|---|---|---|---|---|---|---|---|
| $w_j$ | 2 | 7 | 5 | 5 | 3 | 2 | 6 | 6 |

- Optimal value $z(\text{BPP28}) = 4$, attained by $\{3, 4\}, \{8\}, \{2, 5\}, \{1, 6, 7\}$.
- Used in «Large neighbourhood search 1».

### 3.9.29 Instance BPP29

Large neighbourhood search 2.
Bin capacity $c = 10$, $n = 8$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|---|---|---|---|---|---|---|---|---|
| $w_j$ | 3 | 3 | 6 | 5 | 4 | 4 | 2 | 3 |

- Optimal value $z(\text{BPP29}) = 3$, attained by $\{3, 5\}, \{1, 4, 7\}, \{2, 6, 8\}$.
- Used in «Large neighbourhood search 2».

### 3.9.30 Instance BPP30

Medium instance with $n = 60$ of the MIP heuristics chapter.
Bin capacity $c = 100$, $n = 60$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| $w_j$ | 58 | 29 | 22 | 22 | 55 | 45 | 38 | 40 | 34 | 32 | 43 | 57 |

| $j$ | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 | 21 | 22 | 23 | 24 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| $w_j$ | 20 | 57 | 53 | 32 | 40 | 57 | 50 | 33 | 29 | 49 | 37 | 20 |

| $j$ | 25 | 26 | 27 | 28 | 29 | 30 | 31 | 32 | 33 | 34 | 35 | 36 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| $w_j$ | 27 | 22 | 55 | 40 | 48 | 40 | 52 | 24 | 21 | 48 | 31 | 36 |

| $j$ | 37 | 38 | 39 | 40 | 41 | 42 | 43 | 44 | 45 | 46 | 47 | 48 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| $w_j$ | 38 | 46 | 30 | 20 | 31 | 58 | 40 | 28 | 54 | 50 | 26 | 31 |

| $j$ | 49 | 50 | 51 | 52 | 53 | 54 | 55 | 56 | 57 | 58 | 59 | 60 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| $w_j$ | 27 | 54 | 26 | 26 | 22 | 30 | 33 | 55 | 27 | 50 | 47 | 26 |

- Optimal value $z(\text{BPP30}) = 23$.
- Used in Example «The effect of the heuristics of Gurobi on a medium BPP instance».

### 3.9.31 Instance BPP31

Instance $I_3$ of the algorithm SEP ($c = k = 3$, $2k$ unit items).
Bin capacity $c = 3$, $n = 6$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| $w_j$ | 1 | 1 | 1 | 1 | 1 | 1 |

- Optimal value $z(\text{BPP31}) = 2$.
- Used in «An algorithm without a guarantee (1)».

### 3.9.32 Instance BPP32

Next Fit is not an Any Fit algorithm.
Bin capacity $c = 10$, $n = 4$ items:

| $j$ | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| $w_j$ | 3 | 8 | 3 | 8 |

- Optimal value $z(\text{BPP32}) = 3$.
- Used in Example «NF is not an Any Fit algorithm», Example «Half-full property» (Next Fit).

### 3.9.33 Instance BPP33

Half-full property, Next Fit.
Bin capacity $c = 10$, $n = 4$ items:

| $j$ | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| $w_j$ | 5 | 5 | 5 | 5 |

- Optimal value $z(\text{BPP33}) = 2$.
- Used in Example «Half-full property».

### 3.9.34 Instance BPP34

Half-full property, First Fit.
Bin capacity $c = 10$, $n = 4$ items:

| $j$ | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| $w_j$ | 7 | 7 | 3 | 4 |

- Optimal value $z(\text{BPP34}) = 3$.
- Used in Example «Half-full property» (First Fit).

### 3.9.35 Instance BPP35

Any Fit exercises, instance 1.
Bin capacity $c = 10$, $n = 4$ items:

| $j$ | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| $w_j$ | 6 | 6 | 4 | 4 |

- Optimal value $z(\text{BPP35}) = 2$, attained by $\{1, 3\}, \{2, 4\}$.
- Used in «Next Fit 1», «First Fit 1», «Best Fit 1», «Worst Fit 1».

### 3.9.36 Instance BPP36

Any Fit exercises, instance 2.
Bin capacity $c = 10$, $n = 4$ items:

| $j$ | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| $w_j$ | 7 | 7 | 3 | 3 |

- Optimal value $z(\text{BPP36}) = 2$, attained by $\{1, 3\}, \{2, 4\}$.
- Used in «Next Fit 2», «First Fit 2», «Best Fit 2», «Worst Fit 2».

### 3.9.37 Instance BPP37

Any Fit exercises, instance 3.
Bin capacity $c = 10$, $n = 5$ items:

| $j$ | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| $w_j$ | 4 | 7 | 6 | 3 | 5 |

- Optimal value $z(\text{BPP37}) = 3$, attained by $\{2, 4\}, \{1, 3\}, \{5\}$.
- Used in «Next Fit 3», «First Fit 3», «Best Fit 3», «Worst Fit 3».

## 3.10 Exercises

- «Recognizing the problems» (in the chapter)
- «Counting the solutions»
- «A greedy choice that is not optimal (BPP)»
- «Checking a packing (BPP)»
- «When the weight bound is not tight (BPP)»
- «Counting the packings (BPP)»
- «Copies of the items (cutting stock)»
- «Filling every bin exactly (BPP)»
- «Makespan and bin packing ($P\|C_{\max}$)»
- «ILP model of the Bin Packing Problem (1)»
- «LP relaxations of the BPP model (1)»
- «Feasible solutions of the BPP model (1)»
- «The bounds $y_b \leq 1$ and the choice of m (1)»
- «Integrality gap of the BPP model (1)»
- «Gaps for minimization problems (1)»
- «Decreasing bin packing 1»
- «Decreasing bin packing 2»
- «Decreasing bin packing 3»
- «Local search BPP 1»
- «Local search BPP 2»
- «Local search BPP 3»
- «Large neighbourhood search 1»
- «Large neighbourhood search 2»
- «An algorithm without a guarantee (1)»
- «Next Fit 1»
- «Next Fit 2»
- «Next Fit 3»
- «First Fit 1»

- «First Fit 2»
- «First Fit 3»
- «Best Fit 1»
- «Best Fit 2»
- «Best Fit 3»
- «Worst Fit 1»
- «Worst Fit 2»
- «Worst Fit 3»

## 3.11 Cutting stock problem (CSP)

**Definition 3.2: Cutting stock problem**

Given:

1. an infinite number of bins, each with *capacity* $c \in \mathbb{Z}_{\geq 1}$,
2. a set $\mathscr{I} = \{1, 2, \dots, n\}$ of $n$ items,
3. for each item $j \in \mathscr{I}$, a *weight* $w_j \in \mathbb{Z}_{\geq 1}$ and a *number of required copies* $d_j \in \mathbb{Z}_{\geq 1}$,

the **Cutting Stock Problem** (CSP) asks to decide in which bin to place each copy of every item, so that the total weight of all item copies in any bin does not exceed the bin capacity, the required number of copies of each item is placed into the bins, and the total number of bins used is minimized. With the extended set of items

$$\hat{\mathscr{I}} = \{\, (j,k) : j \in \mathscr{I},\ k \in \{1, 2, \dots, d_j\} \,\}, \qquad |\hat{\mathscr{I}}| = \sum_{j \in \mathscr{I}} d_j,$$

a feasible solution is a partition $\mathcal{P} \subseteq 2^{\hat{\mathscr{I}}}$ of $\hat{\mathscr{I}}$ into nonempty packing patterns with $\sum_{(j,k) \in P} w_j \leq c$ for all $P \in \mathcal{P}$, and $|\mathcal{P}|$ is minimized.

- Defined in Chapter 3 «Bin packing and cutting stock».
- The CSP is the BPP with $d_j$ identical copies of every item $j$, and the BPP is the case $d_j = 1$ for all $j \in \mathscr{I}$.
- **Complexity**: Contains the BPP as the case $d_j = 1$, hence strongly NP-hard.
- LB $= \left\lceil \sum_{j \in \mathscr{I}} d_j\, w_j / c \right\rceil$ (weight bound with the weights of all the copies) (Chapter 3 «Bin packing and cutting stock»).

**Where it is treated**

- Chapter 3 «Bin packing and cutting stock»: Definition, extended set of items, assumptions

- Chapter 3 «Bin packing and cutting stock»: Counting the partitions, a first solution and the weight bound $\lceil \sum_j d_j w_j / c \rceil$
- Exercise 3.4: Exercise on the extended set of items and an optimal packing

### 3.11.1 Instance CSP1

Reference cutting stock instance of the bin packing chapter.
Roll length $c = 10$, 4 item types:

| $j$ | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| $w_j$ | 4 | 5 | 3 | 6 |
| $d_j$ | 2 | 1 | 3 | 1 |

- Optimal value $z(\text{CSP1}) = 3$, attained by

$$\{\{(4,1),(1,1)\},\{(1,2),(3,1),(3,2)\},\{(2,1),(3,3)\}\}$$

.

- Used in Example «CSP instances», Section «Counting the solutions, a first solution and a simple bound».

### 3.11.2 Instance CSP2

Copies of the items.
Roll length $c = 10$, 3 item types:

| $j$ | 1 | 2 | 3 |
|---|---|---|---|
| $w_j$ | 5 | 4 | 3 |
| $d_j$ | 2 | 3 | 2 |

- Optimal value $z(\text{CSP2}) = 3$, attained by

$$\{\{(1,1),(1,2)\},\{(2,1),(3,1),(3,2)\},\{(2,2),(2,3)\}\}$$

.

- Used in «Copies of the items (cutting stock)».

## Chapter 4

# The vertex coloring problem

Color the vertices of a graph so that adjacent vertices get different colors, using the minimum number of colors.

**Where it is treated**

- Chapter 1 «Combinatorial optimization problems»: First definition, the exam calendar and the count of its partitions
- Chapter 4 «Vertex coloring»: Definition, assumptions, the clique bound and the stable set bound
- Chapter 11 «Formulations of the vertex coloring problem»: The $\text{ILP}_{\text{VCP}}$ model and its two LP relaxations, whose value is always 2
- Chapter 28 «$P||C_{\max}$, metric TSP and the limits of approximation»: The greedy coloring $GC$ and $DSATUR$
- Chapter 28 «$P||C_{\max}$, metric TSP and the limits of approximation»: The limits of approximation and what the greedy coloring guarantees

## 4.1 Definition

**Definition 4.1: The vertex coloring problem**

Given:

1. a simple *graph* $G = (V, E)$, where

$$V = \{1, 2, \dots, n\}$$

is the set of $n$ *vertices*, and

$$E \subseteq \{\, \{u, v\} : u, v \in V,\ u \neq v \,\}$$

is the set of $m$ *edges*,

the **Vertex Coloring Problem** (VCP) asks to decide how to assign a color to each vertex, so that no two adjacent vertices share the same color, and the total number of colors used is minimized.

- Defined in Chapter 4 «Vertex coloring».
- The vertices are objects in conflict and the colors are resources that conflicting objects cannot share, as in the exam calendar of Chapter 1 «Combinatorial optimization problems», where the colors are the days of the session.
- A feasible solution is a partition of $V$ into nonempty color classes $\mathcal{Q}$, each of which is a stable set, and the number of colors is $|\mathcal{Q}|$.
- The optimal value is the **chromatic number** $\chi(G)$ of the graph, and the candidate solutions are counted by the Bell numbers $B(n)$.

## 4.2 Bounds and complexity

- **Complexity**: Strongly NP-hard: deciding whether $\chi(G) \leq 3$ is NP-complete, and the instance contains no numbers; the chromatic number is among the 21 NP-complete problems of Karp (1972).
- $\omega(G) \leq \chi(G)$ (clique bound): the vertices of a clique need pairwise different colors (Chapter 4 «Vertex coloring»).
- $\left\lceil \frac{n}{\alpha(G)} \right\rceil \leq \chi(G)$ (stable set bound): every color class is a stable set with at most $\alpha(G)$ vertices (Chapter 4 «Vertex coloring»).
- $z(\text{LP-0}_{\text{VCP}}) = z(\text{LP}_{\text{VCP}}) = 2$ for every graph with an edge: the LP bound is very weak (Chapter 11 «Formulations of the vertex coloring problem»).
- $\chi(G) \leq GC(I) \leq \Delta(G) + 1$ for every list $\lambda$: a primal bound from any greedy coloring (Chapter 28 «$P||C_{\max}$, metric TSP and the limits of approximation»).
- $\chi(G) \leq \Delta$ for a connected graph that is neither complete nor an odd cycle (Brooks, 1941) (Chapter 4 «Vertex coloring»).
- An isolated vertex and a vertex $u$ with $N(u) \subseteq N(v)$ can be removed without changing $\chi(G)$, so the notes assume $\deg(v) \geq 1$ and $N(u) \not\subseteq N(v)$ for all $u \neq v$ (Chapter 4 «Vertex coloring»).

- The two bounds do not always meet $\chi(G)$: on the Petersen graph $\omega(G) = 2 < \chi(G) = 3$, and on the wheel $W_5$ both bounds give 3 while $\chi(W_5) = 4$ (Exercise 4.4).
- Unless $P = NP$, the chromatic number cannot be approximated within $n^{1-\varepsilon}$ for any $\varepsilon > 0$ (Zuckerman, 2007, cited without proof in Chapter 28 «$P\|C_{\max}$, metric TSP and the limits of approximation»).

## 4.3 Formulation

- ILP$_{\text{VCP}}$ **model** (Chapter 11 «Formulations of the vertex coloring problem»):

$$\begin{aligned}
\min \sum_{c\in\mathscr{C}} & y_c & \\
\sum_{c\in\mathscr{C}} & x_{vc} = 1 & \forall v \in V \\
& -x_{uc} - x_{vc} + y_c \geq 0 & \forall c \in \mathscr{C},\ \forall\{u,v\} \in E \\
& x_{vc} \in \{0,1\} & \forall v \in V,\ \forall c \in \mathscr{C} \\
& y_c \in \{0,1\} & \forall c \in \mathscr{C}
\end{aligned}$$

- The ILP$_{\text{VCP}}$ model uses $k\,n$ variables $x_{vc}$ (vertex $v$ takes color $c$) and $k$ variables $y_c$ (color $c$ is used), with $k$ an upper bound on the number of colors, for example $k = n$ (Chapter 11 «Formulations of the vertex coloring problem»).
- The edge constraints $-x_{uc}-x_{vc}+y_c \geq 0$ forbid equal colors on adjacent vertices and switch off the unused colors; the model has $k!$ equivalent copies of every coloring (Chapter 11 «Formulations of the vertex coloring problem»).
- Both relaxations LP-0$_{\text{VCP}}$ and LP$_{\text{VCP}}$ have optimal value 2 for every graph with an edge: $\tilde{x}_{v1} = \tilde{x}_{v2} = \frac{1}{2}$ for all the vertices, and a dual solution built on one edge (Chapter 11 «Formulations of the vertex coloring problem»).
- On the complete graph with $n$ vertices the gap is $n - 2$ and the ratio $\frac{2}{n}$ tends to 0 (Exercise 11.2, Exercise 11.5).

## 4.4 Constructive heuristics

- The greedy coloring $GC$ examines the vertices in the order of a list $\lambda$ and gives each vertex the smallest color not used by its colored neighbours, in $O(n + |E|)$ time (Chapter 28 «$P\|C_{\max}$, metric TSP and the limits of approximation»).
- On the Petersen graph $GC$ uses 3 colors with the list $(1, 2, \ldots, 10)$ and 4 colors with the list $(1, 2, 3, 4, 5, 7, 8, 6, 9, 10)$ (Chapter 28 «$P\|C_{\max}$, metric TSP and the limits of approximation»).
- $DSATUR$ colors first an uncolored vertex of maximum saturation degree (ties by the degree among the uncolored vertices, then by the smallest index), in $O(n^2)$ time (Chapter 28 «$P\|C_{\max}$, metric TSP and the limits of approximation»).

- $DSATUR$ uses $3 = \chi(G)$ colors on the Petersen graph and 2 colors on every bipartite graph (Brélaz, 1979) (Chapter 28 «$P||C_{\max}$, metric TSP and the limits of approximation»).

## 4.5 Approximation

- If $P \neq NP$, no polynomial algorithm has $\frac{\mathscr{A}(I)}{z(I)} < \frac{4}{3}$ for every VCP instance, because it would decide 3-colorability (Chapter 28 «$P||C_{\max}$, metric TSP and the limits of approximation»).
- $GC(I) \leq \Delta(G) + 1$ for every list, which is not a performance guarantee: it compares $GC(I)$ with $\Delta(G) + 1$, not with $\chi(G)$ (Chapter 28 «$P||C_{\max}$, metric TSP and the limits of approximation»).
- On the crown graphs $G_k$, with $\chi(G_k) = 2$, a bad list gives $GC(I_k) = k = \frac{n}{2}$, so $GC$ has no constant performance guarantee (Chapter 28 «$P||C_{\max}$, metric TSP and the limits of approximation»).
- Some list is always optimal for $GC$: the vertices of the classes $Q_1, \ldots, Q_{\chi(G)}$ of an optimal coloring, one class after the other (Chapter 28 «$P||C_{\max}$, metric TSP and the limits of approximation»).

## 4.6 Instances

### 4.6.1 Instance VCP1

The Petersen graph, reference instance of the coloring chapter.
Graph with $n = 10$ vertices $V = \{1, \ldots, 10\}$ and 15 edges:

$$E = \big\{\{1,2\},\{1,5\},\{1,6\},\{2,3\},\{2,7\},\{3,4\},\{3,8\},\{4,5\},\{4,9\},\\ \{5,10\},\{6,8\},\{6,9\},\{7,9\},\{7,10\},\{8,10\}\big\}.$$

- Optimal value $z(\text{VCP1}) = 3$, attained by $\mathcal{Q} = \big\{ \{1,4,7,8\},\ \{2,5,6\},\ \{3,9,10\} \big\}$; $\omega(G) = 2$, $\alpha(G) = 4$, and 7 430 of the $B(10) = 115\,975$ partitions are feasible.
- Used in Example «VCP instance: the Petersen graph», Counting, a first solution and the bounds, Instance of the coloring examples, Example «$GC$ on the Petersen graph, two orders», Example «$DSATUR$ on the Petersen graph».

### 4.6.2 Instance VCP2

The triangle $K_3$.
Graph with $n = 3$ vertices $V = \{1, \ldots, 3\}$ and 3 edges:

$$E = \big\{\{1,2\},\{1,3\},\{2,3\}\big\}.$$

- Optimal value $z(\text{VCP2}) = 3$, attained by $\tilde{Q}_1 = \{1\}$, $\tilde{Q}_2 = \{2\}$, $\tilde{Q}_3 = \{3\}$, the only feasible partition of the $B(3) = 5$; $z(\text{LP-0}_{\text{VCP}}) = z(\text{LP}_{\text{VCP}}) = 2$.
- Used in Example «VCP instance: a triangle», Example «$\text{ILP}_{\text{VCP}}$ model», Example «$\text{LP-0}_{\text{VCP}}$ model», Example «$D(\text{LP-0}_{\text{VCP}})$ model», Example «$\text{LP}_{\text{VCP}}$ model», Example «$D(\text{LP}_{\text{VCP}})$ model».

### 4.6.3 Instance VCP3

An exam calendar: 6 exams and their conflicts.
Graph with $n = 6$ vertices $V = \{1, \dots, 6\}$ and 8 edges:

$$E = \big\{\{1,2\},\{1,3\},\{2,3\},\{2,4\},\{3,5\},\{4,5\},\{4,6\},\{5,6\}\big\}.$$

- Optimal value $z(\text{VCP3}) = 3$, attained by days $\{1,4\}$, $\{2,5\}$, $\{3,6\}$; 22 of the $B(6) = 203$ partitions are feasible.
- Used in Example «an exam calendar», Example «how many candidate solutions».

### 4.6.4 Instance VCP4

The cycle $C_5$.
Graph with $n = 5$ vertices $V = \{1, \dots, 5\}$ and 5 edges:

$$E = \big\{\{1,2\},\{2,3\},\{3,4\},\{4,5\},\{1,5\}\big\}.$$

- Optimal value $z(\text{VCP4}) = 3$, attained by $\tilde{Q}_1 = \{1,3\}$, $\tilde{Q}_2 = \{2,4\}$, $\tilde{Q}_3 = \{5\}$; 5 optimal partitions; $z(\text{LP}_{\text{VCP}}) = 2$.
- Used in Exercise 11.1, Exercise 11.3.

### 4.6.5 Instance VCP5

The complete graph $K_4$.
Graph with $n = 4$ vertices $V = \{1, \dots, 4\}$ and 6 edges:

$$E = \big\{\{1,2\},\{1,3\},\{1,4\},\{2,3\},\{2,4\},\{3,4\}\big\}.$$

- Optimal value $z(\text{VCP5}) = 4$, attained by four singletons; $z(\text{LP-0}_{\text{VCP}}) = z(\text{LP}_{\text{VCP}}) = 2$.
- Used in Exercise 11.2.

### 4.6.6 Instance VCP6

The wheel $W_5$: a cycle on $1, \dots, 5$ and the hub 6.
Graph with $n = 6$ vertices $V = \{1, \dots, 6\}$ and 10 edges:

$$E = \big\{\{1,2\},\{2,3\},\{3,4\},\{4,5\},\{1,5\},\{1,6\},\{2,6\},\{3,6\},\{4,6\},\\ \{5,6\}\big\}.$$

- Optimal value $z(\text{VCP6}) = 4$, attained by $\{\{1,3\},\{2,4\},\{5\},\{6\}\}$; $\omega(W_5) = 3$, $\alpha(W_5) = 2$, both bounds give 3; $z(\text{LP}_{\text{VCP}}) = 2$.
- Used in Exercise 4.4, Exercise 11.5.

### 4.6.7 Instance VCP7

The triangular prism: triangles $\{1,2,3\}$ and $\{4,5,6\}$ joined by $\{1,4\},\{2,5\},\{3,6\}$.
Graph with $n=6$ vertices $V=\{1,\dots,6\}$ and 9 edges:

$$E=\big\{\{1,2\},\{1,3\},\{2,3\},\{4,5\},\{4,6\},\{5,6\},\{1,4\},\{2,5\},\{3,6\}\big\}.$$

- Optimal value $z(\text{VCP7})=3$, attained by $z(\text{LP-0}_{\text{VCP}})=z(\text{LP}_{\text{VCP}})=2$ and $z(\text{ILP}_{\text{VCP}})=3$.
- Used in Exercise 11.4.

### 4.6.8 Instance VCP8

The prism with a seventh vertex adjacent to 1 and 2.
Graph with $n=7$ vertices $V=\{1,\dots,7\}$ and 11 edges:

$$E=\big\{\{1,2\},\{1,3\},\{1,4\},\{1,7\},\{2,3\},\{2,5\},\{2,7\},\{3,6\},\{4,5\},\\ \{4,6\},\{5,6\}\big\}.$$

- Optimal value $z(\text{VCP8})=3$, attained by $\mathcal{Q}_a=\{\{1,6\},\{2,4\},\{3,5,7\}\}$.
- Used in Exercise 4.1.

### 4.6.9 Instance VCP9

A graph with 6 vertices and 7 edges, a unique maximum clique and a unique maximum stable set.
Graph with $n=6$ vertices $V=\{1,\dots,6\}$ and 7 edges:

$$E=\big\{\{1,2\},\{1,4\},\{1,5\},\{2,3\},\{2,4\},\{3,6\},\{5,6\}\big\}.$$

- Optimal value $z(\text{VCP9})=3$, attained by $\{\{3,4,5\},\{1,6\},\{2\}\}$; $\omega(G)=3$ with $\{1,2,4\}$, $\alpha(G)=3$ with $\{3,4,5\}$.
- Used in Exercise 4.2.

### 4.6.10 Instance VCP10

The complement of the cycle $C_7$.
Graph with $n=7$ vertices $V=\{1,\dots,7\}$ and 14 edges:

$$E=\big\{\{1,3\},\{1,4\},\{1,5\},\{1,6\},\{2,4\},\{2,5\},\{2,6\},\{2,7\},\{3,5\},\\ \{3,6\},\{3,7\},\{4,6\},\{4,7\},\{5,7\}\big\}.$$

- Optimal value $z(\text{VCP10})=4$, attained by $\{\{1,2\},\{3,4\},\{5,6\},\{7\}\}$; $\omega(G)=3$, $\alpha(G)=2$, stable set bound $\lceil 7/2\rceil=4$.
- Used in Exercise 4.3.

### 4.6.11 Instance VCP11

A graph with an isolated vertex and two dominated vertices.
Graph with $n = 6$ vertices $V = \{1, \dots, 6\}$ and 6 edges:

$$E = \big\{\{1,2\},\{1,3\},\{2,3\},\{2,4\},\{3,4\},\{4,5\}\big\}.$$

- Optimal value $z(\text{VCP11}) = 3$, attained by $\{\{1,4,6\},\{2,5\},\{3\}\}$; the reduced graph is the triangle $\{2,3,4\}$; $B(6) = 203$ candidates.
- Used in Exercise 4.5.

### 4.6.12 Instance VCP12

A graph with 7 vertices and 11 edges where $GC$ uses one color too many.
Graph with $n = 7$ vertices $V = \{1, \dots, 7\}$ and 11 edges:

$$E = \big\{\{1,2\},\{1,3\},\{1,4\},\{2,4\},\{2,6\},\{3,7\},\{4,5\},\{4,6\},\{5,6\},\\ \{5,7\},\{6,7\}\big\}.$$

- Optimal value $z(\text{VCP12}) = 3$, attained by $DSATUR$: $Q_1 = \{4,7\}$, $Q_2 = \{1,6\}$, $Q_3 = \{2,3,5\}$; $GC$ with $(1, \dots, 7)$ uses 4 colors.
- Used in Exercise 28.7.

### 4.6.13 Instance VCP13

The crown graph $G_4$ ($u_i = 2i - 1$, $v_i = 2i$).
Graph with $n = 8$ vertices $V = \{1, \dots, 8\}$ and 12 edges:

$$E = \big\{\{1,4\},\{1,6\},\{1,8\},\{2,3\},\{3,6\},\{3,8\},\{2,5\},\{4,5\},\{5,8\},\\ \{2,7\},\{4,7\},\{6,7\}\big\}.$$

- Optimal value $z(\text{VCP13}) = 2$, attained by $\{1,3,5,7\}$, $\{2,4,6,8\}$; $GC$ with $(1,2,\dots,8)$ uses 4 colors, $DSATUR$ uses 2.
- Used in Example «$GC$ and $DSATUR$ on the crown graph $G_4$».

### 4.6.14 Instance VCP14

The crown graph $G_5$ (sides $\{1, \dots, 5\}$ and $\{6, \dots, 10\}$).
Graph with $n = 10$ vertices $V = \{1, \dots, 10\}$ and 20 edges:

$$E = \big\{\{1,7\},\{1,8\},\{1,9\},\{1,10\},\{2,6\},\{2,8\},\{2,9\},\{2,10\},\{3,6\},\\ \{3,7\},\{3,9\},\{3,10\},\{4,6\},\{4,7\},\{4,8\},\{4,10\},\{5,6\},\{5,7\},\\ \{5,8\},\{5,9\}\big\}.$$

- Optimal value $z(\text{VCP14}) = 2$, attained by $\{1, \dots, 5\}$, $\{6, \dots, 10\}$; $GC$ with $(1,6,2,7,\dots,5,10)$ uses 5 colors.
- Used in Exercise 28.8.

### 4.6.15 Instance VCP15

A graph with 7 vertices and 10 edges where $DSATUR$ uses 4 colors.
Graph with $n = 7$ vertices $V = \{1, \ldots, 7\}$ and 10 edges:

$$E = \big\{\{1,3\}, \{1,5\}, \{1,6\}, \{2,4\}, \{2,5\}, \{2,7\}, \{3,5\}, \{4,6\}, \{4,7\},\\ \{6,7\}\big\}.$$

- Optimal value $z(\text{VCP15}) = 3$, attained by $Q_1 = \{1,4\}$, $Q_2 = \{2,3,6\}$, $Q_3 = \{5,7\}$; $DSATUR(I) = 4$.
- Used in Exercise 28.9.

## 4.7 Exercises

- Checking a coloring (VCP)
- Neighbourhoods, cliques and stable sets (VCP)
- The stable set bound on a complement graph (VCP)
- When both bounds are not tight (VCP)
- The assumptions and the reduction of the graph (VCP)
- ILP model of the Vertex Coloring Problem (1)
- LP relaxations of the VCP model (1)
- Feasible solutions of the VCP model (1)
- Dual solutions of the VCP relaxations (1)
- Integrality gap of the VCP model (1)
- Vertex coloring approximation 1
- Vertex coloring approximation 2
- Vertex coloring approximation 3

## 4.8 3-colorability (3-COL)

**Definition 4.2: 3-colorability**

Given a graph $G = (V, E)$, decide whether $\chi(G) \leq 3$.

- Defined in Chapter 28 «$P||C_{\max}$, metric TSP and the limits of approximation».
- The decision version of the VCP with the number of colors fixed to 3: an algorithm with ratio below $\frac{4}{3}$ for the VCP would decide it.
- **Complexity**: NP-complete, even for planar graphs of maximum degree 4 (Garey, Johnson and Stockmeyer, 1976).

**Where it is treated**

- Chapter 1 «Combinatorial optimization problems»: Deciding whether $\chi(G) \leq 3$ is NP-complete, so the VCP is strongly NP-hard
- Chapter 4 «Vertex coloring»: NP-complete even on planar graphs of maximum degree 4 (Garey, Johnson and Stockmeyer)
- Chapter 28 «$P||C_{\max}$, metric TSP and the limits of approximation»: The reduction behind the bound $\frac{4}{3}$
- Exercise 28.9: The reduction on a graph where $DSATUR$ uses 4 colors

## Chapter 5

# The maximum clique problem

Find a largest set of pairwise adjacent vertices of a graph.

**Where it is treated**

- Chapter 4 «Vertex coloring»: Cliques, stable sets, $\omega(G)$ and $\alpha(G)$, the clique bound of the VCP
- Chapter 5 «The maximum clique problem»: Definition, complement graph, counting, a first clique by hand, four upper bounds, weighted variants, complexity
- Chapter 12 «Formulations of the maximum clique problem»: The $ILP_{MCP}$ model and its LP relaxations

## 5.1 Definition

**Definition 5.1: The maximum clique problem**

Given:

1. a simple *graph* $G = (V, E)$, where

$$V = \{1, 2, \dots, n\}$$

   is the set of $n$ *vertices*, and

$$E \subseteq \{\, \{u, v\} : u, v \in V,\ u \neq v \,\}$$

   is the set of $m$ *edges*,

the **Maximum Clique Problem** (MCP) asks to find a clique $K \subseteq V$ of $G$ whose number of vertices is maximum. Its optimal value is the clique number $\omega(G)$.

- Defined in Chapter 5 «The maximum clique problem».
- A clique is a set of pairwise adjacent vertices: groups of pairwise connected members of a social network, of pairwise compatible tasks or exams, common substructures of two molecules.

- A feasible solution is a clique $K \subseteq V$, the candidate solutions are the $2^n$ subsets of $V$, and the optimal value is the clique number $\omega(G)$.
- A clique of $G$ is a stable set of the complement graph $\overline{G}$, and in a coloring the vertices of a clique need pairwise different colors.

## 5.2 Bounds and complexity

- **Complexity**: NP-hard: the decision version (is there a clique with $k$ vertices?) is among the 21 NP-complete problems of Karp (1972); for every $\varepsilon > 0$ no polynomial algorithm guarantees $\omega(G)/n^{1-\varepsilon}$ vertices unless NP problems are solvable in randomized polynomial time (Håstad, 1999).
- $\omega(G) \leq \Delta + 1$ (degree bound) (Chapter 5 «The maximum clique problem»).
- $\omega(G)\,(\omega(G) - 1)/2 \leq m$ (edge bound) (Chapter 5 «The maximum clique problem»).
- $\omega(G) \leq k^{\max}$, the largest $k$ such that at least $k$ vertices have degree at least $k - 1$ (degree count bound) (Chapter 5 «The maximum clique problem»).
- $\omega(G) \leq \chi(G)$: the number of colors of any feasible coloring is an upper bound (coloring bound) (Chapter 5 «The maximum clique problem»).
- $z(LP_{MCP}) \geq n/2$: the LP bound can be arbitrarily weak (Chapter 12 «Formulations of the maximum clique problem»).
- For a fixed $k$, the cliques with $k$ vertices are checked in $O(n^k\, k^2)$ steps, while listing all the subsets takes $O(2^n\, n^2)$.
- A maximal clique is not necessarily maximum: in the reference instance $\{4, 5\}$ is maximal but not maximum.
- The coloring bound is not always tight: on the cycle with 5 vertices $\omega(G) = 2$ and $\chi(G) = 3$, and on the wheel $W_5$ $\omega = 3 < \chi = 4$ (Exercise 4.4).

## 5.3 Formulation

- $ILP_{MCP}$ **model** (Chapter 12 «Formulations of the maximum clique problem»):

$$\begin{aligned} \max \sum_{v \in V} x_v & \\ x_u + x_v \leq 1 \qquad & \forall \{u, v\} \in \overline{E} \\ x_v \in \{0, 1\} \qquad & \forall v \in V \end{aligned}$$

- The $ILP_{MCP}$ model has one binary variable $x_v$ per vertex and one constraint $x_u + x_v \leq 1$ per non-edge $\{u, v\} \in \overline{E}$ (Chapter 12 «Formulations of the maximum clique problem»).
- Setting $x_v = \frac{1}{2}$ for every vertex is feasible for $LP_{MCP}$, so $z(LP_{MCP}) \geq n/2$ and the ratio $z(LP_{MCP})/z(ILP_{MCP})$ is not bounded above over all graphs (Chapter 12 «Formulations of the maximum clique problem»).

- For a stable set $S$ of $G$, $\sum_{v\in S} x_v \leq 1$ is a valid inequality, and so is $\sum_{v\in C} x_v \leq (|C|-1)/2$ for a chordless odd cycle $C$ of $\overline{G}$ (Chapter 12 «Formulations of the maximum clique problem»).

## 5.4 Constructive heuristics

- A first clique by hand: start from a vertex of maximum degree and add, one at a time, a vertex adjacent to all the vertices already chosen; on the reference instance it gives $K = \{1, 2, 3\}$ and LB $= 3$ (Chapter 5 «The maximum clique problem»).

## 5.5 Instances

### 5.5.1 Instance MCP1

A cycle with a chord, reference instance of the clique chapter, $z = \omega(G)$.
Graph with $n = 7$ vertices $V = \{1, \ldots, 7\}$ and 8 edges:

$$E = \big\{\{1,2\},\{1,3\},\{1,7\},\{2,3\},\{3,4\},\{4,5\},\{5,6\},\{6,7\}\big\}.$$

- Optimal value $z(\text{MCP1}) = 3$, attained by $\tilde{K} = \{1, 2, 3\}$, unique; 17 of the 128 subsets are cliques; bounds $\Delta + 1 = 4$, edge 4, $k^{\max} = 3$, coloring 3.
- Used in Example «MCP instance: a cycle with a chord».

### 5.5.2 Instance MCP2

A dense graph with 8 vertices (all pairs except the opposite ones), $z = \omega(G)$.
Graph with $n = 8$ vertices $V = \{1, \ldots, 8\}$ and 24 edges:

$$\begin{aligned} E = \big\{&\{1,2\},\{1,3\},\{1,4\},\{1,6\},\{1,7\},\{1,8\},\{2,3\},\{2,4\},\{2,5\},\\ &\{2,7\},\{2,8\},\{3,4\},\{3,5\},\{3,6\},\{3,8\},\{4,5\},\{4,6\},\{4,7\},\\ &\{5,6\},\{5,7\},\{5,8\},\{6,7\},\{6,8\},\{7,8\}\big\}. \end{aligned}$$

- Optimal value $z(\text{MCP2}) = 4$, attained by $\tilde{K} = \{1, 3, 4, 6\}$, one of 16 maximum cliques; 81 cliques; degree bounds 7, coloring bound 4.
- Used in Example «MCP instance: a dense graph with 8 vertices».

### 5.5.3 Instance MCP3

The Petersen graph, $z = \omega(G)$.
Graph with $n = 10$ vertices $V = \{1, \ldots, 10\}$ and 15 edges:

$$\begin{aligned} E = \big\{&\{1,2\},\{1,5\},\{1,6\},\{2,3\},\{2,7\},\{3,4\},\{3,8\},\{4,5\},\{4,9\},\\ &\{5,10\},\{6,8\},\{6,9\},\{7,9\},\{7,10\},\{8,10\}\big\}. \end{aligned}$$

- Optimal value $z(\text{MCP3}) = 2$, attained by no triangle: the maximum cliques are the edges.
- Used in Dual bounds on the Petersen graph, Remark after Example «MCP instance: a dense graph with 8 vertices».

### 5.5.4 Instance MCP4

The cycle $C_5$, $z = \omega(G)$.
Graph with $n = 5$ vertices $V = \{1, \dots, 5\}$ and 5 edges:

$$E = \big\{\{1,2\}, \{2,3\}, \{3,4\}, \{4,5\}, \{1,5\}\big\}.$$

- Optimal value $z(\mathrm{MCP4}) = 2$, attained by $\omega(C_5) = 2 < \chi(C_5) = 3$.
- Used in Typical mistakes «believing that the coloring bound is tight», Example «$ILP_{MCP}$ model».

### 5.5.5 Instance MCP5

The triangle $K_3$, $z = \omega(G)$.
Graph with $n = 3$ vertices $V = \{1, \dots, 3\}$ and 3 edges:

$$E = \big\{\{1,2\}, \{1,3\}, \{2,3\}\big\}.$$

- Optimal value $z(\mathrm{MCP5}) = 3$, attained by $K = \{1, 2, 3\}$.
- Used in Example «VCP instance: a triangle», Dashboard of the reference instances.

### 5.5.6 Instance MCP6

The prism with a seventh vertex adjacent to 1 and 2, $z = \omega(G)$.
Graph with $n = 7$ vertices $V = \{1, \dots, 7\}$ and 11 edges:

$$E = \big\{\{1,2\}, \{1,3\}, \{1,4\}, \{1,7\}, \{2,3\}, \{2,5\}, \{2,7\}, \{3,6\}, \{4,5\}, \\ \{4,6\}, \{5,6\}\big\}.$$

- Optimal value $z(\mathrm{MCP6}) = 3$, attained by cliques with 3 vertices $\{1,2,3\}$, $\{1,2,7\}$, $\{4,5,6\}$; the notes state $\omega(G) \geq 3$ and $\chi(G) = 3$.
- Used in Exercise 4.1.

### 5.5.7 Instance MCP7

A graph with 6 vertices and 7 edges, $z = \omega(G)$.
Graph with $n = 6$ vertices $V = \{1, \dots, 6\}$ and 7 edges:

$$E = \big\{\{1,2\}, \{1,4\}, \{1,5\}, \{2,3\}, \{2,4\}, \{3,6\}, \{5,6\}\big\}.$$

- Optimal value $z(\mathrm{MCP7}) = 3$, attained by the unique maximum clique $\{1, 2, 4\}$.
- Used in Exercise 4.2.

### 5.5.8 Instance MCP8

The complement of the cycle $C_7$, $z = \omega(G)$.
Graph with $n = 7$ vertices $V = \{1, \dots, 7\}$ and 14 edges:

$$E = \big\{\{1,3\}, \{1,4\}, \{1,5\}, \{1,6\}, \{2,4\}, \{2,5\}, \{2,6\}, \{2,7\}, \{3,5\}, \\ \{3,6\}, \{3,7\}, \{4,6\}, \{4,7\}, \{5,7\}\big\}.$$

- Optimal value $z(\mathrm{MCP8}) = 3$, attained by $\{1, 3, 5\}$, a stable set of $C_7$.
- Used in Exercise 4.3.

### 5.5.9 Instance MCP9

The wheel $W_5$, $z = \omega(W_5)$.
Graph with $n = 6$ vertices $V = \{1, \dots, 6\}$ and 10 edges:

$$E = \big\{\{1,2\},\{2,3\},\{3,4\},\{4,5\},\{1,5\},\{1,6\},\{2,6\},\{3,6\},\{4,6\},\\ \{5,6\}\big\}.$$

- Optimal value $z(\text{MCP9}) = 3$, attained by $\{1, 2, 6\}$.
- Used in Exercise 4.4.

### 5.5.10 Instance MCP10

A graph with 7 vertices and 11 edges, $z = \omega(G)$.
Graph with $n = 7$ vertices $V = \{1, \dots, 7\}$ and 11 edges:

$$E = \big\{\{1,2\},\{1,3\},\{1,4\},\{2,4\},\{2,6\},\{3,7\},\{4,5\},\{4,6\},\{5,6\},\\ \{5,7\},\{6,7\}\big\}.$$

- Optimal value $z(\text{MCP10}) = 3$, attained by $\{1, 2, 4\}$; no clique of size 4.
- Used in Exercise 28.7.

## 5.6 Exercises

- Checking a coloring (VCP)
- Neighbourhoods, cliques and stable sets (VCP)
- The stable set bound on a complement graph (VCP)
- When both bounds are not tight (VCP)
- Vertex coloring approximation 1

## 5.7 Maximum stable set problem (MSSP)

**Definition 5.2: Maximum stable set problem**

Let $G = (V, E)$ be a graph.

1. A **stable set** (also called **independent set**) is a subset $Q \subseteq V$ of pairwise non-adjacent vertices: $\{u, v\} \notin E$ for all $u, v \in Q$ with $u \neq v$.
2. The **Maximum Stable Set Problem** (MSSP) asks to find a stable set of $G$ whose number of vertices is maximum. Its optimal value is the **stability number** $\alpha(G)$.

- Defined in Chapter 5 «The maximum clique problem».
- A subset is a clique of $G$ if and only if it is a stable set of $\overline{G}$: $\omega(G) = \alpha(\overline{G})$ and $\alpha(G) = \omega(\overline{G})$.

- **Complexity**: NP-hard, as the MCP (through the complement graph).

**Where it is treated**

- Chapter 6 «The maximum clique problem»: The MCP on $G$ is the MSSP on $\overline{G}$
- Chapter 4 «Vertex coloring»: The stable set bound $\lceil n/\alpha(G) \rceil \leq \chi(G)$ of the VCP
- Exercise 4.3: $\alpha(G)$ as the clique number of the complement graph

### 5.7.1 Instance MSSP1

The Petersen graph, $z = \alpha(G)$.
Graph with $n = 10$ vertices $V = \{1, \ldots, 10\}$ and 15 edges:

$$E = \big\{\{1,2\},\{1,5\},\{1,6\},\{2,3\},\{2,7\},\{3,4\},\{3,8\},\{4,5\},\{4,9\},\\ \{5,10\},\{6,8\},\{6,9\},\{7,9\},\{7,10\},\{8,10\}\big\}.$$

- Optimal value $z(\text{MSSP1}) = 4$, attained by $\{1,4,7,8\}$.
- Used in Dual bounds on the Petersen graph.

### 5.7.2 Instance MSSP2

A graph with 6 vertices and 7 edges, $z = \alpha(G)$.
Graph with $n = 6$ vertices $V = \{1, \ldots, 6\}$ and 7 edges:

$$E = \big\{\{1,2\},\{1,4\},\{1,5\},\{2,3\},\{2,4\},\{3,6\},\{5,6\}\big\}.$$

- Optimal value $z(\text{MSSP2}) = 3$, attained by the unique maximum stable set $\{3,4,5\}$.
- Used in Exercise 4.2.

### 5.7.3 Instance MSSP3

The complement of the cycle $C_7$, $z = \alpha(G)$.
Graph with $n = 7$ vertices $V = \{1, \ldots, 7\}$ and 14 edges:

$$E = \big\{\{1,3\},\{1,4\},\{1,5\},\{1,6\},\{2,4\},\{2,5\},\{2,6\},\{2,7\},\{3,5\},\\ \{3,6\},\{3,7\},\{4,6\},\{4,7\},\{5,7\}\big\}.$$

- Optimal value $z(\text{MSSP3}) = 2$, attained by $\{1,2\}$, an edge of $\overline{G} = C_7$.
- Used in Exercise 4.3.

### 5.7.4 Instance MSSP4

The wheel $W_5$, $z = \alpha(W_5)$.
Graph with $n = 6$ vertices $V = \{1, \ldots, 6\}$ and 10 edges:

$$E = \big\{\{1,2\},\{2,3\},\{3,4\},\{4,5\},\{1,5\},\{1,6\},\{2,6\},\{3,6\},\{4,6\},\\ \{5,6\}\big\}.$$

- Optimal value $z(\text{MSSP4}) = 2$, attained by $\{1,3\}$.
- Used in Exercise 4.4.

## 5.8 Vertex-weighted maximum clique problem (VWMCP)

**Definition 5.3: Vertex-weighted maximum clique problem**

Given a graph $G$ and a vertex-weight vector $\boldsymbol{w} \in \mathbb{Z}^n_{\geq 1}$, the **vertex-weighted maximum clique problem** asks to find a clique $K$ of $G$ with maximum total vertex weight $w(K) = \sum_{v \in K} w_v$.

- Defined in Chapter 5 «The maximum clique problem».
- With all the weights $w_v = 1$ it is the MCP.

**Where it is treated**

- Chapter 6 «The maximum clique problem»: Example «vertex-weighted maximum clique problem»

### 5.8.1 Instance VWMCP1

The dense graph with 8 vertices and vertex weights.
Graph with $n = 8$ vertices $V = \{1, \ldots, 8\}$ and 24 edges:

$$\begin{aligned} E = \big\{&\{1,2\},\{1,3\},\{1,4\},\{1,6\},\{1,7\},\{1,8\},\{2,3\},\{2,4\},\{2,5\},\\ &\{2,7\},\{2,8\},\{3,4\},\{3,5\},\{3,6\},\{3,8\},\{4,5\},\{4,6\},\{4,7\},\\ &\{5,6\},\{5,7\},\{5,8\},\{6,7\},\{6,8\},\{7,8\}\big\}. \end{aligned}$$

Vertex weights:

| $v$ | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|---|---|---|---|---|---|---|---|---|
| $w_v$ | 5 | 8 | 9 | 5 | 6 | 5 | 5 | 9 |

- Optimal value $z(\text{VWMCP1}) = 32$, attained by $\{2, 3, 5, 8\}$, unique.
- Used in Example «vertex-weighted maximum clique problem».

## 5.9 Edge-weighted maximum clique problem (EWMCP)

**Definition 5.4: Edge-weighted maximum clique problem**

Given a graph $G$ and an edge-weight vector $\boldsymbol{r} \in \mathbb{Z}^m_{\geq 1}$, the **edge-weighted maximum clique problem** asks to find a clique $K$ of $G$ with maximum total edge weight $r(K) = \sum_{\{u,v\} \in E[K]} r_{uv}$, where $E[K]$ is the set of the edges with both endpoints in $K$.

- Defined in Chapter 5 «The maximum clique problem».
- With $r_{uv} = 1$ it is again the MCP, since $r(K) = \binom{|K|}{2}$ grows with $|K|$.

**Where it is treated**

- Chapter 6 «The maximum clique problem»: Example «edge-weighted maximum clique problem»

### 5.9.1 Instance EWMCP1

The dense graph with 8 vertices and edge weights.
Graph with $n = 8$ vertices $V = \{1, \dots, 8\}$ and 24 edges:

$$\begin{aligned}E = \{&\{1,2\},\{1,3\},\{1,4\},\{1,6\},\{1,7\},\{1,8\},\{2,3\},\{2,4\},\{2,5\},\\ &\{2,7\},\{2,8\},\{3,4\},\{3,5\},\{3,6\},\{3,8\},\{4,5\},\{4,6\},\{4,7\},\\ &\{5,6\},\{5,7\},\{5,8\},\{6,7\},\{6,8\},\{7,8\}\}.\end{aligned}$$

Edge weights: $r_{12} = 1$, $r_{13} = 4$, $r_{14} = 5$, $r_{16} = 4$, $r_{17} = 3$, $r_{18} = 3$, $r_{23} = 2$, $r_{24} = 5$, $r_{25} = 6$, $r_{27} = 9$, $r_{28} = 4$, $r_{34} = 3$, $r_{35} = 6$, $r_{36} = 1$, $r_{38} = 6$, $r_{45} = 4$, $r_{46} = 7$, $r_{47} = 2$, $r_{56} = 5$, $r_{57} = 7$, $r_{58} = 3$, $r_{67} = 1$, $r_{68} = 2$, $r_{78} = 8$.

- Optimal value $z(\text{EWMCP1}) = 37$, attained by $\{2, 5, 7, 8\}$, unique.
- Used in Example «edge-weighted maximum clique problem».

## Chapter 6

# The travelling salesman problem

Find a shortest closed route that visits every city exactly once and returns to the starting city.

**Where it is treated**

- Chapter 1 «Combinatorial optimization problems»: First definition, counting the tours, nearest neighbour and the two-shortest-edges bound
- Chapter 6 «The travelling salesman problem»: Definition, metric TSP and ATSP, counting and enumeration, nearest neighbour, a combinatorial lower bound
- Chapter 20 «Heuristic algorithms: definitions»: Reference problem of the heuristic part: gap and relative error, the two-edge bound, the search space of the tours
- Chapter 21 «Constructive heuristics»: Append or insert: $NN$, multi-start $NN$, $NI$, $FI$, $CI$, randomized $NN$, regret insertion, the pilot method
- Chapter 22 «Local search»: 2-opt and Or-opt local search, $VND$, multi-start local search
- Chapter 23 «Metaheuristics»: $SA$, tabu search, $ILS$ with the double bridge, GRASP, LNS and ALNS on TSP20
- Chapter 24 «MIP heuristics for the classical problems»: The symmetric edge model and its LP relaxation on the reference instance
- Chapter 13 «Formulations of the travelling salesman problem»: The MTZ and DFJ models and their LP relaxations
- Chapter 28 «$P||C_{\max}$, metric TSP and the limits of approximation»: Double tree and Christofides for the metric TSP
- Chapter 28 «$P||C_{\max}$, metric TSP and the limits of approximation»: No constant ratio for the general TSP unless $P = NP$

## 6.1 Definition

**Definition 6.1: The travelling salesman problem**

Given:

1. a set $\mathscr{V} = \{1, 2, \ldots, n\}$ of $n$ *cities*,
2. for each pair of distinct cities $i, j \in \mathscr{V}$, a *distance* $d_{ij} \in \mathbb{Z}_{\geq 1}$, with $d_{ij} = d_{ji}$,

the **travelling salesman problem** (TSP) asks to find a closed route that starts from a city, visits every other city exactly once and returns to the starting city, so that the total distance travelled is minimized.

- Defined in Chapter 6 «The travelling salesman problem».
- The cities are places to visit and $d_{ij}$ the cost of going from $i$ to $j$, as for a technician who visits her sites and comes back to the office (Chapter 1 «Combinatorial optimization problems»).
- A feasible solution is a **tour**, a permutation $\sigma = (\sigma_1, \ldots, \sigma_n)$ read as the closed route $\sigma_1 \to \cdots \to \sigma_n \to \sigma_1$, of length $L(\sigma) = \sum_{k=1}^{n-1} d_{\sigma_k \sigma_{k+1}} + d_{\sigma_n \sigma_1}$.
- The combinatorial object is a cyclic ordering of the cities: a symmetric instance has $\frac{(n-1)!}{2}$ distinct tours, about $6.1 \cdot 10^{16}$ for 20 cities.

## 6.2 Bounds and complexity

- **Complexity**: Strongly NP-hard: it contains the Hamiltonian cycle problem; for the general TSP no polynomial algorithm has a constant performance ratio unless $P = NP$.
- $z(I) \geq \left\lceil \frac{1}{2} \sum_{i \in \mathscr{V}} (a_i + b_i) \right\rceil$, where $a_i \leq b_i$ are the two smallest distances from city $i$ (two shortest edges) (Chapter 6 «The travelling salesman problem»).
- $d(T) \leq z(I)$ for every minimum spanning tree $T$, metric or not (Chapter 28 «$P||C_{\max}$, metric TSP and the limits of approximation»).
- Assignment relaxation $\leq z(\mathrm{LP}^{\mathrm{MTZ}}_{\mathrm{TSP}}) \leq z(\mathrm{LP}^{\mathrm{DFJ}}_{\mathrm{TSP}}) \leq z(I)$ on the reference instance: $16 \leq 17 \leq 18 = 18$ (Chapter 13 «Formulations of the travelling salesman problem»).
- relative error $\leq gap(I) = \frac{\mathscr{A}(I) - \mathrm{LB}(I)}{\mathrm{LB}(I)}$: on the reference instance 47.1% against 38.9% for the $NN$ tour (Chapter 20 «Heuristic algorithms: definitions»).
- Different permutations describe the same tour: $n$ starting cities and, in the symmetric case, 2 directions (Chapter 6 «The travelling salesman problem»).
- Shortcutting a closed walk does not increase its length in a metric instance, and a minimum perfect matching of an even set of cities weighs at most $\frac{z(I)}{2}$ (Chapter 28 «$P||C_{\max}$, metric TSP and the limits of approximation»).

## 6.3 Formulation

- $\text{ILP}_{\text{TSP}}^{\text{MTZ}}$ **model** (Chapter 13 «Formulations of the travelling salesman problem»):

$$\begin{aligned}
\min & \sum_{i\in\mathscr{V}}\sum_{\substack{j\in\mathscr{V}\\ j\neq i}} d_{ij}x_{ij} & \\
& \sum_{\substack{j\in\mathscr{V}\\ j\neq i}} x_{ij} = 1 & \forall i\in\mathscr{V}\\
& \sum_{\substack{i\in\mathscr{V}\\ i\neq j}} x_{ij} = 1 & \forall j\in\mathscr{V}\\
& u_i - u_j + (n-1)x_{ij} \le n-2 & \forall i,j\in\mathscr{V}\setminus\{1\},\ i\neq j\\
& x_{ij}\in\{0,1\} & \forall (i,j)\in A\\
& 1\le u_i\le n-1 & \forall i\in\mathscr{V}\setminus\{1\}
\end{aligned}$$

- $\text{ILP}_{\text{TSP}}^{\text{DFJ}}$ **model** (Chapter 13 «Formulations of the travelling salesman problem»):

$$\begin{aligned}
\min & \sum_{i\in\mathscr{V}}\sum_{\substack{j\in\mathscr{V}\\ j\neq i}} d_{ij}x_{ij} & \\
& \sum_{\substack{j\in\mathscr{V}\\ j\neq i}} x_{ij} = 1 & \forall i\in\mathscr{V}\\
& \sum_{\substack{i\in\mathscr{V}\\ i\neq j}} x_{ij} = 1 & \forall j\in\mathscr{V}\\
& \sum_{i\in S}\sum_{\substack{j\in S\\ j\neq i}} x_{ij} \le |S|-1 & \forall S\subseteq\mathscr{V}\setminus\{1\},\ |S|\ge 2\\
& x_{ij}\in\{0,1\} & \forall (i,j)\in A
\end{aligned}$$

- The MTZ model uses $n(n-1)$ arc variables and $n-1$ positions $u_i$; the constraints $u_i - u_j + (n-1)x_{ij} \le n-2$ eliminate the subtours without city 1 (Chapter 13 «Formulations of the travelling salesman problem»).

- The DFJ model adds $2^{n-1} - n$ subtour elimination constraints, which are added only when violated (Chapter 13 «Formulations of the travelling salesman problem»).

- The MTZ relaxation is weak: every point with $x_{ij} \le \frac{n-2}{n-1}$ and $u_i = 1$ is feasible; on the reference instance the bounds are 16, 17, 18 (Chapter 13 «Formulations of the travelling salesman problem»).

## 6.4 Constructive heuristics

- The nearest neighbour rule from city 1 (ties by the smallest index) gives $(1, 3, 2, 4, 5)$ of length 25 on the reference instance (Chapter 6 «The travelling salesman problem»).

- Multi-start $NN$ returns $18 = z(I)$ on the reference instance (lengths $25, 18, 18, 19, 18$ from cities $1, \ldots, 5$), in $O(n^3)$ (Chapter 21 «Constructive heuristics»).

- Insertion heuristics use the cost $\Delta(i, k, j) = d_{ik} + d_{kj} - d_{ij}$; $NI$, $FI$ and $CI$ all give 18 on the reference instance, and on metric instances $NI$ and $CI$ are at most twice the optimum (Chapter 21 «Constructive heuristics»).

- Randomized $NN_\alpha$ with a value-based RCL, regret insertion $REG_2$ ($19 = z(I)$ against $CI(I) = 20$ on 6 cities) and the pilot method, with $PILOT\text{-}NN(I) \leq NN(I)$ (Chapter 21 «Constructive heuristics»).

## 6.5 Local search

- A 2-opt move removes two edges and reverses a path, with $\frac{n(n-3)}{2}$ moves and $\Delta = d_{u_1 u_2} + d_{v_1 v_2} - d_{u_1 v_1} - d_{u_2 v_2}$ evaluated in $O(1)$ (Chapter 22 «Local search»).

- From the $NN$ tour of length 25, 2-opt reaches the optimum 18 in two moves with first and with best improvement (Chapter 22 «Local search»).

- Or-opt moves a segment of $\ell \in \{1, 2, 3\}$ cities: on 10 cities it escapes the 2-opt local optimum 41 and reaches $z(I) = 39$, as does $VND$ with 2-opt and Or-opt (Chapter 22 «Local search»).

- Multi-start 2-opt on 30 cities finds $z(I) = 478$ in 14 of 100 runs, but local search gives no dual bound to recognize it (Chapter 22 «Local search»).

## 6.6 Metaheuristics

- Simulated annealing with the 2-opt neighbourhood and the Metropolis rule reaches 18 on the reference instance and 291 on TSP20 with slow cooling (Chapter 23 «Metaheuristics»).

- Tabu search forbids the two removed edges: on TSP20 every tenure from 1 to 20 reaches 291, tenure 0 stops at 319 (Chapter 23 «Metaheuristics»).

- Iterated local search with the double-bridge kick $(A, B, C, D) \rightarrow (A, C, B, D)$ goes from the 2-opt local optimum 319 to $291 = z(I)$ in two kicks (Chapter 23 «Metaheuristics»).

- GRASP (randomized $NN$ plus 2-opt) and LNS / ALNS (removal of $q$ cities and cheapest-insertion repair) are compared on TSP20 (Chapter 23 «Metaheuristics»).

## 6.7 MIP heuristics

- The symmetric model with $\sum_{\{i,j\}\in\delta(i)} x_{ij} = 2$ and the subtour elimination constraints has an integer LP optimum on the reference instance, so rounding gives 18 with gap 0% (Chapter 24 «MIP heuristics for the classical problems»).

## 6.8 Approximation

- Double tree: $DT(I) \le 2\,d(T) \le 2\,z(I)$, and the ratio 2 is tight (Chapter 28 «$P\|C_{\max}$, metric TSP and the limits of approximation»).
- Christofides: $CH(I) \le d(T) + d(M) \le \frac{3}{2}\,z(I)$, tight; on the reference instance $CH(I) = 18 = z(I)$ and $DT(I) = 22$ (Chapter 28 «$P\|C_{\max}$, metric TSP and the limits of approximation»).
- For the general TSP no polynomial algorithm has $\frac{\mathscr{A}(I)}{z(I)} \le \alpha$ for a constant $\alpha$, unless $P = NP$ (Sahni and Gonzalez, 1976) (Chapter 28 «$P\|C_{\max}$, metric TSP and the limits of approximation»).

## 6.9 Instances

### 6.9.1 Instance TSP1

Reference instance with 5 cities (the TSP instance of the MIP modelling course).
$n = 5$ cities, symmetric distances:

| $d_{ij}$ | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| 1 | – | 5 | 2 | 2 | 9 |
| 2 | 5 | – | 4 | 3 | 4 |
| 3 | 2 | 4 | – | 4 | 7 |
| 4 | 2 | 3 | 4 | – | 7 |
| 5 | 9 | 4 | 7 | 7 | – |

- Optimal value $z(\text{TSP1}) = 18$, attained by $\tilde{\sigma} = (1, 3, 5, 2, 4)$, unique; NN from 1: 25; LB $= 17$; $DT(I) = 22$, $CH(I) = 18$.
- Used in Example «a small TSP instance», Example «counting the tours», Example «bounds on the small instances», Example «TSP instance», Example «enumeration of the tours», Example «nearest neighbour from city 1», Example «the bound on the reference instance», Example «the instance of the chapter», Example «$\mathrm{ILP}^{\mathrm{MTZ}}_{\mathrm{TSP}}$ model», Example «$\mathrm{ILP}^{\mathrm{DFJ}}_{\mathrm{TSP}}$ model», Example «comparing the relaxations», Example «$DT$ on the TSP reference instance», Example «$CH$ on the TSP reference instance», Example «TSP reference instance», Example «Bounds and gaps on the reference instances», Proposition «two-edge lower bound», How to evaluate a heuristic algorithm, Example «NN on the TSP reference instance», Example «Multi-start NN on the TSP reference instance», Example «NI and CI on the TSP reference instance», Example «FI on the TSP reference instance», Example «$RNN_{0.3}$ on the TSP reference instance», Example «PILOT-NN on the TSP reference instance», What constructive heuristics give, and what they do not, Proposition «termination of local search», Example «2OPT on the TSP reference instance», Example

«SA with the 2-opt neighbourhood on the TSP reference instance», Example «LNS on the TSP reference instance», The LP relaxations on the reference instances, What the MIP-based heuristics give.

### 6.9.2 Instance TSP2

4 cities, ground set of the edges.
$n = 4$ cities, symmetric distances:

| $d_{ij}$ | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 | – | 3 | 6 | 4 |
| 2 | 3 | – | 5 | 2 |
| 3 | 6 | 5 | – | 7 |
| 4 | 4 | 2 | 7 | – |

- Optimal value $z(\text{TSP2}) = 17$, attained by $\tilde{\sigma} = (1, 3, 2, 4)$.
- Used in Exercise 1.1.

### 6.9.3 Instance TSP3

4 cities, the tours of a small TSP.
$n = 4$ cities, symmetric distances:

| $d_{ij}$ | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 | – | 4 | 6 | 5 |
| 2 | 4 | – | 3 | 7 |
| 3 | 6 | 3 | – | 2 |
| 4 | 5 | 7 | 2 | – |

- Optimal value $z(\text{TSP3}) = 14$, attained by $(1, 2, 3, 4)$.
- Used in Exercise «The tours of a small TSP».

### 6.9.4 Instance TSP4

5 cities, primal bound, dual bound and gap.
$n = 5$ cities, symmetric distances:

| $d_{ij}$ | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| 1 | – | 2 | 2 | 9 | 9 |
| 2 | 2 | – | 4 | 5 | 9 |
| 3 | 2 | 4 | – | 5 | 4 |
| 4 | 9 | 5 | 5 | – | 8 |
| 5 | 9 | 9 | 4 | 8 | – |

- Optimal value $z(\text{TSP4}) = 21$, attained by $\tilde{\sigma} = (1, 2, 4, 5, 3)$; NN gives 27, LB $= 19$.
- Used in Exercise 1.5.

### 6.9.5 Instance TSP5

6 cities, tours and their length.
$n = 6$ cities, symmetric distances:

| $d_{ij}$ | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| 1 | – | 4 | 7 | 6 | 3 | 5 |
| 2 | 4 | – | 3 | 5 | 6 | 8 |
| 3 | 7 | 3 | – | 4 | 7 | 6 |
| 4 | 6 | 5 | 4 | – | 3 | 4 |
| 5 | 3 | 6 | 7 | 3 | – | 5 |
| 6 | 5 | 8 | 6 | 4 | 5 | – |

- Optimal value $z(\text{TSP5}) = 23$.
- Used in Exercise 6.1.

### 6.9.6 Instance TSP6

5 cities with a far city 2.
$n = 5$ cities, symmetric distances:

| $d_{ij}$ | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| 1 | – | 12 | 3 | 5 | 3 |
| 2 | 12 | – | 9 | 7 | 10 |
| 3 | 3 | 9 | – | 4 | 5 |
| 4 | 5 | 7 | 4 | – | 4 |
| 5 | 3 | 10 | 5 | 4 | – |

- Optimal value $z(\text{TSP6}) = 26$, attained by $\tilde{\sigma} = (1, 3, 2, 4, 5)$, unique; NN from 1 gives 33.
- Used in Exercise 6.2, Exercise 6.3.

### 6.9.7 Instance TSP7

4 cities, not metric ($d_{13} = 10$).
$n = 4$ cities, symmetric distances:

| $d_{ij}$ | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 | – | 4 | 10 | 3 |
| 2 | 4 | – | 5 | 2 |
| 3 | 10 | 5 | – | 6 |
| 4 | 3 | 2 | 6 | – |

- Optimal value $z(\text{TSP7}) = 18$, attained by $(1, 2, 3, 4)$.
- Used in Exercise 6.4.

### 6.9.8 Instance TSP8

4 cities, the metric version with $d_{13} = 9$.
$n = 4$ cities, symmetric distances:

| $d_{ij}$ | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 | – | 4 | 9 | 3 |
| 2 | 4 | – | 5 | 2 |
| 3 | 9 | 5 | – | 6 |
| 4 | 3 | 2 | 6 | – |

- Optimal value $z(\text{TSP8}) = 18$, attained by $(1, 2, 3, 4)$.
- Used in Exercise 6.4.

### 6.9.9 Instance TSP9

4 cities for the MTZ and DFJ models.
$n = 4$ cities, symmetric distances:

| $d_{ij}$ | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 | – | 3 | 5 | 4 |
| 2 | 3 | – | 4 | 6 |
| 3 | 5 | 4 | – | 2 |
| 4 | 4 | 6 | 2 | – |

- Optimal value $z(\text{TSP9}) = 13$, attained by $(1, 2, 3, 4)$.
- Used in Exercise 13.1, Exercise 13.2.

### 6.9.10 Instance TSP10

5 cities, feasible solutions of the TSP models.
$n = 5$ cities, symmetric distances:

| $d_{ij}$ | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| 1 | – | 2 | 3 | 3 | 7 |
| 2 | 2 | – | 4 | 12 | 6 |
| 3 | 3 | 4 | – | 6 | 11 |
| 4 | 3 | 12 | 6 | – | 5 |
| 5 | 7 | 6 | 11 | 5 | – |

- Optimal value $z(\text{TSP10}) = 21$, attained by $(1, 3, 2, 5, 4)$.
- Used in Exercise 13.3.

### 6.9.11 Instance TSP11

5 cities, comparing the relaxations.

$n = 5$ cities, symmetric distances:

| $d_{ij}$ | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| 1 | – | 5 | 12 | 2 | 9 |
| 2 | 5 | – | 2 | 8 | 12 |
| 3 | 12 | 2 | – | 9 | 9 |
| 4 | 2 | 8 | 9 | – | 5 |
| 5 | 9 | 12 | 9 | 5 | – |

- Optimal value $z(\text{TSP11}) = 23$, attained by $(1, 2, 3, 5, 4)$; bounds 20, $\frac{43}{2}$, 23.
- Used in Exercise 13.4.

### 6.9.12 Instance TSP12

5 cities, adding subtour elimination constraints.

$n = 5$ cities, symmetric distances:

| $d_{ij}$ | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| 1 | – | 4 | 10 | 2 | 8 |
| 2 | 4 | – | 11 | 2 | 5 |
| 3 | 10 | 11 | – | 4 | 2 |
| 4 | 2 | 2 | 4 | – | 2 |
| 5 | 8 | 5 | 2 | 2 | – |

- Optimal value $z(\text{TSP12}) = 17$, attained by $(1, 2, 5, 3, 4)$; $z(\text{LP}^{\text{MTZ}}_{\text{TSP}}) = 14$.
- Used in Exercise 13.5, Exercise 20.3.

### 6.9.13 Instance TSP13

6 points in the plane, metric approximation 1.

$n = 6$ cities, symmetric distances:

| $d_{ij}$ | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| 1 | – | 4 | 4 | 6 | 3 | 6 |
| 2 | 4 | – | 8 | 7 | 7 | 2 |
| 3 | 4 | 8 | – | 5 | 5 | 9 |
| 4 | 6 | 7 | 5 | – | 9 | 8 |
| 5 | 3 | 7 | 5 | 9 | – | 9 |
| 6 | 6 | 2 | 9 | 8 | 9 | – |

- Optimal value $z(\text{TSP13}) = 27$, attained by $(1, 2, 6, 4, 3, 5)$, unique; $DT(I) = 32$, $CH(I) = 27$.
- Used in Exercise 28.4.

### 6.9.14 Instance TSP14

6 points in the plane, metric approximation 2.
$n = 6$ cities, symmetric distances:

| $d_{ij}$ | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| 1 | – | 6 | 5 | 4 | 4 | 3 |
| 2 | 6 | – | 3 | 9 | 6 | 4 |
| 3 | 5 | 3 | – | 8 | 7 | 5 |
| 4 | 4 | 9 | 8 | – | 6 | 6 |
| 5 | 4 | 6 | 7 | 6 | – | 2 |
| 6 | 3 | 4 | 5 | 6 | 2 | – |

- Optimal value $z(\text{TSP14}) = 24$, attained by $(1, 3, 2, 6, 5, 4)$, unique; $CH(I) = 25$, $DT(I) = 28$.
- Used in Exercise 28.5.

### 6.9.15 Instance TSP15

$I_{G_1}$: Hamiltonian graph $G_1$, non-edges at distance 11.
$n = 5$ cities, symmetric distances:

| $d_{ij}$ | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| 1 | – | 1 | 1 | 11 | 1 |
| 2 | 1 | – | 1 | 11 | 11 |
| 3 | 1 | 1 | – | 1 | 11 |
| 4 | 11 | 11 | 1 | – | 1 |
| 5 | 1 | 11 | 11 | 1 | – |

- Optimal value $z(\text{TSP15}) = 5$, attained by $(1, 2, 3, 4, 5)$.
- Used in Exercise 28.6.

### 6.9.16 Instance TSP16

$I_{G_2}$: $K_{2,3}$, non-edges at distance 11 (not metric).
$n = 5$ cities, symmetric distances:

| $d_{ij}$ | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| 1 | – | 11 | 1 | 1 | 1 |
| 2 | 11 | – | 1 | 1 | 1 |
| 3 | 1 | 1 | – | 11 | 11 |
| 4 | 1 | 1 | 11 | – | 11 |
| 5 | 1 | 1 | 11 | 11 | – |

- Optimal value $z(\text{TSP16}) = 15$, attained by $(1, 3, 2, 4, 5)$; tours of length 15 or 35 only.
- Used in Exercise 28.6.

### 6.9.17 Instance TSP17

Tight family of $DT$ for $n = 5$ ($a = 5$, $b = 6$, $P = (3, 5, 2, 4)$).
$n = 5$ cities, symmetric distances:

| $d_{ij}$ | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| 1 | – | 5 | 5 | 5 | 5 |
| 2 | 5 | – | 10 | 6 | 6 |
| 3 | 5 | 10 | – | 10 | 6 |
| 4 | 5 | 6 | 10 | – | 10 |
| 5 | 5 | 6 | 6 | 10 | – |

- Optimal value $z(\text{TSP17}) = 28$, attained by $(1, 3, 5, 2, 4)$; $DT(I_5) = 40$.
- Used in Proposition (tightness of $DT$).

### 6.9.18 Instance TSP18

Tight family of $CH$ for $n = 5$ (shortest paths in a strip of triangles, $a = 5$, $b = 6$).
$n = 5$ cities, symmetric distances:

| $d_{ij}$ | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| 1 | – | 5 | 6 | 11 | 12 |
| 2 | 5 | – | 5 | 6 | 11 |
| 3 | 6 | 5 | – | 5 | 6 |
| 4 | 11 | 6 | 5 | – | 5 |
| 5 | 12 | 11 | 6 | 5 | – |

- Optimal value $z(\text{TSP18}) = 28$, attained by $CH(I_5) = 32$.
- Used in Proposition (tightness of $CH$).

### 6.9.19 Instance TSP19

6 cities in the plane (comparison of constructive heuristics).
$n = 6$ cities, symmetric distances:

| $d_{ij}$ | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| 1 | – | 5 | 4 | 4 | 5 | 2 |
| 2 | 5 | – | 1 | 5 | 5 | 4 |
| 3 | 4 | 1 | – | 4 | 5 | 3 |
| 4 | 4 | 5 | 4 | – | 9 | 6 |
| 5 | 5 | 5 | 5 | 9 | – | 3 |
| 6 | 2 | 4 | 3 | 6 | 3 | – |

- Optimal value $z(\text{TSP19}) = 19$, attained by (1,4,3,2,5,6).
- Used in Example «A TSP instance with 6 cities», Example «$REG_2$ against CI on the 6-city instance», Example «PILOT-NN on the TSP reference instance».

### 6.9.20 Instance TSP20

10 cities in the plane (2-opt / Or-opt / VND / multi-start).
$n = 10$ cities, symmetric distances:

| $d_{ij}$ | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|---|---|---|---|---|---|---|---|---|---|---|
| 1 | – | 7 | 10 | 12 | 13 | 11 | 7 | 5 | 9 | 4 |
| 2 | 7 | – | 4 | 7 | 10 | 11 | 7 | 4 | 5 | 3 |
| 3 | 10 | 4 | – | 4 | 8 | 11 | 7 | 6 | 4 | 6 |
| 4 | 12 | 7 | 4 | – | 4 | 8 | 6 | 7 | 3 | 9 |
| 5 | 13 | 10 | 8 | 4 | – | 5 | 6 | 8 | 5 | 11 |
| 6 | 11 | 11 | 11 | 8 | 5 | – | 4 | 8 | 7 | 11 |
| 7 | 7 | 7 | 7 | 6 | 6 | 4 | – | 3 | 4 | 6 |
| 8 | 5 | 4 | 6 | 7 | 8 | 8 | 3 | – | 4 | 3 |
| 9 | 9 | 5 | 4 | 3 | 5 | 7 | 4 | 4 | – | 6 |
| 10 | 4 | 3 | 6 | 9 | 11 | 11 | 6 | 3 | 6 | – |

- Optimal value $z(\text{TSP20}) = 39$, attained by (1,10,2,3,9,4,5,6,7,8).
- Used in Example «A TSP instance with 10 cities», Example «OROPT from a 2-opt local optimum», Example «VND on the TSP with 2-opt and Or-opt», Example «MSLS on the TSP with 10 cities».

### 6.9.21 Instance TSP21

30 random cities (TSP30, multi-start 2-opt).

$n = 30$ cities, symmetric distances:

| $d_{ij}$ | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | – | 118 | 79 | 72 | 83 | 69 | 17 | 113 | 85 | 70 | 100 | 102 |
| 2 | 118 | – | 94 | 59 | 76 | 64 | 103 | 13 | 61 | 70 | 20 | 21 |
| 3 | 79 | 94 | – | 93 | 117 | 35 | 79 | 99 | 33 | 27 | 87 | 93 |
| 4 | 72 | 59 | 93 | – | 25 | 59 | 55 | 49 | 71 | 67 | 39 | 38 |
| 5 | 83 | 76 | 117 | 25 | – | 84 | 66 | 64 | 96 | 91 | 58 | 55 |
| 6 | 69 | 64 | 35 | 59 | 84 | – | 61 | 66 | 16 | 8 | 54 | 59 |
| 7 | 17 | 103 | 79 | 55 | 66 | 61 | – | 98 | 77 | 64 | 85 | 87 |
| 8 | 113 | 13 | 99 | 49 | 64 | 66 | 98 | – | 66 | 73 | 13 | 11 |
| 9 | 85 | 61 | 33 | 71 | 96 | 16 | 77 | 66 | – | 15 | 56 | 62 |
| 10 | 70 | 70 | 27 | 67 | 91 | 8 | 64 | 73 | 15 | – | 61 | 66 |
| 11 | 100 | 20 | 87 | 39 | 58 | 54 | 85 | 13 | 56 | 61 | – | 6 |
| 12 | 102 | 21 | 93 | 38 | 55 | 59 | 87 | 11 | 62 | 66 | 6 | – |
| 13 | 85 | 91 | 128 | 38 | 15 | 96 | 68 | 79 | 108 | 103 | 73 | 70 |
| 14 | 30 | 88 | 65 | 47 | 64 | 44 | 17 | 84 | 60 | 47 | 71 | 73 |
| 15 | 77 | 47 | 52 | 48 | 73 | 18 | 66 | 48 | 23 | 25 | 36 | 41 |
| 16 | 97 | 30 | 94 | 30 | 46 | 60 | 80 | 20 | 64 | 67 | 13 | 9 |
| 17 | 91 | 92 | 133 | 42 | 17 | 101 | 75 | 80 | 113 | 108 | 75 | 71 |
| 18 | 116 | 2 | 93 | 57 | 74 | 63 | 101 | 11 | 60 | 69 | 17 | 19 |
| 19 | 84 | 86 | 125 | 34 | 10 | 92 | 68 | 74 | 104 | 99 | 68 | 65 |
| 20 | 79 | 59 | 35 | 64 | 89 | 9 | 71 | 63 | 7 | 12 | 52 | 58 |
| 21 | 66 | 60 | 87 | 7 | 30 | 54 | 49 | 52 | 66 | 61 | 41 | 40 |
| 22 | 69 | 72 | 26 | 68 | 92 | 9 | 63 | 75 | 17 | 2 | 63 | 68 |
| 23 | 105 | 28 | 102 | 36 | 49 | 68 | 89 | 16 | 72 | 76 | 16 | 11 |
| 24 | 46 | 80 | 84 | 27 | 40 | 56 | 28 | 72 | 71 | 61 | 60 | 61 |
| 25 | 62 | 58 | 75 | 18 | 42 | 42 | 46 | 52 | 54 | 49 | 39 | 40 |
| 26 | 38 | 81 | 66 | 39 | 57 | 41 | 23 | 76 | 57 | 46 | 63 | 65 |
| 27 | 25 | 94 | 75 | 47 | 60 | 54 | 9 | 89 | 70 | 57 | 76 | 77 |
| 28 | 31 | 97 | 49 | 65 | 84 | 40 | 30 | 95 | 55 | 40 | 82 | 85 |
| 29 | 27 | 105 | 52 | 72 | 90 | 47 | 31 | 103 | 61 | 46 | 90 | 93 |
| 30 | 19 | 101 | 64 | 61 | 76 | 51 | 16 | 97 | 66 | 52 | 84 | 86 |

| $d_{ij}$ | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 | 21 | 22 | 23 | 24 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 85 | 30 | 77 | 97 | 91 | 116 | 84 | 79 | 66 | 69 | 105 | 46 |
| 2 | 91 | 88 | 47 | 30 | 92 | 2 | 86 | 59 | 60 | 72 | 28 | 80 |
| 3 | 128 | 65 | 52 | 94 | 133 | 93 | 125 | 35 | 87 | 26 | 102 | 84 |
| 4 | 38 | 47 | 48 | 30 | 42 | 57 | 34 | 64 | 7 | 68 | 36 | 27 |
| 5 | 15 | 64 | 73 | 46 | 17 | 74 | 10 | 89 | 30 | 92 | 49 | 40 |
| 6 | 96 | 44 | 18 | 60 | 101 | 63 | 92 | 9 | 54 | 9 | 68 | 56 |
| 7 | 68 | 17 | 66 | 80 | 75 | 101 | 68 | 71 | 49 | 63 | 89 | 28 |
| 8 | 79 | 84 | 48 | 20 | 80 | 11 | 74 | 63 | 52 | 75 | 16 | 72 |
| 9 | 108 | 60 | 23 | 64 | 113 | 60 | 104 | 7 | 66 | 17 | 72 | 71 |
| 10 | 103 | 47 | 25 | 67 | 108 | 69 | 99 | 12 | 61 | 2 | 76 | 61 |
| 11 | 73 | 71 | 36 | 13 | 75 | 17 | 68 | 52 | 41 | 63 | 16 | 60 |
| 12 | 70 | 73 | 41 | 9 | 71 | 19 | 65 | 58 | 40 | 68 | 11 | 61 |
| 13 | – | 70 | 86 | 61 | 6 | 89 | 5 | 102 | 43 | 103 | 63 | 46 |
| 14 | 70 | – | 49 | 68 | 76 | 86 | 68 | 53 | 40 | 46 | 77 | 24 |
| 15 | 86 | 49 | – | 42 | 90 | 45 | 82 | 17 | 43 | 27 | 50 | 52 |
| 16 | 61 | 68 | 42 | – | 63 | 28 | 56 | 60 | 33 | 69 | 9 | 54 |
| 17 | 6 | 76 | 90 | 63 | – | 90 | 8 | 106 | 47 | 108 | 64 | 52 |
| 18 | 89 | 86 | 45 | 28 | 90 | – | 84 | 58 | 58 | 71 | 27 | 78 |
| 19 | 5 | 68 | 82 | 56 | 8 | 84 | – | 97 | 39 | 100 | 58 | 44 |
| 20 | 102 | 53 | 17 | 60 | 106 | 58 | 97 | – | 59 | 14 | 67 | 63 |
| 21 | 43 | 40 | 43 | 33 | 47 | 58 | 39 | 59 | – | 62 | 40 | 21 |
| 22 | 103 | 46 | 27 | 69 | 108 | 71 | 100 | 14 | 62 | – | 77 | 61 |
| 23 | 63 | 77 | 50 | 9 | 64 | 27 | 58 | 67 | 40 | 77 | – | 62 |
| 24 | 46 | 24 | 52 | 54 | 52 | 78 | 44 | 63 | 21 | 61 | 62 | – |
| 25 | 54 | 34 | 33 | 34 | 59 | 56 | 50 | 47 | 12 | 50 | 43 | 22 |
| 26 | 65 | 9 | 43 | 59 | 70 | 78 | 62 | 50 | 32 | 45 | 68 | 19 |
| 27 | 65 | 10 | 57 | 71 | 71 | 92 | 63 | 63 | 41 | 57 | 80 | 21 |
| 28 | 91 | 21 | 52 | 81 | 97 | 95 | 89 | 49 | 58 | 38 | 91 | 45 |
| 29 | 95 | 26 | 60 | 89 | 101 | 103 | 94 | 56 | 65 | 44 | 98 | 50 |
| 30 | 81 | 14 | 59 | 81 | 87 | 99 | 79 | 60 | 54 | 50 | 90 | 36 |

| $d_{ij}$ | 25 | 26 | 27 | 28 | 29 | 30 |
|---|---|---|---|---|---|---|
| 1 | 62 | 38 | 25 | 31 | 27 | 19 |
| 2 | 58 | 81 | 94 | 97 | 105 | 101 |
| 3 | 75 | 66 | 75 | 49 | 52 | 64 |
| 4 | 18 | 39 | 47 | 65 | 72 | 61 |
| 5 | 42 | 57 | 60 | 84 | 90 | 76 |
| 6 | 42 | 41 | 54 | 40 | 47 | 51 |
| 7 | 46 | 23 | 9 | 30 | 31 | 16 |
| 8 | 52 | 76 | 89 | 95 | 103 | 97 |
| 9 | 54 | 57 | 70 | 55 | 61 | 66 |
| 10 | 49 | 46 | 57 | 40 | 46 | 52 |
| 11 | 39 | 63 | 76 | 82 | 90 | 84 |
| 12 | 40 | 65 | 77 | 85 | 93 | 86 |
| 13 | 54 | 65 | 65 | 91 | 95 | 81 |
| 14 | 34 | 9 | 10 | 21 | 26 | 14 |
| 15 | 33 | 43 | 57 | 52 | 60 | 59 |
| 16 | 34 | 59 | 71 | 81 | 89 | 81 |
| 17 | 59 | 70 | 71 | 97 | 101 | 87 |
| 18 | 56 | 78 | 92 | 95 | 103 | 99 |
| 19 | 50 | 62 | 63 | 89 | 94 | 79 |
| 20 | 47 | 50 | 63 | 49 | 56 | 60 |
| 21 | 12 | 32 | 41 | 58 | 65 | 54 |
| 22 | 50 | 45 | 57 | 38 | 44 | 50 |
| 23 | 43 | 68 | 80 | 91 | 98 | 90 |
| 24 | 22 | 19 | 21 | 45 | 50 | 36 |
| 25 | – | 25 | 37 | 49 | 56 | 47 |
| 26 | 25 | – | 14 | 27 | 33 | 22 |
| 27 | 37 | 14 | – | 28 | 31 | 16 |
| 28 | 49 | 27 | 28 | – | 8 | 15 |
| 29 | 56 | 33 | 31 | 8 | – | 16 |
| 30 | 47 | 22 | 16 | 15 | 16 | – |

- Optimal value $z(\text{TSP21}) = 478$.
- Used in Example «MSLS on the TSP with 30 cities», The limits of multi-start, What local search gives, and what it does not.

### 6.9.22 Instance TSP22

TSP20, medium instance of the metaheuristics chapter.

$n = 20$ cities, symmetric distances:

| $d_{ij}$ | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | – | 58 | 33 | 55 | 79 | 83 | 30 | 53 | 72 | 86 | 38 | 88 |
| 2 | 58 | – | 33 | 14 | 28 | 37 | 29 | 18 | 35 | 28 | 42 | 37 |
| 3 | 33 | 33 | – | 39 | 48 | 51 | 9 | 22 | 40 | 59 | 44 | 56 |
| 4 | 55 | 14 | 39 | – | 42 | 51 | 32 | 31 | 49 | 37 | 30 | 50 |
| 5 | 79 | 28 | 48 | 42 | – | 10 | 49 | 26 | 18 | 20 | 70 | 9 |
| 6 | 83 | 37 | 51 | 51 | 10 | – | 54 | 31 | 14 | 29 | 78 | 9 |
| 7 | 30 | 29 | 9 | 32 | 49 | 54 | – | 23 | 44 | 57 | 35 | 58 |
| 8 | 53 | 18 | 22 | 31 | 26 | 31 | 23 | – | 22 | 38 | 50 | 35 |
| 9 | 72 | 35 | 40 | 49 | 18 | 14 | 44 | 22 | – | 38 | 72 | 22 |
| 10 | 86 | 28 | 59 | 37 | 20 | 29 | 57 | 38 | 38 | – | 67 | 23 |
| 11 | 38 | 42 | 44 | 30 | 70 | 78 | 35 | 50 | 72 | 67 | – | 79 |
| 12 | 88 | 37 | 56 | 50 | 9 | 9 | 58 | 35 | 22 | 23 | 79 | – |
| 13 | 46 | 21 | 15 | 31 | 33 | 37 | 16 | 7 | 28 | 44 | 46 | 42 |
| 14 | 92 | 36 | 63 | 46 | 17 | 22 | 62 | 41 | 34 | 11 | 77 | 14 |
| 15 | 23 | 53 | 20 | 57 | 67 | 69 | 25 | 42 | 56 | 79 | 52 | 75 |
| 16 | 21 | 45 | 34 | 38 | 71 | 77 | 26 | 47 | 69 | 73 | 17 | 80 |
| 17 | 63 | 10 | 34 | 24 | 19 | 27 | 33 | 14 | 26 | 24 | 51 | 28 |
| 18 | 36 | 27 | 6 | 33 | 43 | 47 | 7 | 17 | 37 | 53 | 41 | 52 |
| 19 | 35 | 23 | 22 | 21 | 49 | 56 | 13 | 26 | 49 | 51 | 24 | 58 |
| 20 | 93 | 36 | 65 | 45 | 22 | 29 | 64 | 44 | 40 | 8 | 75 | 21 |

| $d_{ij}$ | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 |
|---|---|---|---|---|---|---|---|---|
| 1 | 46 | 92 | 23 | 21 | 63 | 36 | 35 | 93 |
| 2 | 21 | 36 | 53 | 45 | 10 | 27 | 23 | 36 |
| 3 | 15 | 63 | 20 | 34 | 34 | 6 | 22 | 65 |
| 4 | 31 | 46 | 57 | 38 | 24 | 33 | 21 | 45 |
| 5 | 33 | 17 | 67 | 71 | 19 | 43 | 49 | 22 |
| 6 | 37 | 22 | 69 | 77 | 27 | 47 | 56 | 29 |
| 7 | 16 | 62 | 25 | 26 | 33 | 7 | 13 | 64 |
| 8 | 7 | 41 | 42 | 47 | 14 | 17 | 26 | 44 |
| 9 | 28 | 34 | 56 | 69 | 26 | 37 | 49 | 40 |
| 10 | 44 | 11 | 79 | 73 | 24 | 53 | 51 | 8 |
| 11 | 46 | 77 | 52 | 17 | 51 | 41 | 24 | 75 |
| 12 | 42 | 14 | 75 | 80 | 28 | 52 | 58 | 21 |
| 13 | – | 48 | 35 | 42 | 20 | 10 | 22 | 51 |
| 14 | 48 | – | 82 | 81 | 29 | 57 | 59 | 7 |
| 15 | 35 | 82 | – | 37 | 55 | 26 | 37 | 85 |
| 16 | 42 | 81 | 37 | – | 53 | 33 | 22 | 81 |
| 17 | 20 | 29 | 55 | 53 | – | 29 | 30 | 31 |
| 18 | 10 | 57 | 26 | 33 | 29 | – | 17 | 60 |
| 19 | 22 | 59 | 37 | 22 | 30 | 17 | – | 59 |
| 20 | 51 | 7 | 85 | 81 | 31 | 60 | 59 | – |

- Optimal value $z(\text{TSP22}) = 291$.

- Used in Example «The medium TSP instance (TSP20)», An experiment on the medium TSP instance, Tabu search for the TSP, and the effect of the tenure, Example «ILS on the medium TSP instance», GRASP for the TSP, and the effect of $\alpha$, Adaptive large neighbourhood search, The methods on the medium instances, In summary.

### 6.9.23 Instance TSP23

6 cities in the plane, NN / multi-start exercise.
$n = 6$ cities, symmetric distances:

| $d_{ij}$ | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| 1 | – | 7 | 7 | 9 | 4 | 9 |
| 2 | 7 | – | 2 | 5 | 6 | 4 |
| 3 | 7 | 2 | – | 3 | 5 | 2 |
| 4 | 9 | 5 | 3 | – | 5 | 1 |
| 5 | 4 | 6 | 5 | 5 | – | 6 |
| 6 | 9 | 4 | 2 | 1 | 6 | – |

- Optimal value $z(\text{TSP23}) = 21$, attained by (2,3,6,4,5,1) [= (1,2,3,6,4,5); stated as the multi-start NN tour, of length z=21].
- Used in Exercise 21.10.

### 6.9.24 Instance TSP24

6 cities in the plane, insertion heuristics exercise.
$n = 6$ cities, symmetric distances:

| $d_{ij}$ | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| 1 | – | 3 | 4 | 4 | 2 | 3 |
| 2 | 3 | – | 7 | 6 | 4 | 2 |
| 3 | 4 | 7 | – | 4 | 4 | 6 |
| 4 | 4 | 6 | 4 | – | 6 | 4 |
| 5 | 2 | 4 | 4 | 6 | – | 5 |
| 6 | 3 | 2 | 6 | 4 | 5 | – |

- Optimal value $z(\text{TSP24}) = 19$, attained by (1,2,6,4,3,5) [the FI tour, of length z=19].
- Used in Exercise 21.11.

### 6.9.25 Instance TSP25

5 cities in the plane, randomized NN exercise.
$n = 5$ cities, symmetric distances:

| $d_{ij}$ | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| 1 | – | 3 | 4 | 3 | 6 |
| 2 | 3 | – | 4 | 3 | 5 |
| 3 | 4 | 4 | – | 1 | 3 |
| 4 | 3 | 3 | 1 | – | 4 |
| 5 | 6 | 5 | 3 | 4 | – |

- Optimal value $z(\text{TSP25}) = 15$, attained by (1,4,3,5,2) [second RNN run, of length z=15].
- Used in Exercise 21.12.

### 6.9.26 Instance TSP26

6 cities, 2-opt best improvement exercise.
$n = 6$ cities, symmetric distances:

| $d_{ij}$ | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| 1 | – | 7 | 6 | 3 | 4 | 6 |
| 2 | 7 | – | 6 | 5 | 7 | 4 |
| 3 | 6 | 6 | – | 5 | 5 | 2 |
| 4 | 3 | 5 | 5 | – | 5 | 3 |
| 5 | 4 | 7 | 5 | 5 | – | 4 |
| 6 | 6 | 4 | 2 | 3 | 4 | – |

- Optimal value $z(\text{TSP26}) = 23$, attained by (1,4,2,6,3,5) [final 2-opt tour, stated optimal].
- Used in Exercise 22.7.

### 6.9.27 Instance TSP27

6 cities, 2-opt first vs best improvement exercise.
$n = 6$ cities, symmetric distances:

| $d_{ij}$ | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| 1 | – | 8 | 5 | 6 | 5 | 6 |
| 2 | 8 | – | 6 | 7 | 7 | 6 |
| 3 | 5 | 6 | – | 4 | 8 | 2 |
| 4 | 6 | 7 | 4 | – | 6 | 5 |
| 5 | 5 | 7 | 8 | 6 | – | 7 |
| 6 | 6 | 6 | 2 | 5 | 7 | – |

- Optimal value $z(\text{TSP27}) = 30$, attained by (1,5,2,6,3,4) [first-improvement local optimum, stated optimal].
- Used in Exercise 22.8.

### 6.9.28 Instance TSP28

6 cities, 2-opt local optimum + Or-opt exercise.
$n = 6$ cities, symmetric distances:

| $d_{ij}$ | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| 1 | – | 7 | 8 | 8 | 9 | 9 |
| 2 | 7 | – | 7 | 4 | 9 | 8 |
| 3 | 8 | 7 | – | 7 | 3 | 9 |
| 4 | 8 | 4 | 7 | – | 5 | 6 |
| 5 | 9 | 9 | 3 | 5 | – | 7 |
| 6 | 9 | 8 | 9 | 6 | 7 | – |

- Optimal value $z(\text{TSP28}) = 35$, attained by (1,2,4,6,5,3) [after the Or-opt move, stated optimal].
- Used in Exercise 22.9.

### 6.9.29 Instance TSP29

6 cities, common instance of the metaheuristics exercises (SA, TS, ILS, GRASP, LNS).
$n = 6$ cities, symmetric distances:

| $d_{ij}$ | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| 1 | – | 5 | 1 | 7 | 7 | 4 |
| 2 | 5 | – | 4 | 5 | 2 | 1 |
| 3 | 1 | 4 | – | 6 | 6 | 4 |
| 4 | 7 | 5 | 6 | – | 7 | 6 |
| 5 | 7 | 2 | 6 | 7 | – | 2 |
| 6 | 4 | 1 | 4 | 6 | 2 | – |

- Optimal value $z(\text{TSP29}) = 20$, attained by (1,3,4,2,5,6).
- Used in Exercise 23.1, Exercise 23.3, Exercise 23.6, Exercise 23.7, Exercise 23.8, Exercise 23.11, Exercise 23.15.

## 6.10 Exercises

- Ground set, feasible solutions and objective (TSP)
- Counting the solutions
- Primal bound, dual bound and gap (TSP)
- Tours and their length (TSP)
- Enumeration of the tours (TSP)
- Nearest neighbour from every city and a lower bound (TSP)
- The triangle inequality (metric TSP)
- Asymmetric distances (ATSP)
- The MTZ model of the TSP (1)
- The DFJ model of the TSP (1)
- Feasible solutions of the TSP models (1)
- Comparing the TSP relaxations (1)
- Adding subtour elimination constraints (1)
- Metric TSP approximation 1
- Metric TSP approximation 2

- Metric TSP approximation 3
- A dual bound for the TSP (1)
- TSP construction 1
- TSP construction 2
- TSP construction 3
- Local search TSP 1
- Local search TSP 2
- Local search TSP 3
- Simulated annealing 1
- Simulated annealing 3
- Tabu search 3
- Iterated local search and VNS 1
- Iterated local search and VNS 2
- GRASP 2
- Large neighbourhood search 3

## 6.11 Metric travelling salesman problem (metric TSP)

**Definition 6.2: Metric travelling salesman problem**

A TSP instance is **metric** if its distances satisfy the **triangle inequality**

$$d_{ij} \leq d_{ik} + d_{kj} \qquad \text{for all distinct } i, j, k \in \mathscr{V}.$$

- Defined in Chapter 6 «The travelling salesman problem».
- The special case of the TSP with the triangle inequality, where approximation guarantees exist.
- **Complexity**: Approximable within $\frac{3}{2}$ (Christofides, 1976; Serdyukov, 1978).

**Where it is treated**

- Chapter 7 «The travelling salesman problem»: Definition; the reference instance is metric
- Chapter 28 «$P||C_{\max}$, metric TSP and the limits of approximation»: Definition in the approximation chapter
- Chapter 28 «$P||C_{\max}$, metric TSP and the limits of approximation»: $DT$ is a 2-approximation, tight

- Chapter 28 «$P||C_{\max}$, metric TSP and the limits of approximation»: $CH$ is a $\frac{3}{2}$-approximation, tight
- Exercise 7.4: Checking the triangle inequality
- Exercise 28.4:
- Exercise 28.5:

## 6.12 Asymmetric travelling salesman problem (ATSP)

**Definition 6.3: Asymmetric travelling salesman problem**

In the **asymmetric travelling salesman problem** (ATSP) the distances are not required to be symmetric: $d_{ij}$ may differ from $d_{ji}$. The definition of tour and of its length $L(\sigma)$ is unchanged, but a tour and the same tour travelled backwards are different solutions.

- Defined in Chapter 6 «The travelling salesman problem».
- The TSP is the special case of the ATSP in which $d_{ij} = d_{ji}$ for all $i, j$; an ATSP instance has $(n-1)!$ tours.

**Where it is treated**

- Chapter 7 «The travelling salesman problem»: Definition
- Chapter 7 «The travelling salesman problem»: $(n-1)!$ tours
- Exercise 7.5: An ATSP instance with 4 cities

### 6.12.1 Instance ATSP1

4 cities with asymmetric distances.

$n = 4$ cities, asymmetric distances:

| $d_{ij}$ | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 | – | 3 | 8 | 5 |
| 2 | 6 | – | 2 | 9 |
| 3 | 4 | 7 | – | 3 |
| 4 | 2 | 6 | 5 | – |

- Optimal value $z(\text{ATSP1}) = 10$, attained by its reverse has length 23.
- Used in Exercise 6.5.

## 6.13 Hamiltonian cycle problem (HCP)

**Definition 6.4: Hamiltonian cycle problem**

The **Hamiltonian cycle problem** asks whether a graph $G = (V, E)$ has a cycle through all its vertices.

- Defined in Chapter 28 «$P||C_{\max}$, metric TSP and the limits of approximation».
- With $d_{ij} = 1$ on the edges and a large distance elsewhere, $G$ is Hamiltonian if and only if $z(I_G) = n$: the TSP contains it.
- **Complexity**: NP-complete (Karp, 1972).

**Where it is treated**

- Chapter 1 «Combinatorial optimization problems»: Why the TSP is strongly NP-hard
- Chapter 28 «$P||C_{\max}$, metric TSP and the limits of approximation»: The reduction behind the inapproximability of the general TSP
- Exercise 28.6:

### Chapter 7

# Scheduling on identical parallel machines

Assign jobs to $m$ identical machines so that the makespan, the largest machine load, is minimum.

**Where it is treated**

- Chapter 1 «Combinatorial optimization problems»: One of the six problems of the course, with the reference instance, the count of the assignments and the first bounds
- Chapter 7 «Parallel machine scheduling»: Definition, three-field notation, counting the schedules, a first schedule by hand, the lower bound and the relation with bin packing
- Chapter 14 «Formulations of parallel machine scheduling»: The ILP model $\mathrm{ILP}_{P\|C_{\max}}$, its LP relaxation and dual, the ratio $1/m$, valid inequalities and symmetry
- Chapter 20 «Heuristic algorithms: definitions»: Reference instance of the heuristic chapters, gap and relative error of a schedule
- Chapter 21 «Constructive heuristics»: List scheduling $LIST$ and $LPT$, with Graham's guarantees
- Chapter 22 «Local search»: Move and swap neighbourhoods, plateaus and the auxiliary objective $\sum_h L_h^2$, VND and multi-start
- Chapter 23 «Metaheuristics»: Tabu search with the lexicographic comparison $(C_{\max}, \sum_h L_h^2)$
- Chapter 23 «Metaheuristics»: Variable neighbourhood search, on the reference instance and on the medium instance PCM15
- Chapter 24 «MIP heuristics for the classical problems»: Rounding, diving, feasibility pump, RINS, local branching and proximity search on the assignment model
- Chapter 25 «Approximation algorithms: definitions»: Exercise on the algorithm ONE, an $m$-approximation with a tight ratio
- Chapter 28 «$P\|C_{\max}$, metric TSP and the limits of approximation»: Guarantees of $LIST$ and $LPT$, proved, with Graham's tight families

## 7.1 Definition

**Definition 7.1: Scheduling on identical parallel machines**

Given:

1. a set of $m$ identical *machines*,
2. a set $\mathscr{J} = \{1, 2, \dots, n\}$ of $n$ *jobs*,
3. for each job $j \in \mathscr{J}$, a *processing time* $t_j \in \mathbb{Z}_{\geq 1}$,

the problem $P\,||\,C_{\max}$ asks to assign each job to a machine, so that the time needed to complete all the jobs, i.e., the largest total processing time assigned to a machine, is minimized. With $J_h$ the set of jobs of machine $h$ and $L_h = \sum_{j \in J_h} t_j$ its load, the problem reads

$$\min \left\{ \max_{h \in \{1,\dots,m\}} \sum_{j \in J_h} t_j \;:\; J_1, J_2, \dots, J_m \text{ is a partition of } \mathscr{J} \right\}.$$

- Defined in Chapter 7 «Parallel machine scheduling».
- It models the balancing of work among identical resources (machines, processors, workers), where the plan ends when the last resource finishes.
- A solution is an **assignment** $\text{mach}_j = h$ of the jobs to the machines; since the machines are identical, a schedule is a partition of $\mathscr{J}$ into at most $m$ sets $J_1, \dots, J_m$.
- The order of the jobs on a machine does not change its load, so the makespan $C_{\max} = \max_h L_h$ depends only on the sets $J_h$.

## 7.2 Bounds and complexity

- **Complexity**: Strongly NP-hard when $m$ is part of the input; weakly NP-hard for a fixed number of machines, e.g. $m = 2$.
- $z(I) \geq \max\left\{\max_{j \in \mathscr{J}} t_j,\ \left\lceil \frac{1}{m} \sum_{j \in \mathscr{J}} t_j \right\rceil\right\}$: the longest job and the rounded average load (Chapter 7 «Parallel machine scheduling»).
- $z(\text{LP}_{P||C_{\max}}) = \frac{1}{m} \sum_{j \in \mathscr{J}} t_j$ (LP bound, at least $1/m$ of the optimum) (Chapter 14 «Formulations of parallel machine scheduling»).
- There are $m^n$ assignments but only $\sum_{k=1}^{m} S(n,k)$ distinct schedules, with $S(n,k)$ the Stirling numbers of the second kind (Chapter 7 «Parallel machine scheduling»).
- A schedule of makespan at most $T$ exists if and only if the jobs, as items of weight $t_j$, fit into $m$ bins of capacity $T$: $P\,||\,C_{\max}$ fixes the number of bins and minimizes the capacity (Chapter 7 «Parallel machine scheduling»).
- The bound $\max\{\max_j t_j, \lceil \sum_j t_j / m \rceil\}$ is not always attained: $(4, 3, 3)$ on two machines gives 5, while the optimum is 6.

## 7.3 Formulation

- **ILP model of $P\,||\,C_{\max}$ ($\mathrm{ILP}_{P||C_{\max}}$), with $\mathscr{H} = \{1, 2, \ldots, m\}$** (Chapter 14 «Formulations of parallel machine scheduling»):

$$\begin{aligned} &\min C_{\max} \\ \text{subject to } &\sum_{h\in\mathscr{H}} x_{jh} = 1, && \forall j \in \mathscr{J}, \\ &-\sum_{j\in\mathscr{J}} t_j x_{jh} + C_{\max} \geq 0, && \forall h \in \mathscr{H}, \\ &x_{jh} \in \{0,1\}, && \forall j \in \mathscr{J},\ \forall h \in \mathscr{H}, \\ &C_{\max} \geq 0. \end{aligned}$$

- The model $\mathrm{ILP}_{P||C_{\max}}$ has $nm$ binary variables $x_{jh}$, the continuous variable $C_{\max}$, $n$ assignment constraints and $m$ load constraints $C_{\max} \geq \sum_j t_j x_{jh}$ (Chapter 14 «Formulations of parallel machine scheduling»).
- The LP relaxation has value $z(\mathrm{LP}_{P||C_{\max}}) = \sum_j t_j/m$, attained by $x_{jh} = 1/m$, with the dual solution $\pi_h = 1/m$, $\mu_j = t_j/m$ (Chapter 14 «Formulations of parallel machine scheduling»).
- The ratio $z(\mathrm{LP}_{P||C_{\max}})/z(\mathrm{ILP}_{P||C_{\max}})$ is at least $1/m$, and one long job of time $M$ with $m$ unit jobs makes it tend to $1/m$ (Chapter 14 «Formulations of parallel machine scheduling»).
- The valid inequalities $C_{\max} \geq t_j$ and $C_{\max} \geq \lceil \sum_j t_j/m \rceil$ strengthen the model; the load ordering constraints break the symmetry only when the loads are distinct (Chapter 14 «Formulations of parallel machine scheduling»).

## 7.4 Constructive heuristics

- *LIST* assigns the jobs, in the order of a list, to a least loaded machine, in $O(nm)$ or $O(n \log m)$ time; on the reference instance with the shortest jobs first it gives 12 (Chapter 21 «Constructive heuristics»).
- *LPT* applies *LIST* to the jobs by non-increasing $t_j$; on the reference instance it gives 11, and $11/9 = 4/3 - 1/9$ is Graham's bound for $m = 3$ (Chapter 21 «Constructive heuristics»).

## 7.5 Local search

- The *move* neighbourhood has $n(m-1)$ elements and the *swap* neighbourhood at most $n(n-1)/2$; a transfer lowers the makespan only from the unique critical machine (Chapter 22 «Local search»).
- The $LPT$ solution of the reference instance is a local optimum for move, while two swaps reach the optimum 9 (Chapter 22 «Local search»).
- The auxiliary objective $\sum_h L_h^2$ gives a slope to the plateaus of the makespan: a move that improves the makespan improves it, and none that worsens the makespan does (Chapter 22 «Local search»).
- VND with move and swap and the auxiliary objective reaches the optimum 19 of the instance with 12 jobs in 68 of 100 runs (Chapter 22 «Local search»).

## 7.6 Metaheuristics

- Tabu search with the move neighbourhood leaves the plateau of makespan 11 of the reference instance and stops at $\mathrm{LB} = 9$ at iteration 8 (Chapter 23 «Metaheuristics»).
- VNS with $k_{\max} = 3$ reaches the optimum 9 of the reference instance in three iterations, and the optimum 69 of PCM15 in 7 runs out of 10 (Chapter 23 «Metaheuristics»).

## 7.7 MIP heuristics

- Rounding the LP solution violates an assignment constraint, fractional diving ends at makespan 10, and guided diving from $LPT$ finds the optimum 9 (Chapter 24 «MIP heuristics for the classical problems»).
- The feasibility pump returns a schedule of makespan 11, since $C_{\max}$ does not enter the distance $\Delta$ (Chapter 24 «MIP heuristics for the classical problems»).
- RINS, local branching and proximity search, started from $LPT$, all reach makespan 9; proximity search also proves its optimality (Chapter 24 «MIP heuristics for the classical problems»).

## 7.8 Approximation

- $LIST$ is a $\left(2-\frac{1}{m}\right)$-approximation algorithm for every list, and the instance with $m(m-1)$ unit jobs followed by a job of time $m$ shows that the ratio is tight (Chapter 28 «$P\|C_{\max}$, metric TSP and the limits of approximation»).
- $LPT$ is a $\left(\frac{4}{3}-\frac{1}{3m}\right)$-approximation algorithm, tight on Graham's family $t = (2m-1, 2m-1, \ldots, m+1, m+1, m, m, m)$, which for $m = 3$ is the reference instance (Chapter 28 «$P\|C_{\max}$, metric TSP and the limits of approximation»).
- The algorithm ONE, all the jobs on machine 1, is an $m$-approximation algorithm with $r(\mathrm{ONE}) = m$ and no finite additive guarantee (Exercise 25.2).

## 7.9 Instances

### 7.9.1 Instance PCM1

Reference instance (Graham's tight instance of $LPT$ for $m = 3$).
$m = 3$ machines, $n = 7$ jobs:

| $j$ | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|
| $t_j$ | 5 | 5 | 4 | 4 | 3 | 3 | 3 |

- Optimal value $z(\text{PCM1}) = 9$, attained by $J_1 = \{1, 3\}$, $J_2 = \{2, 4\}$, $J_3 = \{5, 6, 7\}$ (two optimal schedules, twelve optimal assignments).
- Used in A small $P \,||\, C_{\max}$ instance, Primal and dual bounds ($LPT$ 11, LB $= 9$), $P \,||\, C_{\max}$ instance, Counting the schedules, Assigning every job to the least loaded machine, The bound on the reference instance, $\text{ILP}_{P||C_{\max}}$ model, $P \,||\, C_{\max}$ reference instance, Gaps of the reference instances, $LIST$ on the reference instance, shortest jobs first, $LPT$ on the reference instance, The solution of $LPT$ is a local optimum for move, but not for swap, $VND$ on $P \,||\, C_{\max}$ with move and swap, $TS$ on the $P \,||\, C_{\max}$ reference instance, $VNS$ on the $P \,||\, C_{\max}$ reference instance, The LP solutions of the reference instances, Rounding on the reference instances, Fractional diving on $P \,||\, C_{\max}$, Guided diving on $P \,||\, C_{\max}$, The feasibility pump on $P \,||\, C_{\max}$, RINS on $P \,||\, C_{\max}$, Local branching on $P \,||\, C_{\max}$, Proximity search on $P \,||\, C_{\max}$, $LIST$ on the reference instance, shortest jobs first, $LPT$ on the reference instance, Tight family of $LPT$, case $m = 3$, instance $I_3$.

### 7.9.2 Instance PCM2

Three jobs on two machines: the lower bound is not attained.
$m = 2$ machines, $n = 3$ jobs:

| $j$ | 1 | 2 | 3 |
|---|---|---|---|
| $t_j$ | 4 | 3 | 3 |

- Optimal value $z(\text{PCM2}) = 6$, attained by $\{1\}, \{2, 3\}$ (the bound gives 5).
- Used in A combinatorial lower bound.

### 7.9.3 Instance PCM3

Two machines, five jobs: LPT, bounds and gap.
$m = 2$ machines, $n = 5$ jobs:

| $j$ | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| $t_j$ | 5 | 4 | 3 | 3 | 3 |

- Optimal value $z(\text{PCM3}) = 9$, attained by $J_1 = \{1, 2\}$, $J_2 = \{3, 4, 5\}$ ($LPT$ gives 10).
- Used in Exercise «LPT, bounds and gap on $P||C_{\max}$».

### 7.9.4 Instance PCM4

Loads, makespan and Gantt chart.

$m = 3$ machines, $n = 7$ jobs:

| $j$ | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|
| $t_j$ | 4 | 2 | 5 | 3 | 6 | 2 | 3 |

- Optimal value $z(\text{PCM4}) = 9$, attained by $J_1 = \{1, 3\}$, $J_2 = \{4, 5\}$, $J_3 = \{2, 6, 7\}$.
- Used in Exercise 7.1.

### 7.9.5 Instance PCM5

A first schedule, a lower bound and the optimum.

$m = 3$ machines, $n = 7$ jobs:

| $j$ | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|
| $t_j$ | 3 | 5 | 2 | 6 | 4 | 4 | 3 |

- Optimal value $z(\text{PCM5}) = 9$, attained by $J_1 = \{4, 7\}$, $J_2 = \{2, 6\}$, $J_3 = \{1, 3, 5\}$ (four optimal schedules).
- Used in Exercise 7.3.

### 7.9.6 Instance PCM6

When the lower bound is not attained.

$m = 2$ machines, $n = 4$ jobs:

| $j$ | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| $t_j$ | 8 | 5 | 4 | 3 |

- Optimal value $z(\text{PCM6}) = 11$, attained by $J_1 = \{1, 4\}$, $J_2 = \{2, 3\}$, unique (the bound gives 10).
- Used in Exercise 7.4.

### 7.9.7 Instance PCM7

Makespan and bin packing.

$m = 3$ machines, $n = 5$ jobs:

| $j$ | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| $t_j$ | 7 | 6 | 5 | 5 | 4 |

- Optimal value $z(\text{PCM7}) = 10$, attained by $J_1 = \{1\}$, $J_2 = \{2, 5\}$, $J_3 = \{3, 4\}$, unique (the bound gives 9).
- Used in Exercise 7.5.

### 7.9.8 Instance PCM8

An integrality gap of the LP relaxation.

$m = 2$ machines, $n = 3$ jobs:

| $j$ | 1 | 2 | 3 |
|---|---|---|---|
| $t_j$ | 3 | 3 | 2 |

- Optimal value $z(\text{PCM8}) = 5$, attained by loads 5 and 3; $z(\text{LP}_{P\|C_{\max}}) = 4$.
- Used in An integrality gap.

### 7.9.9 Instance PCM9

Two machines, five jobs: ILP model and algorithm ONE.

$m = 2$ machines, $n = 5$ jobs:

| $j$ | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| $t_j$ | 7 | 5 | 4 | 3 | 3 |

- Optimal value $z(\text{PCM9}) = 11$, attained by $J_1 = \{1, 3\}$, $J_2 = \{2, 4, 5\}$, unique partition.
- Used in Exercise 14.1, instance $I_1$.

### 7.9.10 Instance PCM10

LP relaxation and its dual.

$m = 3$ machines, $n = 6$ jobs:

| $j$ | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| $t_j$ | 8 | 6 | 5 | 4 | 4 | 2 |

- Optimal value $z(\text{PCM10}) = 10$, attained by $J_1 = \{1, 6\}$, $J_2 = \{2, 4\}$, $J_3 = \{3, 5\}$; $z(\text{LP}_{P\|C_{\max}}) = 29/3$.
- Used in Exercise 14.2.

### 7.9.11 Instance PCM11

Feasible solutions of the model.

$m = 2$ machines, $n = 5$ jobs:

| $j$ | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| $t_j$ | 6 | 5 | 4 | 3 | 2 |

- Optimal value $z(\text{PCM11}) = 10$, attained by $J_1 = \{1, 3\}$, $J_2 = \{2, 4, 5\}$; no integrality gap.
- Used in Exercise 14.3.

### 7.9.12 Instance PCM12

One long job: valid inequalities and symmetry.

$m = 3$ machines, $n = 6$ jobs:

| $j$ | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| $t_j$ | 12 | 3 | 3 | 2 | 2 | 2 |

- Optimal value $z(\text{PCM12}) = 12$, attained by job 1 alone; 96 optimal solutions of the model, 18 with the symmetry-breaking constraints.
- Used in Exercise 14.4.

### 7.9.13 Instance PCM13

Integrality gap on three jobs.

$m = 2$ machines, $n = 3$ jobs:

| $j$ | 1 | 2 | 3 |
|---|---|---|---|
| $t_j$ | 5 | 5 | 3 |

- Optimal value $z(\text{PCM13}) = 8$, attained by $\{1,3\},\{2\}$; $z(\text{LP}_{P\|C_{\max}}) = 13/2$.
- Used in Exercise 14.5.

### 7.9.14 Instance PCM14

The family of the ratio $1/m$ with $M = 10$, $m = 3$.

$m = 3$ machines, $n = 4$ jobs:

| $j$ | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| $t_j$ | 10 | 1 | 1 | 1 |

- Optimal value $z(\text{PCM14}) = 10$, attained by the long job alone; $z(\text{LP}_{P\|C_{\max}}) = 13/3$.
- Used in question 4, The ratio $1/m$ is asymptotically attained (family).

### 7.9.15 Instance PCM15

Gap of a heuristic schedule.

$m = 3$ machines, $n = 7$ jobs:

| $j$ | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|
| $t_j$ | 7 | 6 | 5 | 4 | 3 | 3 | 2 |

- Optimal value $z(\text{PCM15}) = 10$, attained by $\{1,5\},\{2,4\},\{3,6,7\}$.
- Used in instance (c).

### 7.9.16 Instance PCM16

Constructive heuristic, local search and multi-start on two machines.

$m = 2$ machines, $n = 6$ jobs:

| $j$ | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| $t_j$ | 8 | 4 | 4 | 6 | 3 | 7 |

- Optimal value $z(\text{PCM16}) = 16$, attained by $J_1 = \{1,2,3\}$, $J_2 = \{4,5,6\}$.
- Used in Exercise 20.5.

### 7.9.17 Instance PCM17

List scheduling and LPT 1.

$m = 3$ machines, $n = 8$ jobs:

| $j$ | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|---|---|---|---|---|---|---|---|---|
| $t_j$ | 2 | 8 | 7 | 6 | 8 | 3 | 4 | 9 |

- Optimal value $z(\text{PCM17}) = 16$, attained by the *LPT* schedule $J_1 = \{6,7,8\}$, $J_2 = \{2,3\}$, $J_3 = \{1,4,5\}$.
- Used in Exercise 21.7.

### 7.9.18 Instance PCM18

List scheduling and LPT 2.
$m = 3$ machines, $n = 7$ jobs:

| $j$ | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|
| $t_j$ | 3 | 6 | 5 | 4 | 4 | 9 | 7 |

- Optimal value $z(\text{PCM18}) = 13$, attained by $J_1 = \{1, 2, 4\}$, $J_2 = \{3, 7\}$, $J_3 = \{5, 6\}$ (*LPT* gives 14).
- Used in Exercise 21.8.

### 7.9.19 Instance PCM19

List scheduling and LPT 3: the input order beats LPT.
$m = 3$ machines, $n = 7$ jobs:

| $j$ | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|
| $t_j$ | 5 | 8 | 8 | 4 | 5 | 7 | 4 |

- Optimal value $z(\text{PCM19}) = 15$, attained by $J_1 = \{1, 2\}$, $J_2 = \{3, 5\}$, $J_3 = \{4, 6, 7\}$ (*LIST* gives 15, *LPT* 16).
- Used in Exercise 21.9.

### 7.9.20 Instance PCM20

Medium instance with 12 jobs on 4 machines (PCM12).
$m = 4$ machines, $n = 12$ jobs:

| $j$ | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| $t_j$ | 11 | 10 | 9 | 8 | 7 | 7 | 6 | 5 | 4 | 4 | 3 | 2 |

- Optimal value $z(\text{PCM20}) = 19$, attained by $z(I) = \lceil 76/4 \rceil$; *LPT* gives 20, a local optimum for move and swap.
- Used in A larger neighbourhood does not always help, Plateaus, Experiments on two medium instances, VND from 100 random starts.

### 7.9.21 Instance PCM21

Local search with move from LIST.
$m = 3$ machines, $n = 6$ jobs:

| $j$ | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| $t_j$ | 5 | 7 | 3 | 4 | 4 | 9 |

- Optimal value $z(\text{PCM21}) = 12$, attained by final schedule of the move local search, makespan 12 (the bound 11 is not attained).
- Used in Exercise 22.10.

### 7.9.22 Instance PCM22

LPT is a local optimum for move, not for swap.

$m = 3$ machines, $n = 7$ jobs:

| $j$ | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|
| $t_j$ | 8 | 4 | 6 | 7 | 5 | 9 | 8 |

- Optimal value $z(\text{PCM22}) = 16$, attained by final schedule of the swap local search, makespan $16 = \lceil 47/3 \rceil$.
- Used in Exercise 22.11.

### 7.9.23 Instance PCM23

A plateau and the auxiliary objective.

$m = 3$ machines, $n = 6$ jobs:

| $j$ | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| $t_j$ | 5 | 5 | 7 | 9 | 2 | 7 |

- Optimal value $z(\text{PCM23}) = 12$, attained by final schedule of the move local search with $\sum_h L_h^2$, makespan 12.
- Used in Exercise 22.12.

### 7.9.24 Instance PCM24

Medium instance with 15 jobs on 4 machines (PCM15).

$m = 4$ machines, $n = 15$ jobs:

| $j$ | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| $t_j$ | 10 | 23 | 26 | 16 | 23 | 11 | 14 | 18 | 18 | 24 | 29 | 22 |

| $j$ | 13 | 14 | 15 |
|---|---|---|---|
| $t_j$ | 13 | 13 | 15 |

- Optimal value $z(\text{PCM24}) = 69$, attained by $z(I) = \lceil 275/4 \rceil$; $LPT$ gives 73.
- Used in The medium instances of the KP, the BPP and $P \,||\, C_{\max}$, VNS on PCM15, The methods on the medium instances.

### 7.9.25 Instance PCM25

Tabu search on two machines.

$m = 2$ machines, $n = 5$ jobs:

| $j$ | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| $t_j$ | 3 | 3 | 2 | 2 | 2 |

- Optimal value $z(\text{PCM25}) = 6$, attained by $J_1 = \{1, 2\}$, $J_2 = \{3, 4, 5\}$.
- Used in Exercise 23.4, Exercise 23.5.

### 7.9.26 Instance PCM26

Variable neighbourhood search on three machines.
$m = 3$ machines, $n = 7$ jobs:

| $j$ | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|
| $t_j$ | 4 | 4 | 3 | 3 | 2 | 2 | 2 |

- Optimal value $z(\text{PCM26}) = 7$, attained by loads $(7, 6, 7)$, makespan $7 = \text{LB}$.
- Used in Exercise 23.9.

### 7.9.27 Instance PCM27

Proximity search on two machines.
$m = 2$ machines, $n = 6$ jobs:

| $j$ | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| $t_j$ | 9 | 3 | 9 | 5 | 8 | 2 |

- Optimal value $z(\text{PCM27}) = 18$, attained by $J_1 = \{2, 4, 5, 6\}$, $J_2 = \{1, 3\}$.
- Used in Exercise 24.5.

### 7.9.28 Instance PCM28

Algorithm ONE, instance $I_2$.
$m = 3$ machines, $n = 5$ jobs:

| $j$ | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| $t_j$ | 9 | 2 | 2 | 2 | 1 |

- Optimal value $z(\text{PCM28}) = 9$.
- Used in instance $I_2$.

### 7.9.29 Instance PCM29

Algorithm ONE: $2m$ unit jobs with $m = 4$.
$m = 4$ machines, $n = 8$ jobs:

| $j$ | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|---|---|---|---|---|---|---|---|---|
| $t_j$ | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |

- Optimal value $z(\text{PCM29}) = 2$, attained by two jobs per machine.
- Used in question 3.

### 7.9.30 Instance PCM30

Algorithm ONE: instance $I_1$ scaled by 10.
$m = 2$ machines, $n = 5$ jobs:

| $j$ | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| $t_j$ | 70 | 50 | 40 | 30 | 30 |

- Optimal value $z(\text{PCM30}) = 110$.
- Used in question 4.

### 7.9.31 Instance PCM31

Graham's tight instance of *LIST* for $m = 3$.
$m = 3$ machines, $n = 7$ jobs:

| $j$ | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|
| $t_j$ | 1 | 1 | 1 | 1 | 1 | 1 | 3 |

- Optimal value $z(\text{PCM31}) = 3$, attained by $J_1 = \{7\}$, $J_2 = \{1, 2, 3\}$, $J_3 = \{4, 5, 6\}$ (*LIST* gives 5).
- Used in Tight family of *LIST*, case $m = 3$, Exercise 28.2.

### 7.9.32 Instance PCM32

*LIST* and *LPT* guarantees on seven jobs.
$m = 3$ machines, $n = 7$ jobs:

| $j$ | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|
| $t_j$ | 2 | 2 | 4 | 8 | 8 | 6 | 6 |

- Optimal value $z(\text{PCM32}) = 12$, attained by the *LPT* schedule (*LIST* gives 16).
- Used in Exercise 28.1.

### 7.9.33 Instance PCM33

Graham's tight instance of *LPT* for $m = 4$.
$m = 4$ machines, $n = 9$ jobs:

| $j$ | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|
| $t_j$ | 7 | 7 | 6 | 6 | 5 | 5 | 4 | 4 | 4 |

- Optimal value $z(\text{PCM33}) = 12$, attained by $J_1 = \{1, 5\}$, $J_2 = \{2, 6\}$, $J_3 = \{3, 4\}$, $J_4 = \{7, 8, 9\}$ (*LPT* gives 15).
- Used in Exercise 28.3.

## 7.10 Exercises

- Exercises «Recognizing the problems» (counting the assignments) and «LPT, bounds and gap on $P||C_{\max}$»
- Counting the solutions (instance (d))
- Loads, makespan and Gantt chart
- Counting the schedules
- A first schedule, a lower bound and the optimum
- When the lower bound is not attained
- Makespan and bin packing

- ILP model of $P||C_{\max}$
- LP relaxation of the $P||C_{\max}$ model
- Feasible solutions of the $P||C_{\max}$ model
- Valid inequalities and symmetry for $P||C_{\max}$
- Integrality gap of the $P||C_{\max}$ model
- Gaps for minimization problems
- Constructive heuristics, local search and metaheuristics
- List scheduling and LPT 1
- List scheduling and LPT 2
- List scheduling and LPT 3
- Local search $P\,||\,C_{\max}$ 1
- Local search $P\,||\,C_{\max}$ 2
- Local search $P\,||\,C_{\max}$ 3
- Tabu search 1
- Tabu search 2
- Iterated local search and VNS 3
- Proximity search on $P\,||\,C_{\max}$
- A tight guarantee for a simple algorithm
- $P\,||\,C_{\max}$ approximation 1
- $P\,||\,C_{\max}$ approximation 2
- $P\,||\,C_{\max}$ approximation 3

## 7.11 Scheduling on a fixed number of identical machines ($m$ not part of the input) ($Pm||C_{\max}$)

**Definition 7.2: Scheduling on a fixed number of identical machines**

The problem $P\,||\,C_{\max}$ in which the number $m$ of machines is a constant, for example $m = 2$, and only the jobs $\mathscr{J}$ and the processing times $t_j \in \mathbb{Z}_{\geq 1}$ form the input.

- Defined in Chapter 1 «Combinatorial optimization problems».
- Special case of $P\,||\,C_{\max}$ with $m$ fixed; with $m = 2$ it asks for a split of the jobs into two sets of loads as equal as possible.

- **Complexity**: Weakly NP-hard for a fixed number of machines, e.g. $m = 2$.

**Where it is treated**

- Chapter 1 «Combinatorial optimization problems»: Complexity: weakly NP-hard for a fixed number of machines
- Chapter 8 «Parallel machine scheduling»: Instances with $m = 2$ in the examples and exercises

## 7.12 Scheduling on parallel machines with different speeds, $Q \,||\, C_{\max}$

**Definition 7.3: Scheduling on parallel machines with different speeds, $Q \,||\, C_{\max}$**

In the three-field notation $\alpha \,|\, \beta \,|\, \gamma$, the machine environment $\alpha = Q$ denotes parallel machines with different speeds; $Q \,||\, C_{\max}$ assigns jobs without additional characteristics to such machines, minimizing the makespan $C_{\max}$.

- Defined in Chapter 7 «Parallel machine scheduling».
- Generalizes $P \,||\, C_{\max}$, which is the case of machines with equal speeds.

**Where it is treated**

- Chapter 8 «Parallel machine scheduling»: Three-field notation

## 7.13 Scheduling on unrelated parallel machines, $R \,||\, C_{\max}$

**Definition 7.4: Scheduling on unrelated parallel machines, $R \,||\, C_{\max}$**

In the three-field notation $\alpha \,|\, \beta \,|\, \gamma$, the machine environment $\alpha = R$ denotes unrelated parallel machines; $R \,||\, C_{\max}$ assigns jobs without additional characteristics to such machines, minimizing the makespan $C_{\max}$.

- Defined in Chapter 7 «Parallel machine scheduling».
- Generalizes $Q \,||\, C_{\max}$ and $P \,||\, C_{\max}$.

**Where it is treated**

- Chapter 8 «Parallel machine scheduling»: Three-field notation

## Chapter 8

# Capacitated lot sizing

Plan the production of every period, with capacities, setup costs and stock, at minimum total cost.

**Where it is treated**

- Chapter 1 «Combinatorial optimization problems»: One of the six problems of the course: the subset of setup periods and the complexity
- Chapter 8 «Capacitated lot sizing»: Definition, variants, counting the setup choices, a first plan by hand and the setup lower bound
- Chapter 15 «Formulations of capacitated lot sizing»: The big-M formulation with weak and strong values, the shortest-path formulation, several products
- Chapter 19 «Dynamic programming for lot sizing»: Wagner–Whitin DP for the ULS in $\Theta(m^2)$ and the pseudo-polynomial DP on (period, inventory) for the CLS
- Chapter 24 «MIP heuristics for the classical problems»: Relax-and-fix and fix-and-optimize on an uncapacitated and a capacitated instance

## 8.1 Definition

**Definition 8.1: Capacitated lot sizing**

Given:

1. a set $T = \{1, 2, \ldots, m\}$ of $m$ *time periods*,
2. for each period $t \in T$, a *demand* $d_t \in \mathbb{Z}_{\geq 1}$ and a *capacity* $cap_t \in \mathbb{Z}_{\geq 1}$, both counted in units of one product,
3. for each period $t \in T$, three non-negative integer costs: the *setup cost* $sc_t$, the *variable production cost* $vc_t$ per unit produced and the *holding cost* $hc_t$ per unit kept in stock at the end of the period,

the **capacitated lot sizing problem** (CLS) asks to decide how much to produce in every period, so that every demand is satisfied with what is produced and what is in stock, the capacities are respected, and the total cost of setups, production and holding is minimized. With the stock $s_t(\boldsymbol{x}) = \sum_{u=1}^{t}(x_u - d_u)$ and $y_t = 1$ when $x_t > 0$, the cost of a plan is $c(\boldsymbol{x}) = \sum_{t\in T}\big(sc_t\, y_t + vc_t\, x_t + hc_t\, s_t(\boldsymbol{x})\big)$, and the problem reads

$$\min_{\boldsymbol{x}} \left\{ c(\boldsymbol{x}) \;:\; x_t \le cap_t,\; s_t(\boldsymbol{x}) \ge 0,\; \forall t \in T,\; s_m(\boldsymbol{x}) = 0,\; \boldsymbol{x} \in \mathbb{Z}^m_{\ge 0} \right\}.$$

- Defined in Chapter 8 «Capacitated lot sizing».
- It models the production planning of one product over a horizon of periods, where every production run pays a fixed setup cost and anticipated units pay a holding cost.
- A solution is a **production plan** $\boldsymbol{x} = (x_1, \dots, x_m) \in \mathbb{Z}^m_{\ge 0}$; its combinatorial part is the **subset** of periods with a setup, one of $2^m$ choices.
- Once the setup periods are fixed, the quantities are found by linear programming (a minimum cost flow problem with an integer optimal solution).

## 8.2 Bounds and complexity

- **Complexity**: NP-hard with general capacities; with integer data the DP algorithm on (period, inventory) is pseudo-polynomial, $O(m\,D\,C)$; polynomial with constant capacity, and the ULS in $\Theta(m^2)$.
- $z(I) \ge$ the sum of the $k$ smallest setup costs, with $k$ the smallest number of periods whose capacities sum up to at least $D = \sum_t d_t$ (Chapter 8 «Capacitated lot sizing»).
- $z(\mathrm{LP}^{\mathrm{strong}}_{\mathrm{CLSP}})$ with $M_t = \min\{cap_t, D_{t,m}\}$: the smaller the big-M value, the stronger the LP relaxation (Chapter 15 «Formulations of capacitated lot sizing»).
- $z(\mathrm{LP}^{\mathrm{SP}}_{\mathrm{CLSP}})$ of the shortest-path formulation, the strongest of the LP bounds of the chapter (170 on the reference instance) (Chapter 15 «Formulations of capacitated lot sizing»).
- Of the $2^m$ setup choices only those whose capacities cover the cumulative demands are feasible, and each carries one or more plans: 8 choices and 94 plans on the reference instance.
- With binding capacities an optimal plan may produce while the stock is positive: $(3, 0, 4, 4, 0)$ produces in period 4 with one unit in stock.
- Increasing the capacities can only lower the optimal cost: 132, 112 and 104 with $cap_t = 5$, $cap_t = 6$ and no capacity in Exercise 8.4.

## 8.3 Formulation

- **Big-M formulation of the CLS** ($\mathrm{ILP_{CLSP}}$) (Chapter 15 «Formulations of capacitated lot sizing»):

$$
\begin{aligned}
\min\ & \sum_{t\in T} (sc_t\, y_t + vc_t\, x_t + hc_t\, s_t) && \\
& s_{t-1} + x_t = d_t + s_t, && \forall t \in T,\\
& x_t \le M_t\, y_t, && \forall t \in T,\\
& x_t \le cap_t, && \forall t \in T,\\
& s_0 = s_m = 0, && \\
& x_t,\ s_t \in \mathbb{Z}_{\ge 0}, && \forall t \in T,\\
& y_t \in \{0,1\}, && \forall t \in T.
\end{aligned}
$$

- **Shortest-path formulation of the CLS, with** $cv_{tk} = vc_t\, D_{t,k} + \sum_{s=t+1}^{k} \sum_{u=t}^{s-1} hc_u\, d_s$ (Chapter 15 «Formulations of capacitated lot sizing»):

$$
\begin{aligned}
\min\ & \sum_{t\in T} sc_t\, y_t + \sum_{t\in T} \sum_{k=t}^{m} cv_{tk}\, zv_{tk} && \\
& \sum_{k=1}^{m} zv_{1k} = 1, && \\
& \sum_{u=1}^{t-1} zv_{u,t-1} = \sum_{k=t}^{m} zv_{tk}, && \forall t \in T \setminus \{1\},\\
& \sum_{k=t}^{m} zv_{tk} \le y_t, && \forall t \in T,\\
& \sum_{k=t}^{m} D_{t,k}\, zv_{tk} \le cap_t, && \forall t \in T,\\
& zv_{tk} \ge 0, && \forall t,k \in T,\ k \ge t,\\
& y_t \in \{0,1\}, && \forall t \in T.
\end{aligned}
$$

- The big-M formulation $\mathrm{ILP_{CLSP}}$ has the balance constraints $s_{t-1} + x_t = d_t + s_t$, the link $x_t \le M_t\, y_t$ and the capacities $x_t \le cap_t$ (Chapter 15 «Formulations of capacitated lot sizing»).

- The strong value $M_t = \min\{cap_t, D_{t,m}\}$ raises the LP bound of the reference instance from 99.65 ($M_t = D_{t,m}$) and 137.5 ($M_t = cap_t$) to 157.5 (Chapter 15 «Formulations of capacitated lot sizing»).

- The shortest-path formulation, with $O(m^2)$ variables $zv_{tk}$ and the link $\sum_k zv_{tk} \le y_t$, gives the bound 170, the optimal value without capacities (Chapter 15 «Formulations of capacitated lot sizing»).

- A plan that is not a path is a combination of paths: the optimal plan $(3,0,4,4,0)$ uses $zv_{33} = \frac{4}{5}$ and $zv_{35} = \frac{1}{5}$ (Chapter 15 «Formulations of capacitated lot sizing»).

## 8.4 Dynamic programming

- Without capacities an optimal plan produces only with empty stock (Wagner and Whitin), so it is a sequence of production runs (Chapter 19 «Dynamic programming for lot sizing»).
- The recurrence $v(t) = \min_j\{v(j-1) + c_{j,t}\}$ over the last production run solves the ULS in $\Theta(m^2)$ steps; it is a shortest path (Chapter 19 «Dynamic programming for lot sizing»).
- With capacities the property fails: the state is the pair (period $t$, final inventory $k$), and the recurrence $v(t,k)$ with the production cost $f_t(x)$ gives $v(m,0)$ (Chapter 19 «Dynamic programming for lot sizing»).
- The DP algorithm for the CLS takes $O(m\,D\,C)$ steps, with $D$ the total demand and $C$ the largest capacity: it is pseudo-polynomial (Chapter 19 «Dynamic programming for lot sizing»).

## 8.5 Constructive heuristics

- The lot-for-lot plan produces every demand in its period, and merging moves a demand to an earlier period, saving a setup and paying the stock: 250 → 210 → 180 on the reference instance (Chapter 8 «Capacitated lot sizing»).
- On the uncapacitated instance of the MIP course the least unit cost heuristic gives 200, against the optimum 170 (Chapter 24 «MIP heuristics for the classical problems»).

## 8.6 MIP heuristics

- Relax-and-fix with one setup variable per block finds the optimum 170 of the uncapacitated instance of the MIP course (Chapter 24 «MIP heuristics for the classical problems»).
- On a capacitated instance relax-and-fix saves a setup too early and stops at 170; the capacity forces a setup that the relaxation paid only in part (Chapter 24 «MIP heuristics for the classical problems»).
- Fix-and-optimize with windows of two consecutive setups revises that decision and reaches the optimum 160 (Chapter 24 «MIP heuristics for the classical problems»).

## 8.7 Instances

### 8.7.1 Instance CLS1

Reference instance (lot sizing instance of the MIP course in units of ten items, capacity 4).
$T = 5$ periods:

| $t$ | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| $d_t$ | 2 | 1 | 3 | 4 | 1 |
| $C_t$ | 4 | 4 | 4 | 4 | 4 |
| $f_t$ | 50 | 50 | 50 | 50 | 50 |
| $p_t$ | 0 | 0 | 0 | 0 | 0 |
| $h_t$ | 10 | 10 | 10 | 10 | 10 |

- Optimal value $z(\text{CLS1}) = 180$, attained by $\tilde{\boldsymbol{x}} = (3,0,4,4,0)$, $\tilde{\boldsymbol{y}} = (1,0,1,1,0)$, stocks $(1,0,1,1,0)$; unique.
- Used in CLS instance, Counting the plans, From lot-for-lot to a better plan, The bound on the reference instance, The reference instance, Big-M values and LP bounds, Costs and combinations of paths, Comparing the formulations, Capacitated lot sizing, Execution of the DP algorithm, Computing an optimal solution, The case of constant capacity.

### 8.7.2 Instance CLS2

Capacitated instance of the MIP course ($F = 40$, $h = 1$, $Q = 80$).
$T = 6$ periods:

| $t$ | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| $d_t$ | 50 | 30 | 10 | 10 | 40 | 10 |
| $C_t$ | 80 | 80 | 80 | 80 | 80 | 80 |
| $f_t$ | 40 | 40 | 40 | 40 | 40 | 40 |
| $p_t$ | 0 | 0 | 0 | 0 | 0 | 0 |
| $h_t$ | 1 | 1 | 1 | 1 | 1 | 1 |

- Optimal value $z(\text{CLS2}) = 160$, attained by $(50,50,0,0,50,0)$, setups in periods $1,2,5$.
- Used in Relax-and-fix on a capacitated instance, Fix-and-optimize on the capacitated instance, Number of steps of the DP (in units of 10: $d = (5,3,1,1,4,1)$, $hc_t = 10$, $cap_t = 8$).

### 8.7.3 Instance CLS3

Four periods, capacity 5.
$T = 4$ periods:

| $t$ | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| $d_t$ | 3 | 2 | 4 | 3 |
| $C_t$ | 5 | 5 | 5 | 5 |
| $f_t$ | 30 | 40 | 30 | 40 |
| $p_t$ | 2 | 2 | 2 | 2 |
| $h_t$ | 4 | 4 | 4 | 4 |

- Optimal value $z(\text{CLS3}) = 132$, attained by $(5,0,4,3)$, setups in periods $1,3,4$; unique.
- Used in Exercise 8.1, Exercise 8.2, Exercise 8.3, Exercise 8.4.

### 8.7.4 Instance CLS4

The same four periods with capacity 6.
$T = 4$ periods:

| $t$ | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| $d_t$ | 3 | 2 | 4 | 3 |
| $C_t$ | 6 | 6 | 6 | 6 |
| $f_t$ | 30 | 40 | 30 | 40 |
| $p_t$ | 2 | 2 | 2 | 2 |
| $h_t$ | 4 | 4 | 4 | 4 |

- Optimal value $z(\text{CLS4}) = 112$, attained by $(6, 0, 6, 0)$; unique.
- Used in capacity 6.

### 8.7.5 Instance CLS5

Lot-for-lot, a lower bound and the optimum.
$T = 3$ periods:

| $t$ | 1 | 2 | 3 |
|---|---|---|---|
| $d_t$ | 4 | 3 | 5 |
| $C_t$ | 7 | 6 | 7 |
| $f_t$ | 50 | 20 | 60 |
| $p_t$ | 0 | 0 | 0 |
| $h_t$ | 3 | 3 | 3 |

- Optimal value $z(\text{CLS5}) = 91$, attained by $(6, 6, 0)$ (lot-for-lot 130, bound 70).
- Used in Exercise 8.5.

### 8.7.6 Instance CLS6

Four periods, capacity 4: formulations.
$T = 4$ periods:

| $t$ | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| $d_t$ | 1 | 4 | 1 | 3 |
| $C_t$ | 4 | 4 | 4 | 4 |
| $f_t$ | 40 | 40 | 40 | 40 |
| $p_t$ | 0 | 0 | 0 | 0 |
| $h_t$ | 5 | 5 | 5 | 5 |

- Optimal value $z(\text{CLS6}) = 130$, attained by $(2, 4, 0, 3)$, $\tilde{\boldsymbol{y}} = (1, 1, 0, 1)$; unique.
- Used in Exercise 15.1, Exercise 15.2, Exercise 15.3, Exercise 15.4.

### 8.7.7 Instance CLS7

Five periods, capacity 4: comparing the formulations.
$T = 5$ periods:

| $t$ | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| $d_t$ | 2 | 1 | 3 | 3 | 3 |
| $C_t$ | 4 | 4 | 4 | 4 | 4 |
| $f_t$ | 40 | 40 | 40 | 40 | 40 |
| $p_t$ | 0 | 0 | 0 | 0 | 0 |
| $h_t$ | 10 | 10 | 10 | 10 | 10 |

- Optimal value $z(\text{CLS7}) = 170$, attained by $(3, 0, 3, 3, 3)$; unique.
- Used in Exercise 15.5.

### 8.7.8 Instance CLS8

DP on (period, inventory), exercise 4.
$T = 3$ periods:

| $t$ | 1 | 2 | 3 |
|---|---|---|---|
| $d_t$ | 1 | 2 | 3 |
| $C_t$ | 2 | 4 | 4 |
| $f_t$ | 10 | 5 | 8 |
| $p_t$ | 1 | 1 | 2 |
| $h_t$ | 1 | 1 | 2 |

- Optimal value $z(\text{CLS8}) = 25$, attained by $(2, 4, 0)$.
- Used in Exercise 19.4.

### 8.7.9 Instance CLS9

DP on (period, inventory), exercise 5.
$T = 4$ periods:

| $t$ | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| $d_t$ | 3 | 1 | 1 | 2 |
| $C_t$ | 4 | 3 | 3 | 2 |
| $f_t$ | 5 | 6 | 10 | 10 |
| $p_t$ | 2 | 1 | 1 | 1 |
| $h_t$ | 1 | 1 | 2 | 2 |

- Optimal value $z(\text{CLS9}) = 30$, attained by $(4, 3, 0, 0)$.
- Used in Exercise 19.5.

## 8.8 Exercises

- Exercise «Recognizing the problems» (choices of the setup periods)
- Feasibility and cost of a plan
- Counting the setup choices
- The best plan by enumeration of the setups
- The effect of the capacity
- Lot-for-lot, a lower bound and the optimum
- The big-M formulation of lot sizing
- Big-M values and LP bounds for lot sizing
- Feasible solutions of the lot-sizing formulation
- The shortest-path formulation of lot sizing
- Comparing the lot-sizing formulations

- Uncapacitated lot sizing (Wagner–Whitin) 1
- Uncapacitated lot sizing (Wagner–Whitin) 2
- Uncapacitated lot sizing (Wagner–Whitin) 3
- Capacitated lot sizing, DP on (period, inventory) 1
- Capacitated lot sizing, DP on (period, inventory) 2

## 8.9 Uncapacitated lot sizing problem (ULS)

**Definition 8.2: Uncapacitated lot sizing problem**

Given:

1. a set $T = \{1, 2, \dots, m\}$ of $m \in \mathbb{Z}_{\geq 1}$ *time periods*,
2. for each period $t \in T$, a *demand* $d_t \in \mathbb{Z}_{\geq 1}$,
3. for each period $t \in T$, a *setup cost* $sc_t \in \mathbb{Z}_{\geq 0}$, a *variable production cost* $vc_t \in \mathbb{Z}_{\geq 0}$ per unit, and a *holding cost* $hc_t \in \mathbb{Z}_{\geq 0}$ per unit in stock at the end of period $t$,

the **Uncapacitated Lot-Sizing Problem** (ULS) asks to decide how many units to produce in each period, so that, starting with an empty inventory and ending with an empty inventory, the demand of every period is satisfied, and the total cost of setups, production, and holding is minimized.

- Defined in Chapter 19 «Dynamic programming for lot sizing».
- Special case of the CLS in which the capacities are not binding, for example $cap_t \geq \sum_u d_u$.
- **Complexity**: Polynomial: the DP algorithm of Wagner and Whitin runs in $\Theta(m^2)$ steps.
- $v(t) = \min\{v(j-1) + c_{j,t} : j \in \{1, \dots, t\}\}$, $v(0) = 0$, with the run cost $c_{j,t} = sc_j + vc_j\, D_{j,t} + \sum_{k=j}^{t-1} hc_k\, D_{k+1,t}$ (exact recurrence) (Chapter 19 «Dynamic programming for lot sizing»).
- **MIP model of the ULS** (Chapter 19 «Dynamic programming for lot sizing»):

$$
\begin{aligned}
\min \sum_{t \in T} & (sc_t\, y_t + vc_t\, x_t + hc_t\, s_t) \\
& s_{t-1} + x_t = d_t + s_t, && t \in T \\
& x_t \leq D_{t,m}\, y_t, && t \in T \\
& s_m = 0 \\
& x_t,\ s_t \geq 0, && t \in T \\
& y_t \in \{0, 1\}, && t \in T
\end{aligned}
$$

**Where it is treated**

- Chapter 9 «Capacitated lot sizing»: Variant of the CLS, and the optimal cost 170 of the reference instance without capacities
- Chapter 17 «Formulations of capacitated lot sizing»: Its optimal value is the LP bound of the shortest-path formulation
- Chapter 21 «Dynamic programming for lot sizing»: Zero-inventory ordering property (Wagner and Whitin, 1958)
- Chapter 21 «Dynamic programming for lot sizing»: DP algorithm on the last production run, $\Theta(m^2)$ steps
- Chapter 26 «MIP heuristics for the classical problems»: Relax-and-fix on the instance of the MIP course

### 8.9.1 Instance ULS1

The reference instance without capacities.
$T = 5$ periods:

| $t$ | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| $d_t$ | 2 | 1 | 3 | 4 | 1 |
| $f_t$ | 50 | 50 | 50 | 50 | 50 |
| $p_t$ | 0 | 0 | 0 | 0 | 0 |
| $h_t$ | 10 | 10 | 10 | 10 | 10 |

- Optimal value $z(\text{ULS1}) = 170$, attained by $(3, 0, 8, 0, 0)$.
- Used in Counting the solutions: without the capacities the optimal cost is 170, Bound of the shortest-path formulation, Optimal ULS solution, not feasible with capacity 4.

### 8.9.2 Instance ULS2

Uncapacitated instance of the MIP course (the ULS above in original units: demands ×10, holding cost 1).
$T = 5$ periods:

| $t$ | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| $d_t$ | 20 | 10 | 30 | 40 | 10 |
| $f_t$ | 50 | 50 | 50 | 50 | 50 |
| $p_t$ | 0 | 0 | 0 | 0 | 0 |
| $h_t$ | 1 | 1 | 1 | 1 | 1 |

- Optimal value $z(\text{ULS2}) = 170$, attained by $(30, 0, 80, 0, 0)$ and $(30, 0, 30, 50, 0)$, two optimal plans.
- Used in Uncapacitated lot sizing, Enumeration of the solutions with the zero-inventory ordering property, Computation of the production-run costs, Computing the values of the $v$ function, Execution of the DP algorithm, Computing an optimal solution, Relax-and-fix on the lot sizing instance of the MIP course.

### 8.9.3 Instance ULS3

The same four periods without capacities.
$T = 4$ periods:

| $t$ | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| $d_t$ | 3 | 2 | 4 | 3 |
| $f_t$ | 30 | 40 | 30 | 40 |
| $p_t$ | 2 | 2 | 2 | 2 |
| $h_t$ | 4 | 4 | 4 | 4 |

- Optimal value $z(\text{ULS3}) = 104$, attained by $(5, 0, 7, 0)$.
- Used in no capacity.

### 8.9.4 Instance ULS4

Wagner–Whitin DP, exercise 1.
$T = 4$ periods:

| $t$ | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| $d_t$ | 30 | 10 | 40 | 10 |
| $f_t$ | 50 | 80 | 70 | 60 |
| $p_t$ | 2 | 3 | 1 | 2 |
| $h_t$ | 1 | 2 | 2 | 1 |

- Optimal value $z(\text{ULS4}) = 280$, attained by $(40, 0, 50, 0)$.
- Used in Exercise 19.1.

### 8.9.5 Instance ULS5

Wagner–Whitin DP, exercise 2.
$T = 4$ periods:

| $t$ | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| $d_t$ | 30 | 40 | 10 | 20 |
| $f_t$ | 50 | 80 | 80 | 50 |
| $p_t$ | 1 | 3 | 2 | 1 |
| $h_t$ | 1 | 1 | 1 | 2 |

- Optimal value $z(\text{ULS5}) = 260$, attained by $(80, 0, 0, 20)$.
- Used in Exercise 19.2.

### 8.9.6 Instance ULS6

Wagner–Whitin DP, exercise 3.
$T = 5$ periods:

| $t$ | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| $d_t$ | 10 | 30 | 40 | 30 | 40 |
| $f_t$ | 70 | 80 | 50 | 80 | 50 |
| $p_t$ | 1 | 1 | 1 | 1 | 2 |
| $h_t$ | 1 | 1 | 2 | 1 | 2 |

- Optimal value $z(\text{ULS6}) = 420$, attained by $(40, 0, 40, 70, 0)$.
- Used in Exercise 19.3.

## 8.10 Capacitated lot sizing with constant capacity (CLS-C)

**Definition 8.3: Capacitated lot sizing with constant capacity**

The CLS in which the capacity is the same in all periods, $cap_t = C$ for all $t \in T$.

- Defined in Chapter 19 «Dynamic programming for lot sizing».
- Special case of the CLS; the zero-inventory ordering property is replaced by a weaker one: between two periods with empty inventory at most one period produces a quantity strictly between 0 and $C$.
- **Complexity**: Solvable in polynomial time.

**Where it is treated**

- Chapter 21 «Dynamic programming for lot sizing»: The weaker property, checked on the reference instance ($C = 4$)

## 8.11 Lot sizing with several products and setup times (MLS)

**Definition 8.4: Lot sizing with several products and setup times**

A set $P$ of products shares the resource of each period: $d_{pt}$, $sc_{pt}$, $vc_{pt}$ and $hc_{pt}$ are the data of product $p \in P$ in period $t \in T$; producing a unit of $p$ takes the time $vt_{pt} \in \mathbb{Z}_{\geq 1}$, a setup takes the time $st_{pt} \in \mathbb{Z}_{\geq 0}$, and the capacity $cap_t$ is a time.

- Defined in Chapter 15 «Formulations of capacitated lot sizing».
- Generalizes the CLS, which is the case of one product with $vt_t = 1$ and no setup time.
- Strong big-M value $M_{pt} = \min\left\{ \left\lfloor \frac{cap_t - st_{pt}}{vt_{pt}} \right\rfloor, \ sd_{ptm} \right\}$, with $sd_{ptk} = \sum_{j=t}^{k} d_{pj}$ (Chapter 15 «Formulations of capacitated lot sizing»).
- **Capacity and big-M constraints with several products** (Chapter 15 «Formulations of capacitated lot sizing»):

$$\begin{aligned} \sum_{p \in P} (st_{pt}\, y_{pt} + vt_{pt}\, x_{pt}) &\leq cap_t, && \forall t \in T, \\ x_{pt} &\leq M_{pt}\, y_{pt}, && \forall p \in P, \ \forall t \in T \end{aligned}$$

**Where it is treated**

- Chapter 9 «Capacitated lot sizing»: Variant of the CLS

- Chapter 17 «Formulations of capacitated lot sizing»: Big-M and shortest-path formulations, strong big-M values
- Chapter 17 «Formulations of capacitated lot sizing»: Instance with two products and three periods

### 8.11.1 Instance MLS1

Two products, three periods, setup times and initial inventory.

2 products, $T = 3$ periods, capacities (time) $cap_1 = 2500$, $cap_2 = 2100$, $cap_3 = 2500$, unit costs of the initial stock 3000, 3000: product 1:

| $t$ | 1 | 2 | 3 |
|---|---|---|---|
| $d_{1t}$ | 100 | 200 | 10 |
| $vt_{1t}$ | 20 | 10 | 10 |
| $st_{1t}$ | 100 | 100 | 100 |
| $f_{1t}$ | 10000 | 10000 | 10000 |
| $p_{1t}$ | 1100 | 1000 | 1000 |
| $h_{1t}$ | 110 | 100 | 100 |

product 2:

| $t$ | 1 | 2 | 3 |
|---|---|---|---|
| $d_{2t}$ | 10 | 100 | 150 |
| $vt_{2t}$ | 10 | 5 | 5 |
| $st_{2t}$ | 110 | 110 | 110 |
| $f_{2t}$ | 10000 | 10000 | 10000 |
| $p_{2t}$ | 900 | 800 | 800 |
| $h_{2t}$ | 105 | 105 | 105 |

- Optimal value $z(\text{MLS1}) = 677400$, attained by $x_{11} = 59$, $x_{12} = 200$, $x_{13} = 10$, $x_{21} = 110$, $x_{22} = 0$, $x_{23} = 150$, $si_1 = 41$; unique.
- Used in Two products.

## 8.12 Lot sizing with initial stock (LS-SI)

**Definition 8.5: Lot sizing with initial stock**

The quantity $si_p$ of product $p$ available at the beginning of the horizon is a decision, bought at the cost $fc_p$ per unit; the balance of the first period becomes $si_p + x_{p1} = d_{p1} + s_{p1}$.

- Defined in Chapter 8 «Capacitated lot sizing».
- Removes the assumption of an empty initial stock of the CLS.

**Where it is treated**

- Chapter 9 «Capacitated lot sizing»: Variant of the CLS

- Chapter 17 «Formulations of capacitated lot sizing»: Formulations, with an arc $w_{pt}$ of cost $ci_{pt}$ in the shortest-path network
- Chapter 17 «Formulations of capacitated lot sizing»: Instance with two products, $fc_1 = fc_2 = 3000$

## 8.13 Lot sizing with overtime (LS-OT)

**Definition 8.6: Lot sizing with overtime**

The capacity is not fixed: working time in normal hours costs $cw_t$ per unit up to $cap_t^N$, and overtime costs $co_t$ per unit up to $cap_t^O$.

- Defined in Chapter 8 «Capacitated lot sizing».
- Replaces the fixed capacity $cap_t$ of the CLS by two levels of capacity with different costs.

### Where it is treated

- Chapter 9 «Capacitated lot sizing»: Variant of the CLS

## 8.14 Lot sizing with unsatisfied demand (LS-UD)

**Definition 8.7: Lot sizing with unsatisfied demand**

A part of the demand may be left unsatisfied, at a penalty $cu_{pt}$ per unit.

- Defined in Chapter 8 «Capacitated lot sizing».
- Relaxes the demand constraints of the CLS, with a penalty in the objective.

### Where it is treated

- Chapter 9 «Capacitated lot sizing»: Variant of the CLS

## Chapter 9

# Knapsack family

## 9.1 Rod-Cutting Problem

**Where it is treated**

- Chapter 16 «Principles of dynamic programming»: Summary table of the DP algorithms: state $v(\ell)$, $\Theta(n^2)$ steps
- Chapter 18 «Rod cutting and coin change»: Definition, encoding of the solutions, two DP algorithms
- Exercise 18.1: Exercises 18.1–18.5: both DP algorithms by hand

### 9.1.1 Definition

**Definition 9.1: Rod-Cutting Problem**

Given:

1. a rod of *length* $n \in \mathbb{Z}_{\geq 1}$,
2. a set $\{1, 2, \ldots, n\}$ of $n$ possible piece lengths,
3. for each piece length $\ell \in \{1, 2, \ldots, n\}$, a *profit* $p_\ell \in \mathbb{Z}_{\geq 1}$,

the **Rod-Cutting Problem** (RCP) asks to decide how many copies of each piece length to cut, so that the total length of the obtained pieces is equal to the rod length, and the total profit of the obtained pieces is maximized.

- Defined in Chapter 18 «Rod cutting and coin change».
- It models the cutting of a stock piece into smaller pieces with prices, each length available any number of times.
- A solution is the integer vector $(x_1, \ldots, x_n)$ of the numbers of pieces, with $\sum_{\ell=1}^{n} \ell\, x_\ell = n$; the $2^{n-1}$ cutting patterns are more than these vectors.
- An unbounded knapsack with one item type per piece length $\ell$, weight $\ell$, profit $p_\ell$ and capacity $n$, where the total length must be exactly $n$.

### 9.1.2 Bounds and complexity

- **Complexity**: Solved in $\Theta(n^2)$ steps by DP, polynomial because the input has $n$ profits.

### 9.1.3 Dynamic programming

- First recurrence on the rod length: $v(\ell) = \max\{p_j + v(\ell - j) : j \in \{1, 2, \ldots, \ell\}\}$ for $1 \le \ell \le n$, with $v(0) = 0$ (Chapter 18 «Rod cutting and coin change»).
- The vector $s$ of the first pieces gives an optimal solution for every rod length $\ell \le n$ at no extra cost (Chapter 18 «Rod cutting and coin change»).
- Second recurrence on pairs (largest piece length, rod length): $v(j, \ell) = \max\{v(j-1, \ell), p_j + v(j, \ell - j)\}$ if $1 \le j \le \ell$, with $v(j, 0) = 0$ and $v(0, \ell) = -\infty$ for $\ell > 0$ (Chapter 18 «Rod cutting and coin change»).
- Both bottom-up algorithms take $S(n) = \Theta(n^2)$ steps, polynomial because the input has $n$ profits (Chapter 18 «Rod cutting and coin change»).

#### Instances

**Instance RCP1** Reference instance of Example 18.1.
Rod of length $n = 4$, prices:

| $i$ | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| $p_i$ | 1 | 5 | 8 | 9 |

- Optimal value $z(\text{RCP1}) = 10$, attained by $(x_1, x_2, x_3, x_4) = (0, 2, 0, 0)$: two pieces of length 2.
- Used in Example 18.1, Example 18.2 (enumeration), Execution of the first DP algorithm, Execution of the second DP algorithm.

**Instance RCP2** Instance of Exercise 18.1.
Rod of length $n = 6$, prices:

| $i$ | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| $p_i$ | 2 | 5 | 7 | 8 | 10 | 13 |

- Optimal value $z(\text{RCP2}) = 15$, attained by $(0, 3, 0, 0, 0, 0)$.
- Used in Exercise 18.1.

**Instance RCP3** Instance of Exercise 18.2.
Rod of length $n = 5$, prices:

| $i$ | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| $p_i$ | 1 | 4 | 6 | 8 | 9 |

- Optimal value $z(\text{RCP3}) = 10$, attained by $(0, 1, 1, 0, 0)$.
- Used in Exercise 18.2.

**Instance RCP4** Instance of Exercise 18.3.
Rod of length $n = 5$, prices:

| $i$ | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| $p_i$ | 3 | 5 | 8 | 9 | 10 |

- Optimal value $z(\text{RCP4}) = 15$, attained by $(5, 0, 0, 0, 0)$.
- Used in Exercise 18.3.

**Instance RCP5** Instance of Exercise 18.4.
Rod of length $n = 6$, prices:

| $i$ | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| $p_i$ | 2 | 6 | 7 | 11 | 13 | 17 |

- Optimal value $z(\text{RCP5}) = 18$, attained by $(0, 3, 0, 0, 0, 0)$.
- Used in Exercise 18.4.

**Instance RCP6** Instance of Exercise 18.5.
Rod of length $n = 6$, prices:

| $i$ | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| $p_i$ | 1 | 5 | 8 | 9 | 12 | 14 |

- Optimal value $z(\text{RCP6}) = 16$, attained by $(0, 0, 2, 0, 0, 0)$.
- Used in Exercise 18.5.

### 9.1.4 Exercises

- Exercise 20.1: Rod Cutting Problem
- Exercise 20.2: Rod Cutting Problem
- Exercise 20.3: Rod Cutting Problem
- Exercise 20.4: Rod Cutting Problem
- Exercise 20.5: Rod Cutting Problem

## 9.2 Coin Change Problem

**Where it is treated**

- Chapter 16 «Principles of dynamic programming»: Summary table of the DP algorithms: state $v(k)$, $\Theta(nc)$ steps
- Chapter 18 «Rod cutting and coin change»: Definition, encoding of the solutions, DP algorithm
- Exercise 18.6: Exercises 18.6–18.10: the DP algorithm by hand, one infeasible instance

### 9.2.1 Definition

**Definition 9.2: Coin Change Problem**

Given:

1. a vector $w \in \mathbb{Z}^n_{\geq 1}$ of $n$ *denominations*, each available in unlimited supply,
2. an integer *amount of change* $c \in \mathbb{Z}_{\geq 0}$ to return,

the **Coin Change Problem** (CCP) asks to decide how many coins of each denomination to use so that their total value is exactly equal to $c$, and the total number of coins used is minimized. If no combination of coins returns exactly $c$, the instance is *infeasible*.

- Defined in Chapter 18 «Rod cutting and coin change».
- It models returning a given amount of change with the fewest coins, each denomination available in unlimited supply.
- A solution is the integer vector $(x_1, \ldots, x_n)$ with $\sum_{j=1}^{n} w_j x_j = c$; an instance can be infeasible, for example when every denomination is even and $c$ is odd.
- Like the UKP it has unlimited copies of every item type, but it asks for exactly $c$ and minimizes the number of coins, while the UKP asks for at most $c$ and maximizes the profit.

### 9.2.2 Bounds and complexity

- **Complexity**: Solved in $\Theta(nc)$ steps by DP, which is pseudo-polynomial.

### 9.2.3 Dynamic programming

- Recurrence on the amount of change: $v(0) = 0$, $v(k) = +\infty$ if $w_j > k$ for every $j$, otherwise $v(k) = \min_{j\colon w_j \leq k}\{1 + v(k - w_j)\}$ (Chapter 18 «Rod cutting and coin change»).
- The array $s[k]$ stores the index of the denomination of the first coin; the reconstruction is done only if $v[c] < +\infty$ (Chapter 18 «Rod cutting and coin change»).
- The bottom-up algorithm takes $S(n, c) = \Theta(nc)$ steps, which is pseudo-polynomial (Chapter 18 «Rod cutting and coin change»).

### Instances

**Instance CCP1** Reference instance of Example 18.15.
Coin values $\{2, 3, 5\}$, amount $S = 10$.

- Optimal value $z(\text{CCP1}) = 2$, attained by $(x_1, x_2, x_3) = (0, 0, 2)$.
- Used in Example 18.15, Execution of the DP algorithm, Reconstruction of an optimal solution.

**Instance CCP2** Instance of Exercise 18.6.
Coin values $\{1, 4\}$, amount $S = 6$.

- Optimal value $z(\text{CCP2}) = 3$, attained by $(2, 1)$.
- Used in Exercise 18.6.

**Instance CCP3** Instance of Exercise 18.7.
Coin values $\{3, 5\}$, amount $S = 11$.

- Optimal value $z(\text{CCP3}) = 3$, attained by $(2, 1)$.
- Used in Exercise 18.7.

**Instance CCP4** Infeasible instance of Exercise 18.8.
Coin values $\{2, 4\}$, amount $S = 7$.

- The instance is infeasible, $z(\text{CCP4}) = +\infty$.
- Used in Exercise 18.8.

**Instance CCP5** Instance of Exercise 18.9.
Coin values $\{3, 7\}$, amount $S = 12$.

- Optimal value $z(\text{CCP5}) = 4$, attained by $(4, 0)$.
- Used in Exercise 18.9.

**Instance CCP6** Instance of Exercise 18.10.
Coin values $\{2, 3, 7\}$, amount $S = 9$.

- Optimal value $z(\text{CCP6}) = 2$, attained by $(1, 0, 1)$.
- Used in Exercise 18.10.

### 9.2.4 Exercises

- Exercise 20.6: Coin Change Problem
- Exercise 20.7: Coin Change Problem
- Exercise 20.8: Coin Change Problem (infeasible instance)
- Exercise 20.9: Coin Change Problem
- Exercise 20.10: Coin Change Problem

## Chapter 10

# DP examples

## 10.1 Minimum-cost staircase

**Where it is treated**

- Exercise 16.1: Exercise 16.1: recurrence, recursion tree, lookup table, reconstruction

### 10.1.1 Definition

**Definition 10.1: Minimum-cost staircase**

A staircase has $n$ steps. Stepping on step $k$ costs $c_k$, and from a step we can climb to the next one or skip one step. We start from the ground, step 0, at no cost, and we want to reach step $n$ at minimum total cost.

- Defined in Exercise 16.1.
- The smallest example of the two properties of DP: the top-down recursion evaluates the same values $v(k)$ many times, the lookup table once.
- A shortest path on the steps $0, 1, \ldots, n$ with arcs $k-1 \to k$ and $k-2 \to k$; used to show the optimal substructure and the overlapping subproblems.

### 10.1.2 Bounds and complexity

- **Complexity**: $\Theta(n)$ steps with the lookup table.

### 10.1.3 Dynamic programming

- $v(k) = c_k + \min\{v(k-1), v(k-2)\}$ for $2 \leq k \leq n$, with $v(1) = c_1$ and $v(0) = 0$; the recursion tree of $v(6)$ has 25 calls (Exercise 16.1).

**Instances**

**Instance STAIR1** Instance of Exercise 16.1.

$n = 6$ steps with costs:

| $k$ | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| $c_k$ | 3 | 1 | 4 | 1 | 5 | 2 |

- Optimal value $z(\text{STAIR1}) = 4$, attained by steps $0 \to 2 \to 4 \to 6$.
- Used in Exercise 16.1.

### 10.1.4 Exercises

- Exercise 18.1: Optimal substructure and overlapping subproblems

## 10.2 Billboards with no two consecutive positions

**Where it is treated**

- Exercise 16.2: Exercise 16.2: recurrence, lookup table, reconstruction, greedy comparison

### 10.2.1 Definition

**Definition 10.2: Billboards with no two consecutive positions**

Along a road there are $n$ positions for billboards, numbered in order. A billboard in position $k$ brings a profit $p_k$, and two billboards cannot be in consecutive positions. We want a set of positions, no two consecutive, of maximum total profit.

- Defined in Exercise 16.2.
- An exercise on writing a recurrence by distinguishing whether the last position is used, and on a greedy choice that is not optimal.
- A maximum weight stable set on a path with $n$ vertices; a subset problem like the KP, with a conflict constraint instead of the capacity.

### 10.2.2 Bounds and complexity

- **Complexity**: $\Theta(n)$ steps with the lookup table.

### 10.2.3 Dynamic programming

- $v(k) = \max\{v(k-1),\ p_k + v(k-2)\}$ for $2 \leq k \leq n$, with $v(1) = p_1$ and $v(0) = 0$ (Exercise 16.2).

### 10.2.4 Constructive heuristics

- The greedy choice by non-increasing profit, skipping the neighbours of a chosen position, gives $20 < 22$ on the instance of the exercise (Exercise 16.2).

**Instances**

**Instance NOCONS1** Instance of Exercise 16.2.

$n = 7$ positions with profits:

| $k$ | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|
| $p_k$ | 5 | 8 | 6 | 3 | 7 | 9 | 4 |

- Optimal value $z(\text{NOCONS1}) = 22$, attained by positions $\{1, 3, 5, 7\}$ (greedy: 20).
- Used in Exercise 16.2.

### 10.2.5 Exercises

- Exercise 18.2: Writing a recurrence: no two consecutive positions

## 10.3 Maximum-profit path on a grid

**Where it is treated**

- Exercise 16.3: Exercise 16.3: counting the paths, recurrence, table, reconstruction

### 10.3.1 Definition

**Definition 10.3: Maximum-profit path on a grid**

A robot moves on a grid of $r$ rows and $q$ columns, from the cell $(1, 1)$ to the cell $(r, q)$, one cell down or one cell right at each move. Every cell $(i, j)$ it visits gives a profit $a_{ij}$. We want a path of maximum total profit.

- Defined in Exercise 16.3.
- An exercise on filling a two-dimensional DP table row by row and reading the optimal path backwards.
- A longest path in an acyclic grid network; the exercise compares the $\binom{r+q-2}{r-1}$ paths with the $rq$ cells of the DP table.

### 10.3.2 Bounds and complexity

- **Complexity**: One cell per subproblem, $rq$ cells, while the number of paths grows exponentially with the size of the grid.

### 10.3.3 Dynamic programming

- $v(i, j) = a_{ij} + \max\{v(i-1, j), v(i, j-1)\}$ for $i, j \geq 2$, with the first row and column accumulated and $v(1, 1) = a_{11}$ (Exercise 16.3).

**Instances**

**Instance GRID1** Instance of Exercise 16.3.
Grid with 3 rows and 4 columns, profits $a_{ij}$:

| | | | |
|---|---|---|---|
| 3 | 1 | 4 | 2 |
| 2 | 6 | 1 | 5 |
| 4 | 2 | 3 | 1 |

- Optimal value $z(\text{GRID1}) = 18$, attained by

$$(1,1) \to (2,1) \to (2,2) \to (2,3) \to (2,4) \to (3,4)$$

.

- Used in Exercise 16.3.

### 10.3.4 Exercises

- Exercise 18.3: Filling a DP table: paths on a grid

## 10.4 Shortest path in a layered network

**Where it is treated**

- Exercise 16.4: Exercise 16.4: recurrence, backward computation, reconstruction
- Chapter 19 «Dynamic programming for lot sizing»: Remark: the recurrence of the ULS computes a shortest path from node 0 to node $t$

### 10.4.1 Definition

**Definition 10.4: Shortest path in a layered network**

We look for a shortest path from $s$ to $t$ in a directed network $G = (N, A)$, where every arc $(i, j) \in A$ has a cost $c_{ij}$.

- Defined in Exercise 16.4.
- The value $v(i)$ of a shortest path from $i$ to $t$ is used by every predecessor of $i$: the overlapping subproblems are the shared nodes.
- The DP algorithm of the uncapacitated lot sizing problem computes a shortest path in a network whose arcs are production runs (Chapter 19 «Dynamic programming for lot sizing»).

### 10.4.2 Dynamic programming

- $v(i) = \min\{c_{ij} + v(j) : (i, j) \in A\}$ for $i \neq t$, with $v(t) = 0$, computed backwards from $t$ (Exercise 16.4).

**Instances**

**Instance SPP1** Layered network of Exercise 16.4.
Network with 8 nodes, from $s$ to $t$; arcs with their costs:

$$(s,a_1)\colon 4,\ (s,a_2)\colon 2,\ (s,a_3)\colon 5,\ (a_1,b_1)\colon 3,\ (a_1,b_2)\colon 6,$$
$$(a_2,b_1)\colon 7,\ (a_2,b_2)\colon 4,\ (a_2,b_3)\colon 6,\ (a_3,b_2)\colon 2,\ (a_3,b_3)\colon 3,$$
$$(b_1,t)\colon 6,\ (b_2,t)\colon 3,\ (b_3,t)\colon 2.$$

- Optimal value $z(\text{SPP1}) = 9$, attained by $s \to a_2 \to b_2 \to t$ (the network has 7 paths from $s$ to $t$).
- Used in Exercise 16.4.

### 10.4.3 Exercises

- Exercise 18.4: Shortest path in a layered network

## 10.5 Compatible requests

**Where it is treated**

- Exercise 16.5: Exercise 16.5: indices $q(j)$, recurrence, table, reconstruction, greedy comparison

### 10.5.1 Definition

**Definition 10.5: Compatible requests**

A room can host one event at a time. There are $n$ requests: request $j$ occupies the room from time $a_j$ to time $b_j$ and pays $w_j$; two requests are compatible if one ends before (or when) the other starts. The requests are numbered by non-decreasing end time. We want a set of pairwise compatible requests of maximum total payment.

- Defined in Exercise 16.5.
- An exercise on designing a recurrence for a new problem, with the index $q(j)$ of the last request compatible with $j$.
- A single-machine selection problem: a subset of jobs (intervals) of maximum weight in which no two overlap.

### 10.5.2 Dynamic programming

- $v(j) = \max\{v(j-1),\ w_j + v(q(j))\}$ for $1 \le j \le n$, with $v(0) = 0$, where $q(j)$ is the largest $i < j$ compatible with $j$ (Exercise 16.5).

### 10.5.3 Constructive heuristics

- The greedy choice by payment gives only 8 against the optimal value 12 on the instance of the exercise (Exercise 16.5).

### Instances

**Instance REQ1** Instance of Exercise 16.5.
6 requests (start $a_j$, end $b_j$, payment $w_j$):

| $j$ | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| $a_j$ | 0 | 1 | 4 | 2 | 5 | 7 |
| $b_j$ | 3 | 5 | 6 | 8 | 9 | 10 |
| $w_j$ | 4 | 6 | 3 | 8 | 5 | 5 |

- Optimal value $z(\text{REQ1}) = 12$, attained by requests $\{1, 3, 6\}$ (greedy by payment: 8).
- Used in Exercise 16.5.

## 10.5.4 Exercises

- Exercise 18.5: A recurrence for a new problem: compatible requests